

Week6~8 算

Updated 2026-04-20 23:37 GMT+8

Compiled by Hongfei Yan (2025 Spring)

说明：

为了让大家对  有更完整的认识，将原本作为附件的内容，包括线段树（Segment Tree）、字典树（Trie）和KD树（KD Tree），全部整合到正文中进行讲解。这些都属于拓展性的超纲内容，旨在帮助大家构建更全面的知识体系。

课件中编程题目，“示例。。。 ”是学习了概念，顺便看懂示例程序。“练习。。。 ”是希望先不看答案，自己尝试完成。

与树相关的题目，还挺难的，因为通常涉及到把输入字符串构建成一棵树，这个过程比较麻烦，通常用到stack+dfs。然后再按照题面要求输出。

1) 第一讲：概念与表示法；第二讲：构建、遍历、哈夫曼算法；第三讲：堆、BST、AVL、并查集。

2) md文件有目录，内容推进思路是原理学习+编程题目实践。

前言：树形结构学习方法

在计算机科学领域内，树形结构是最为关键同时也是掌握难度较高的概念之一。不同于计算概论，数据结构与算法的学习不仅需要借助面向对象编程的抽象和复用理念，还要通过 **使用链表来表达树结构，并采用递归的方法进行树的遍历**。在这一过程中，经典算法和编程技巧的应用至关重要，而对于时间复杂度的优化技巧也是必不可少的。

同学可能会发现树结构的学习颇具挑战。一些学生可能会通过类比和综合的方式来促进理解，比如通过深度优先搜索（DFS）来深入理解树的前序遍历和后序遍历；有的学生可能会通过吟诵诗歌来激励自己；还有的学生则投入大量时间复现经典算法，或在集成开发环境（IDE）中探索如何调试递归程序。

教材中不免存在一些内容遗漏或逻辑不连贯的部分，因此我们制作的课件也在持续更新以补充和完善这些内容。

预计需要至少三周掌握树结构的相关知识。

《Python数据结构与算法分析》这本书，树这一章没有讲generic tree，直接就到binary tree了。之前感觉它的 算法分析 那章不好，现在感觉树这章也差强人意，写的比较随意。树有个重要的应用，文本压缩——哈夫曼编码。在该书中，搜不到“哈夫曼”，也搜不到"Huffman"。

Q: 有没有直接的理解，就是比如给我一棵树，我可以直接用遍历方法给出中序和后序？

前序遍历就是dfs序列，后序的反序是不是就是原树优先走右边的dfs。

你的问题涉及到树的遍历方法以及它们之间的关系。我们来逐一分析：

1. 树的遍历方法

对于一棵树（尤其是二叉树），常见的遍历方法有三种：

- **前序遍历**（Preorder）：根 -> 左子树 -> 右子树
- **中序遍历**（Inorder）：左子树 -> 根 -> 右子树
- **后序遍历**（Postorder）：左子树 -> 右子树 -> 根

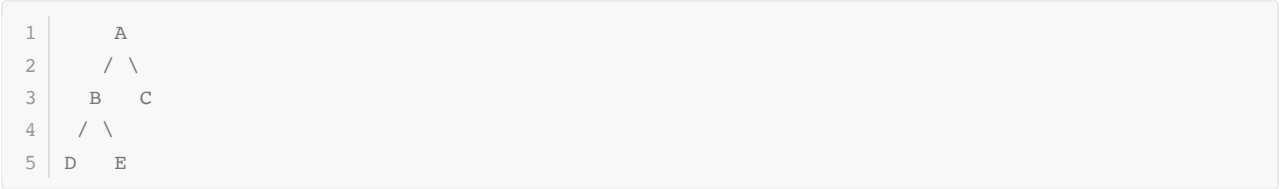
这些遍历方法可以通过递归或迭代实现，本质上是深度优先搜索（DFS）的不同变体。

2. 前序遍历和DFS的关系

你提到“前序遍历就是DFS序列”，这是正确的！

- 前序遍历的顺序正是标准的深度优先搜索（DFS）访问节点的顺序。
- 在DFS过程中，第一次访问某个节点时就记录它，这就是前序遍历的结果。

例如，给定以下二叉树：



前序遍历结果是：A -> B -> D -> E -> C

这正是DFS从根节点开始访问的顺序。

3. 后序遍历的反序与DFS的关系

你提到“后序的反序是不是就是原树优先走右边的DFS”，这个说法在某些情况下是对的，但需要具体分析。

后序遍历的特点

后序遍历的顺序是：先访问左子树，再访问右子树，最后访问根节点。

以上面的树为例，后序遍历结果是：D -> E -> B -> C -> A

后序遍历的反序

将后序遍历结果反转，得到：A -> C -> B -> E -> D

优先走右边的DFS

如果我们在DFS过程中优先访问右子树（即先访问右子树再访问左子树），那么得到的访问顺序是：A -> C -> B -> E -> D

可以看到，后序遍历的反序确实等于优先走右边的DFS的顺序。

4. 中序遍历

中序遍历的顺序是：左子树 -> 根 -> 右子树。

以上面的树为例，中序遍历结果是：D -> B -> E -> A -> C

中序遍历没有直接等价于某种DFS顺序，因为它的顺序依赖于左子树、根、右子树的特定排列。

总结

- 前序遍历 = 标准DFS访问顺序。
- 后序遍历的反序 = 优先走右边的DFS访问顺序。
- 中序遍历 = 左子树 -> 根 -> 右子树，没有直接等价于某种DFS顺序。

【往年同学作业总结】：

树状数组惨遭内存超出，现学分治依旧遥遥领先。
合法出栈序列简单模拟，树节无树求二叉树深度。
双端队列还是宝宝巴士，波兰表达式仍游刃有余。
中序转后序写了两小时，水平不够还得多家练习！

本周算是真正进入了数算的重点章节，难度肉眼可见地增长。百字令一首，聊表感受。

树

不熟

真可恶

无能狂怒

找大佬求助

WA百般苦楚

心茫然拔剑四顾

栈与队列前后弹出

好在学东西可以顿悟

各种序表达式需要关注

逐渐解决了本周题目

尤其写栈有如神助

但没到精通程度

递归循环次数

AC趣味足

乐不思蜀

刚上路

刻苦

赴

一、概念和表示法

0 Recap

0.1 递归 & BFS

鉴于各班在《计算概论B》课程中的进度不一，各位老师对于《计算概论》和《数据结构与算法》内容的覆盖和理解也存在差异，我们建议复习递归和BFS的相关内容。

1) **递归是数算中不可或缺的核心技能**，强烈推荐优先掌握。可以参考 https://github.com/GMyhf/2025fall-cs101/blob/main/20251028_ADS_week08-09_recursion_backtracking.md

2) **队列在广度优先搜索（BFS）中有着广泛的应用**。考虑到其他班级可能尚未讲授搜索算法，我们提供学习资源，https://github.com/GMyhf/2025fall-cs101/blob/main/20251125_ADS_week12_searching.md

0.2用OOP写程序

接下来开始学习树，有时程序会给出类的实现代码。复习类写法，可以debug模式运行，或者 <https://pythontutor.com> 可视化运行，辅助理解。类实现程序，补充缺失行代码。

OOP 的写法属于语法范畴，可参看 <https://www.runoob.com/python3/python3-class.html>

数算的精华是复制（类的精华是复制），不需要深拷贝，只要创建多个对象，就有各自的内存空间。例如下面并查集的例子，创建了A, B两个对象，A的修改不影响B的。并查集的内容在 三.9部分。

```
1 class DisjSet:
2     def __init__(self, n):
3         # Constructor to create and
4         # initialize sets of n items
5         self.rank = [1] * n
6         self.parent = [i for i in range(n)]
7
8     def find(self, x):
9         # Find the root of the set in which element x belongs
10        if self.parent[x] != x:
11            # Path compression: Make the parent of x the root of its set
12            self.parent[x] = self.find(self.parent[x])
13        return self.parent[x]
14
15    def union(self, x, y):
16        # Perform union of two sets
17        x_root = self.find(x)
18        y_root = self.find(y)
19
20        if x_root == y_root:
21            return
22
23        # Attach smaller rank tree under root of higher rank tree
24        if self.rank[x_root] < self.rank[y_root]:
25            self.parent[x_root] = y_root
26        elif self.rank[x_root] > self.rank[y_root]:
27            self.parent[y_root] = x_root
28        else:
29            self.parent[y_root] = x_root
30            self.rank[x_root] += 1
31
32
33 if __name__ == '__main__':
34     A = DisjSet(5)
35     B = DisjSet(5)
36
37     A.union(0, 1)
38     A.union(2, 3)
39
40     print(A.rank) # Output: [2, 1, 2, 1, 1]
41     print(A.parent) # Output: [0, 0, 2, 2, 4]
42     print(B.rank) # Output: [1, 1, 1, 1, 1]
43     print(B.parent) # Output: [0, 1, 2, 3, 4]
44
```

练习M2069.模拟行走机器人 II

OOP, implementation, <https://leetcode.cn/problems/walking-robot-simulation-ii/>

给你一个在 XY 平面上的 `width x height` 的网格图，左下角的格子为 `(0, 0)`，右上角的格子为 `(width - 1, height - 1)`。网格图中相邻格子为四个基本方向之一（"North"，"East"，"South" 和 "West"）。一个机器人初始在格子 `(0, 0)`，方向为 "East"。

机器人可以根据指令移动指定的 步数。每一步，它可以执行以下操作。

1. 沿着当前方向尝试 往前一步。
2. 如果机器人下一步将到达的格子 超出了边界，机器人会 逆时针 转 90 度，然后再尝试往前一步。

如果机器人完成了指令要求的移动步数，它将停止移动并等待下一个指令。

请你实现 `Robot` 类：

- `Robot(int width, int height)` 初始化一个 `width x height` 的网格图，机器人初始在 `(0, 0)`，方向朝 `"East"`。
- `void step(int num)` 给机器人下达前进 `num` 步的指令。
- `int[] getPos()` 返回机器人当前所处的格子位置，用一个长度为 2 的数组 `[x, y]` 表示。
- `String getDir()` 返回当前机器人的朝向，为 `"North"`，`"East"`，`"South"` 或者 `"West"`。

示例 1：



```
1  输入：
2  ["Robot", "step", "step", "getPos", "getDir", "step", "step", "step", "getPos", "getDir"]
3  [[6, 3], [2], [2], [], [], [2], [1], [4], [], []]
4  输出：
5  [null, null, null, [4, 0], "East", null, null, null, [1, 2], "West"]
6
7  解释：
8  Robot robot = new Robot(6, 3); // 初始化网格图，机器人在 (0, 0)，朝东。
9  robot.step(2); // 机器人朝东移动 2 步，到达 (2, 0)，并朝东。
10 robot.step(2); // 机器人朝东移动 2 步，到达 (4, 0)，并朝东。
11 robot.getPos(); // 返回 [4, 0]
12 robot.getDir(); // 返回 "East"
13 robot.step(2); // 朝东移动 1 步到达 (5, 0)，并朝东。
14                  // 下一步继续往东移动将出界，所以逆时针转变方向朝北。
15                  // 然后，往北移动 1 步到达 (5, 1)，并朝北。
16 robot.step(1); // 朝北移动 1 步到达 (5, 2)，并朝 北 （不是朝西）。
17 robot.step(4); // 下一步继续往北移动将出界，所以逆时针转变方向朝西。
18                  // 然后，移动 4 步到 (1, 2)，并朝西。
19 robot.getPos(); // 返回 [1, 2]
20 robot.getDir(); // 返回 "West"
```

提示：

- `2 <= width, height <= 100`
- `1 <= num <= 10^5`
- `step` , `getPos` 和 `getDir` 总共 调用次数不超过 `10^4` 次。

这道题的核心在于观察机器人的移动轨迹。机器人实际上是在网格的边界（外圈）上循环移动。

解题思路

1. 周长与循环：机器人只会在网格的四条边上移动。总的周长 $C = 2 \times (width - 1) + 2 \times (height - 1)$ 。由于机器人一直在边界循环，我们可以对步数 num 取模。
2. 特殊情况 **((0,0)点)** :
 - 初始状态：位置 $(0, 0)$ ，方向 "East"。
 - 移动一圈回到 $(0, 0)$ 后：位置 $(0, 0)$ ，由于是由于从南边边界顶到头转弯回来的，方向会变为 "South"。
 - 为了处理这个逻辑，如果移动步数取模后为 0，且已经发生过移动，我们需要判断它是在哪个位置。如果回到了起点，方向应设为 "South"。
3. 轨迹建模：我们可以预先计算出边界上所有点的坐标和对应的方向，或者根据步数 num 所落在的区间来计算：
 - `0 < num <= width - 1`：在底边，向东。
 - `width - 1 < num <= (width-1) + (height-1)`：在右边，向北。
 - `(width-1) + (height-1) < num <= 2*(width-1) + (height-1)`：在顶边，向西。
 - 其他：在左边，向南。

代码实现

```
1 class Robot:
2
3     def __init__(self, width: int, height: int):
4         self.w = width
5         self.h = height
6         self.pos = 0 # 用相对于(0,0)沿边界走的步数来记录位置
7         self.perimeter = 2 * (width - 1) + 2 * (height - 1)
8         self.moved = False
9
10    def step(self, num: int) -> None:
11        self.moved = True
12        # 取模减少无效循环
13        self.pos = (self.pos + num) % self.perimeter
14
15    def getPos(self) -> List[int]:
16        p = self.pos
17        w, h = self.w, self.h
18        # 底部边: [0, w-1]
19        if p < w:
20            return [p, 0]
21        # 右侧边: [w, w + h - 2] (减去重复的角点)
22        if p < w + h - 1:
23            return [w - 1, p - (w - 1)]
24        # 顶部边
25        if p < 2 * w + h - 2:
26            return [w - 1 - (p - (w + h - 2)), h - 1]
27        # 左侧边
28        return [0, h - 1 - (p - (2 * w + h - 3))]
```

```

29
30     def getDir(self) -> str:
31         # 特殊处理：回到起点的情况
32         if self.moved and self.pos == 0:
33             return "South"
34
35         p = self.pos
36         w, h = self.w, self.h
37         # 初始状态且未移动
38         if not self.moved:
39             return "East"
40
41         # 根据当前步数所处的范围判断方向
42         # 注意：边界点在转向后，方向会立即变为新边的方向
43         if 0 < p <= w - 1:
44             return "East"
45         elif w - 1 < p <= w + h - 2:
46             return "North"
47         elif w + h - 2 < p <= 2 * w + h - 3:
48             return "West"
49         else:
50             return "South"
51

```

关键点解释

- `self.moved` 标志位：这是为了区分“从未移动过的 $(0,0)$ ”和“走了一圈回到 $(0,0)$ ”。
- 方向判定范围：
 - 当机器人走到拐角点（例如 $(width - 1, 0)$ ）时，如果它是动作的终点，它的方向取决于它是如何到达这里的。如果步数正好落在 `w-1`，它依然是 "East"；如果超过了 `w-1`，它就会转向。
 - 取模运算 `num % perimeter` 会导致原本回到原点应为 "South" 的情况变成 0。代码中通过 `if self.pos == 0 and self.moved` 专门捕获了这一逻辑。

Q: 周长为什么不是 $2 * (width + height)$?

这是一个非常经典且容易混淆的“植树问题”（或称为点与间距的问题）。

在网格坐标系中，周长指的是机器人可以移动的总步数。让我们通过一个具体的例子来直观理解。

假设网格是 `width = 4, height = 3` :

坐标范围是 $x \in [0, 3], y \in [0, 2]$ 。

1. 手动数一下边界上的格子

机器人沿着边界走一圈经过的路径如下：

1. 底边：从 $(0,0)$ 到 $(3,0)$ ，移动了 3 步。
2. 右边：从 $(3,0)$ 到 $(3,2)$ ，移动了 2 步。
3. 顶边：从 $(3,2)$ 到 $(0,2)$ ，移动了 3 步。
4. 左边：从 $(0,2)$ 到 $(0,0)$ ，移动了 2 步。

总步数 = $3 + 2 + 3 + 2 = 10$ 步。

2. 公式计算

如果我们使用 $2 \times (width + height)$:
 $2 \times (4 + 3) = 14$ —— 错误。

如果我们使用 $2 \times (width - 1) + 2 \times (height - 1)$:
 $2 \times (4 - 1) + 2 \times (3 - 1) = 2 \times 3 + 2 \times 2 = 10$ —— 正确。

3. 为什么是 $width - 1$?

因为周长计算的是相邻格子之间的间隙（步数），而不是格子的总数。

- 横向有 `width` 个格子，但只有 `width - 1` 段距离。
- 纵向有 `height` 个格子，但只有 `height - 1` 段距离。

4. 形象理解：四个顶点

如果你直接用 $2 \times width + 2 \times height$ ，你会发现四个角上的点被重复计算了两次。

为了修正，你需要减去重复算的 4 个角：

$$2 \times width + 2 \times height - 4$$

这等同于：

$$2 \times (width - 1) + 2 \times (height - 1)$$

总结

在 XY 坐标系网格中，宽度为 W 高度为 H 的矩形轨迹，其外围周长（总步数）恒等于：

$$\text{Perimeter} = 2 \times (W - 1) + 2 \times (H - 1)$$

1 术语Notations

Q. 请讲解数据结构中的树

数据结构中的树

树（Tree）是一种重要的非线性数据结构，它由节点（Node）组成，节点之间具有层级关系。树结构广泛应用于计算机科学领域，如文件系统、数据库索引、搜索算法等。

1. 树的基本概念

(1) 节点的组成

每个节点通常包含：

- 值（Value）：存储的数据。
- 子节点（Children）：指向其他节点的指针。
- 父节点（Parent）：指向其上级节点的指针（根节点除外）。
- 度（Degree）：节点的子节点数量。
- 深度（Depth）：从根节点到当前节点的路径长度。
- 高度（Height）：从当前节点到叶子节点的最长路径长度。

(2) 树的术语

- 根节点（Root）：树的起始节点，没有父节点。
- 叶子节点（Leaf）：没有子节点的节点。

- **子树 (Subtree)**：以某个节点为根的部分树。
- **层级 (Level)**：从根节点开始，根为第 0 层，根的子节点为第 1 层，以此类推。
- **节点间关系**：父节点、子节点、兄弟节点、祖先节点、后代节点等。

2. 树的分类

(1) 普通树

普通树的每个节点可以有任意多个子节点。

(2) 二叉树 (Binary Tree)

二叉树是树的一种特殊形式，每个节点最多有**两个子节点**：

- **满二叉树 (Full Binary Tree)**：所有非叶子节点都有两个子节点。
- **完全二叉树 (Complete Binary Tree)**：只有最后一层可以不满，并且节点从左到右排列。
- **平衡二叉树 (Balanced Binary Tree)**：左右子树的高度差不超过 1，如 AVL 树。
- **二叉搜索树 (Binary Search Tree, BST)**：对于任意节点，左子树的所有节点值小于该节点值，右子树的所有节点值大于该节点值。

(3) 特殊树

- **平衡搜索树 (Balanced Search Tree)**：如 AVL 树、红黑树 (Red-Black Tree)。
- **B 树 (B-Tree)** 和 **B+ 树 (B+ Tree)**：用于数据库索引。
- **Trie 树 (前缀树)**：用于字符串存储和搜索。例如：**OJ的按照标题搜索功能**
- **堆 (Heap)**：如最大堆 (Max Heap)、最小堆 (Min Heap)，用于优先队列。

3. 树的基本操作

树的常见操作包括：

1. **插入 (Insertion)**
2. **删除 (Deletion)**
3. **查找 (Search)**
4. **遍历 (Traversal)**：
 - **深度优先遍历 (DFS)**：
 - **前序遍历 (Preorder)**：根 → 左 → 右
 - **中序遍历 (Inorder)**：左 → 根 → 右 (BST 中序遍历结果为有序数组)
 - **后序遍历 (Postorder)**：左 → 右 → 根
 - **广度优先遍历 (BFS)** (层序遍历)：按层从左到右依次遍历。

4. 树的应用

- **文件系统** (目录结构)
- **数据库索引** (B 树、B+ 树)
- **编译器** (语法树、抽象语法树 AST)
- **网络路由** (Trie 树)
- **人工智能** (搜索树、决策树)

- 数据压缩 (Huffman 树)

5. 树的 Python 实现

(1) 二叉树的基本实现

```
1 class TreeNode:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6
7 # 创建二叉树
8 root = TreeNode(1)
9 root.left = TreeNode(2)
10 root.right = TreeNode(3)
11 root.left.left = TreeNode(4)
12 root.left.right = TreeNode(5)
```

(2) 二叉树的遍历

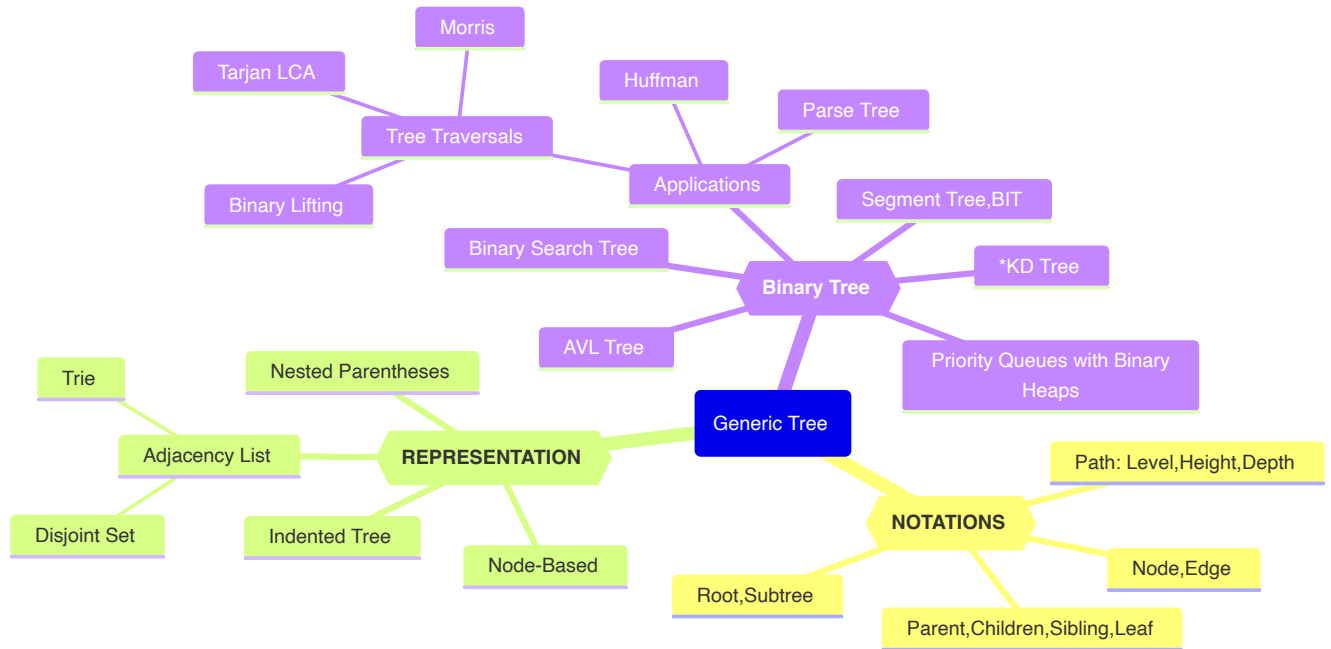
```
1 # 前序遍历
2 def preorder_traversal(node):
3     if node:
4         print(node.value, end=" ")
5         preorder_traversal(node.left)
6         preorder_traversal(node.right)
7
8 # 中序遍历
9 def inorder_traversal(node):
10     if node:
11         inorder_traversal(node.left)
12         print(node.value, end=" ")
13         inorder_traversal(node.right)
14
15 # 后序遍历
16 def postorder_traversal(node):
17     if node:
18         postorder_traversal(node.left)
19         postorder_traversal(node.right)
20         print(node.value, end=" ")
21
22 print("前序遍历:")
23 preorder_traversal(root)
24 print("\n中序遍历:")
25 inorder_traversal(root)
26 print("\n后序遍历:")
27 postorder_traversal(root)
```

总结

- 树是一种层级结构的数据结构，适用于多种应用场景。
- 二叉树是最常见的树类型，具有多种变体（如 BST、AVL 树、红黑树）。
- 树的遍历方式主要包括 DFS（前序、中序、后序）和 BFS（层序遍历）。

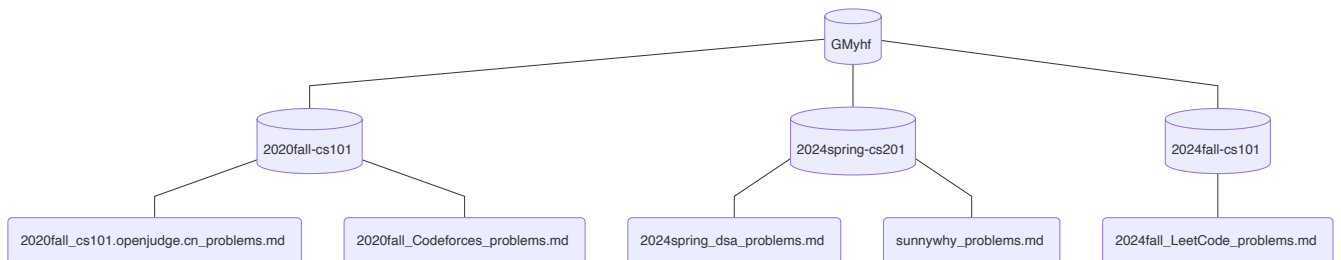
- 树在数据库、编译器、搜索算法等领域有广泛应用。

树算覆盖的内容和知识点总结如下：树的知识图谱。



树的知识图谱

🌲 的示例，树适合表示层级结构，例如：我的github中数算题解的分布, <https://github.com/GMyhf>



在看了树的例子之后，现在来正式地定义树及其构成。

节点 (node): 节点是树的基础部分。

每个节点具有名称，或“键值”。节点还可以保存额外数据项，数据项根据不同的应用而变。

边 (edge): 边是组成树的另一个基础部分。

每条边恰好连接两个节点，表示节点之间具有关联，边具有出入方向；

每个节点（除根节点）恰有一条来自另一节点的入边；

每个节点可以有零条/一条/多条连到其它节点的出边。如果加限制不能有“多条边”，这里树结构就特殊化为线性表

根 (root): 树中唯一没有入边的节点。

路径 (path): 由边依次连接在一起的有序节点列表。比如，哺乳纲→食肉目→猫科→猫属→家猫就是一条路径。

祖先 (ancestor): 一个节点到根节点的路径上，除了它本身外的节点。根节点的祖先集合为空。

父节点 (parent): 一个节点是其所有出边连接节点的父节点。根节点没有父节点。

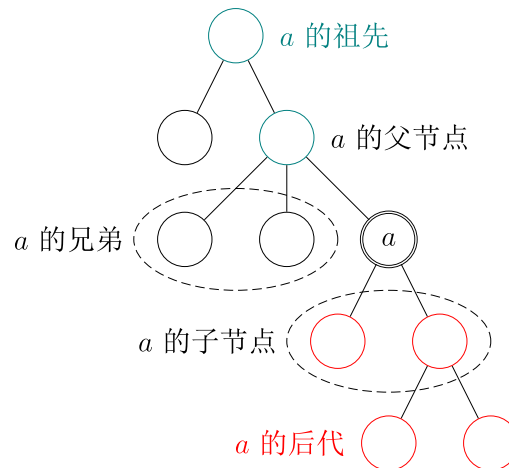
子节点 (children): 入边均来自于同一个节点的若干节点，称为这个节点的子节点。

节点的深度 (depth): 到根节点的路径上的边数。

树的高度 (height): 所有节点的深度的最大值。

兄弟 (sibling): 同一个父亲的多个子节点互为兄弟。

后代 (descendant): 子节点和子节点的后代。



子树 (subtree): 一个节点和其所有子孙节点，以及相关边的集合。

叶节点 (leaf): 没有子节点的节点称为叶节点。

层级 (level):

从根节点开始到达一个节点的路径，**所包含的边的数量**，称为这个节点的层级。

下图中 D 的层级为 2，根节点的层级为 0。

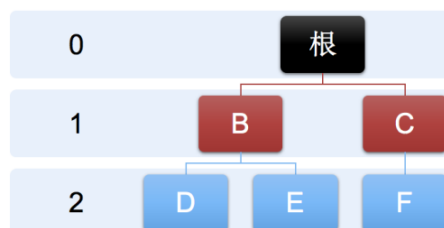


图 树的层级显示

高度 (height): 树中所有节点的最大层级称为树的高度，如图1所示树的高度为 2。

对于只有一个节点的树来说，高度为0，深度为0。如果是空树，高度、深度都是 -1。

这是合理的定义方式，但需要⚠️：

高度：通常定义为从根节点到最远叶子节点的边数。对于空树，高度为 -1 是一种常见的约定（**但也有人定义为空树的高度为 0**）。

深度 (depth)：通常是指从根节点到某个节点的边数。对于空树，深度没有意义，也可以定义为 -1。

1 教材《Python数据结构与算法分析（第2版）》第六章

层级 Level：从根节点开始到达一个节点的路径，所包含的边的数量，称为这个节点的层级。根节点的层级为 0。

高度 Height：树中所有节点的最大层级称为树的高度。因此空树的高度是-1。

2 Tree (graph theory)

[https://en.wikipedia.org/wiki/Tree_\(graph_theory\)#:~:text=The%20height%20of%20a%20vertex,its%20root%20\(root%20path\).](https://en.wikipedia.org/wiki/Tree_(graph_theory)#:~:text=The%20height%20of%20a%20vertex,its%20root%20(root%20path).)

The *height* of a vertex in a rooted tree is the length of the longest downward path to a leaf from that vertex. The *height* of the tree is the height of the root. The *depth* of a vertex is the length of the path to its root (*root path*). This is commonly needed in the manipulation of the various self-balancing trees, AVL trees in particular. The root has depth zero, leaves have height zero, and a tree with only a single vertex (hence both a root and leaf) has depth and height zero.

Conventionally, an empty tree (a tree with no vertices, if such are allowed) has depth and height -1 .

3 2013-book-DataStructuresAndAlgorithmsInPython, page 308, Chapter 8. Trees

Let p be the position of a node of a tree T . The depth of p is the number of ancestors of p , excluding p itself. Note that this definition implies that the depth of the root of T is 0. The depth of p can also be recursively defined as follows:

- If p is the root, then the depth of p is 0.
- Otherwise, the depth of p is one plus the depth of the parent of p

```
1 def depth(self, p):
2     # Return the number of levels separating Position p from the root.
3     if self.is_root(p):
4         return 0
5     else:
6         return 1 + self.depth(self.parent(p))
7
```

Height and Depth of a node in a Binary Tree, <https://www.geeksforgeeks.org/height-and-depth-of-a-node-in-a-binary-tree/> The depth of a node is the number of edges present in path from the root node of a tree to that node. The height of a node is the number of edges present in the longest path connecting that node to a leaf node.

根据前四个参考出处，高度、深度，都是数边的个数。

4 ⚠️ 有的题目中定义：与上面常见的深度定义不一致。例如：

06646: 二叉树的深度，<http://cs101.openjudge.cn/practice/06646/>

二叉树深度：从根结点到叶结点依次经过的结点（含根、叶结点）形成树的一条路径，最长路径的节点个数为树的深度。

M104. 二叉树的最大深度，<https://leetcode.cn/problems/maximum-depth-of-binary-tree/>

给定一个二叉树 `root`，返回其最大深度。

二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

树 Tree:

定义基本术语后，就可以进一步给出树的正式定义。将提供两种定义，其中一种涉及节点和边，另一种涉及递归。在后面会看到，递归定义很有用。

定义一：树由节点及连接节点的边构成。树有以下属性：

- ❑ 有一个根节点；
- ❑ 除根节点外，其他每个节点都与其唯一的父节点相连；
- ❑ 从根节点到其他每个节点都有且仅有一条路径；
- ❑ 如果每个节点最多有两个子节点，我们就称这样的树为二叉树。

图2展示了一棵符合定义一的树。边的箭头表示连接方向。

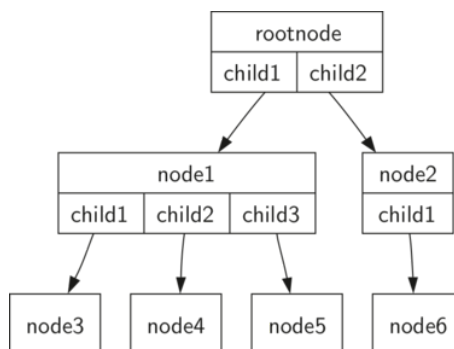


图2 由节点和边构成的树

定义二：一棵树要么为空，要么由一个根节点和零棵或多棵子树构成，子树本身也是一棵树。每棵子树的根节点通过一条边连到父树的根节点。图3展示了树的递归定义。从树的递归定义可知，图中的树至少有4个节点，因为三角形代表的子树必定有一个根节点。这棵树或许有更多的节点，但必须更深入地查看子树后才能确定。

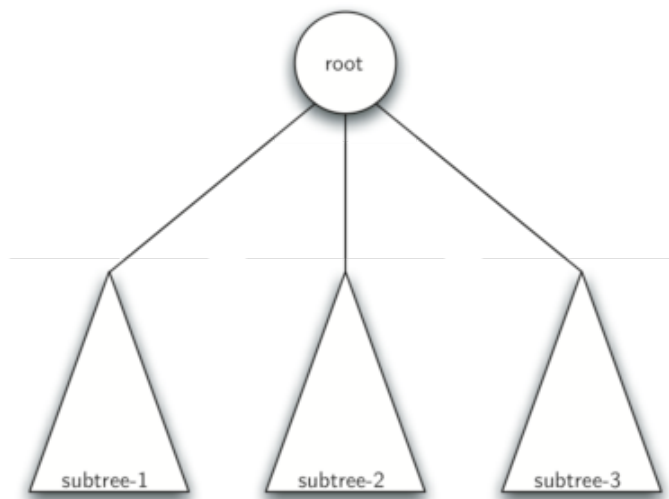


图3 树的递归定义

Q: 树用递归实现才简洁易懂吧？

A: 是的，递归实现通常更简洁易懂，特别是在处理树这种递归结构时。递归的思想与树的结构自然契合，因为树的定义本身就是递归的：树由根节点和若干子树组成，每个子树也可以看作是一棵树。

递归实现的优点包括：

1. 自然而直观：递归实现能够直接模拟树的结构，更符合我们对树的直觉认知，使代码更易理解和编写。

2. 代码简洁：递归实现通常比迭代实现代码更为简洁，因为递归能够利用函数的自身调用来处理子树。
3. 逻辑清晰：递归实现能够清晰地表达树的遍历和处理逻辑，使代码更易读。

然而，递归实现也有一些潜在的缺点，例如在处理大规模的树时可能导致栈溢出（Stack Overflow）的问题。此外，递归实现有时可能会导致性能较差，因为在递归过程中可能会进行重复的计算。

Q: 树的规模较小，较大，具体大概指多少个节点呢？

A: 一般来说，树的规模较小和较大是相对的概念，具体的节点数量取决于问题的上下文和实际需求。以下是一些常见的节点数量范围，尽管这只是一种常见的划分，并不是严格的界定：

1. 小型树：通常指节点数量在几十到几百个之间的树。在这种情况下，递归实现往往是简洁易懂的选择，并且性能方面的差异可能并不明显。
2. 中型树：通常指节点数量在几百到几千个之间的树。对于中型树，递归实现仍然可以考虑使用，但在性能方面可能需要更加谨慎，以避免潜在的栈溢出问题。在这种情况下，迭代实现可能是更好的选择。
3. 大型树：通常指节点数量在几千到几百万（甚至更多）之间的树。对于大型树，递归实现可能会面临栈溢出的风险，并且性能方面的差异可能会更加明显。在这种情况下，迭代实现通常是更可靠和高效的选择。

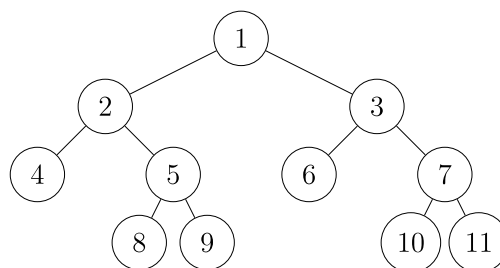
森林 (forest)：每个连通分量（连通块）都是树的图。按照定义，一棵树也是森林。

生成树 (spanning tree)：一个连通无向图的生成子图，同时要求是树。也即在图的边集中选择 $n-1$ 条，将所有顶点连通。

特殊的树

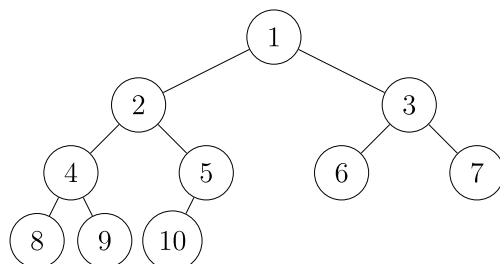
<https://oi-wiki.org/graph/tree-basic/>

- **二叉树 (binary tree)**：每个结点最多只有两个儿子（子节点）的有根树称为二叉树。常常对两个子节点的顺序加以区分，分别称之为左子节点和右子节点。
- **完整二叉树 (full/proper binary tree)**：每个结点的子节点数量均为 0 或者 2 的二叉树。换言之，每个节点或者是树叶，或者左右子树均非空。



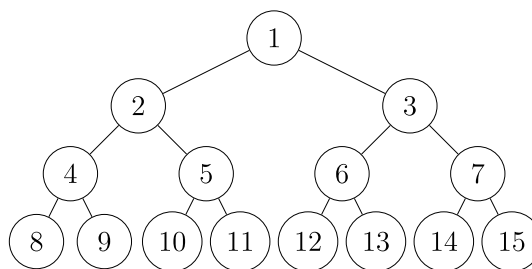
完整二叉树 (proper binary tree)

- **完全二叉树 (complete binary tree)**：只有最下面两层节点的度数可以小于 2，且最下面一层的节点都集中在该层最左边的连续位置上。



完全二叉树 (complete binary tree)

- **完美二叉树 (perfect binary tree)**：所有叶节点的深度均相同，且所有非叶节点的子节点数量均为 2 的二叉树称为完美二叉树。



完美二叉树 (perfect binary tree)

⚠：Proper binary tree 的汉译名称不固定，且完全二叉树和满二叉树的定义在不同教材中定义不同，遇到的时候需根据上下文加以判断。

所说的「满二叉树」多指完美二叉树。

Application of Tree Data Structure

树数据结构的应用

- **文件系统**：这使得文件的导航和组织更加高效。
- **数据压缩**：**哈夫曼编码 (Huffman coding)** 是一种流行的数据压缩技术，它通过构建一棵二叉树来实现，其中叶子节点表示字符及其出现频率。生成的树用于以最小化存储需求的方式对数据进行编码。
- **编译器设计**：在编译器设计中，**抽象语法树 (Abstract Syntax Tree)** 用于表示程序的结构。
- **数据库索引**：B 树和其他树结构被用于数据库索引，以便高效地搜索和检索数据。

树数据结构的优点

- 树提供**高效的搜索**，具体效率取决于树的类型。例如，像 AVL 树这样的平衡树，其平均搜索时间为 $O(\log n)$ 。
- 树以分层方式表示数据，使大量信息的**组织和导航变得简单**。
- 树的**递归特性使其能够通过递归算法轻松遍历和操作**。

树数据结构的缺点

- 不平衡的树会导致树的高度偏向一侧，从而可能引发**低效的搜索时间**。
- 树需要**更多的内存空间**，相比数组和链表等其他数据结构，尤其是在树非常大的情况下。

1.1 n阶多叉树 (N-ary Trees)

普通树 (Generic trees) 是由若干节点组成的集合，其中每个节点是一个数据结构，包含记录和一个指向其子节点的引用列表（不允许重复引用）。与链表不同，每个节点存储了多个节点的地址。**每个节点存储其子节点的地址**，而第一个节点的地址则存储在一个名为**根 (root)** 的独立指针中。

普通树是 N 叉树的一种，具有以下特性：

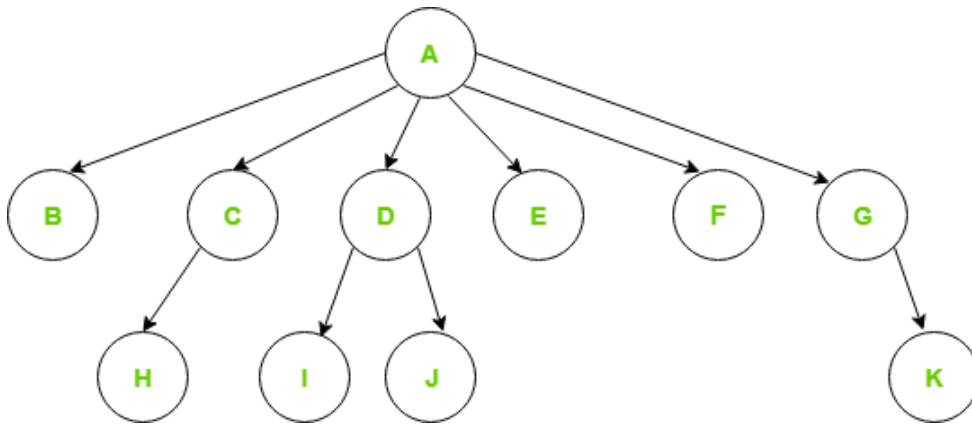
1. 每个节点可以有多个子节点。

2. 每个节点的子节点数量事先未知。

<https://www.geeksforgeeks.org/generic-treesn-array-trees/?ref=outind>

Last Updated : 27 Jul, 2024

Example:



Generic Tree

为了表示上述树结构，我们必须考虑最坏的情况，即拥有最多子节点的节点（在上面的例子中，有 6 个子节点），并为每个节点分配相应数量的指针。

基于此方法的节点表示可以写为：

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.firstchild = None
5         self.secondchild = None
6         self.thirdchild = None
7         self.fourthchild = None
8         self.fifthchild = None
9         self.sixthchild = None
10
```

上述表示方法的缺点是：

1. **内存浪费** – 并非所有情况下都需要用到所有的指针，因此会造成大量的内存浪费。
2. **子节点数量未知** – 每个节点的子节点数量事先无法确定。

简单方法：

为了存储节点中子节点的地址，我们可以使用数组或链表。但两种方法都会带来一些问题。

1. 在链表中，我们无法随机访问任意子节点的地址，因此效率较低，成本较高。
2. 在数组中，我们可以随机访问任意子节点的地址，但只能存储固定数量的子节点地址。

更好的方法：

我们可以使用动态数组来存储子节点的地址。它既可以随机访问任意子节点的地址，其大小（容量）也没有固定的限制。

```

1 class Node:
2
3     def __init__(self,data):
4         self.data=data
5         self.children=[ ]
6

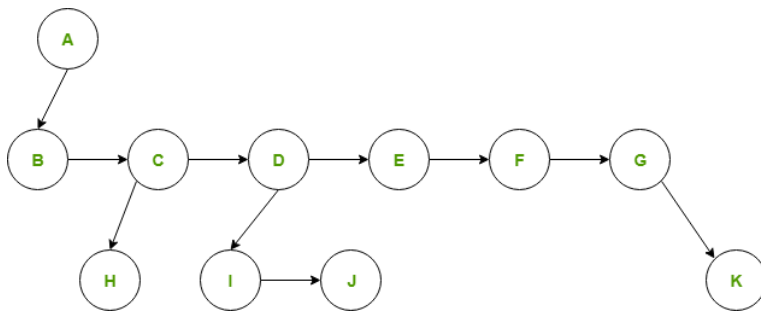
```

高效方法：长子-兄弟表示法

在“长子 / 下一个兄弟”表示法中，采取的步骤如下：

1. 将同一父节点的所有子节点（即兄弟节点）从左到右链接起来。
2. 移除父节点到所有子节点的链接，只保留到第一个孩子的链接。

由于子节点之间已经建立了链接，因此不需要从父节点到所有子节点的额外链接。这种表示法允许我们通过从父节点的第一个孩子开始，遍历所有的元素。



FIRST CHILD/NEXT SIBLING REPRESENTATION

The node declaration for first child / next sibling representation can be written as:

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.firstChild = None
5         self.nextSibling = None
6
7         # This code is contributed by aadityamaharshi
8

```

优点：

- **内存高效** – 不需要额外的链接，因此节省了大量内存。
- **可视为二叉树** – 由于我们能够将任何通用树转换为二叉树表示形式，因此可以**将所有使用“第一个孩子/下一个兄弟”表示法的通用树视为二叉树**。我们只需使用 `firstChild`（第一个孩子）和 `nextSibling`（下一个兄弟），而不用传统的左指针和右指针。
- **算法更易表达** – 因为它本质上是一棵二叉树，许多算法可以更轻松地实现。
- **节点大小固定** – 每个节点的大小是固定的，因此不需要辅助数组或向量来存储子节点信息。

下面是“长子-兄弟表示法 (First Child / Next Sibling Representation)”的一个完整 Python 示例，包括建树、遍历、插入等操作，帮助你理解这种多叉树的表示方式。

基本结构

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.firstChild = None    # 指向第一个孩子
5         self.nextSibling = None  # 指向下一个兄弟
```

构造示例树

假设我们要建立如下通用树：

```
1      A
2     / | \
3    B  C  D
4   / \  |
5  E  F  G
```

构建树

```
1  # 创建节点
2  A = Node('A')
3  B = Node('B')
4  C = Node('C')
5  D = Node('D')
6  E = Node('E')
7  F = Node('F')
8  G = Node('G')
9
10 # 构建长子-兄弟关系
11 A.firstChild = B      # A 的第一个孩子是 B
12 B.nextSibling = C     # B 的下一个兄弟是 C
13 C.nextSibling = D     # C 的下一个兄弟是 D
14
15 B.firstChild = E      # B 的第一个孩子是 E
16 E.nextSibling = F     # E 的下一个兄弟是 F
17
18 D.firstChild = G      # D 的第一个孩子是 G
```

遍历（前序遍历）

```

1 def preorder(node):
2     if node is None:
3         return
4     print(node.data, end=' ')
5     preorder(node.firstChild)
6     preorder(node.nextSibling)
7
8 # 输出: A B E F C D G
9 preorder(A)

```

层序遍历 (使用队列)

```

1 from collections import deque
2
3 def level_order(root):
4     if not root:
5         return
6     q = deque([root])
7     while q:
8         node = q.popleft()
9         print(node.data, end=' ')
10        child = node.firstChild
11        while child:
12            q.append(child)
13            child = child.nextSibling
14
15 # 输出: A B C D E F G
16 level_order(A)

```

插入函数 (在某节点下添加孩子)

```

1 def add_child(parent, data):
2     new_node = Node(data)
3     if not parent.firstChild:
4         parent.firstChild = new_node
5     else:
6         child = parent.firstChild
7         while child.nextSibling:
8             child = child.nextSibling
9         child.nextSibling = new_node
10    return new_node
11
12 # 示例: 在 C 下添加孩子 H
13 H = add_child(C, 'H')
14 preorder(A) # 输出: A B E F C H D G

```

总结:

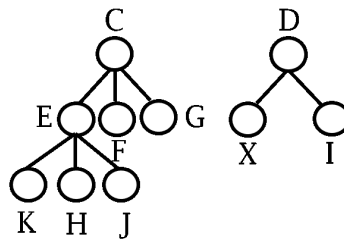
- `firstChild` → 进入下一层 (孩子)
- `nextSibling` → 同层横向移动 (兄弟)
- 空间上节省了多叉树需要的孩子数组指针, 结构类似二叉树
- 所有多叉树都可等价地用这种二叉树结构表示

练习M07161: 森林的带度数层次序列存储 (assign#6)

dfs + bfs, <http://cs101.openjudge.cn/practice/07161/>

对于树和森林等非线性结构，我们往往需要将其序列化以便存储。有一种树的存储方式称为带度数的层次序列。我们可以通过层次遍历的方式将森林序列转化为多个带度数的层次序列。

例如对于以下森林：



两棵树的层次遍历序列分别为：C E F G K H J / D X I

每个结点对应的度数为：3 3 0 0 0 0 0 / 2 0 0

我们将以上序列存储起来，就可以在以后的应用中恢复这个森林。在存储中，我们可以将第一棵树表示为C 3 E 3 F 0 G 0 K 0 H 0 J 0，第二棵树表示为D 2 X 0 I 0。

现在有一些通过带度数的层次遍历序列存储的森林数据，为了能够对这些数据进行进一步处理，首先需要恢复他们。

输入

输入数据的第一行包括一个正整数n，表示森林中非空的树的数目。

随后的 n 行，每行给出一棵树的带度数的层次序列。

树的节点名称为A-Z的单个大写字母。

输出

输出包括一行，输出对应森林的后根遍历序列。

样例输入

```
1 | 2
2 | C 3 E 3 F 0 G 0 K 0 H 0 J 0
3 | D 2 X 0 I 0
```

样例输出

```
1 | K H J E F G C X I D
```

【马振博 物院】长子-兄弟法

```
1 | from collections import deque
2 |
```

```

3  n = int(input())
4  ans = []
5
6  class TreeNode:
7      def __init__(self, x):
8          self.val = x
9          self.first_child = None
10         self.next_sibling = None
11     def __str__(self):
12         return str(self.val)
13
14 def postorder(x):
15     global ans
16     if x is None:
17         return
18     y = x.first_child
19     while y:
20         postorder(y)
21         y = y.next_sibling
22     ans.append(x.val)
23
24 for _ in range(n):
25     s = input().split()
26     root = TreeNode(s[0])
27     q = deque([[root, int(s[1])]])
28     i = 2
29     while q:
30         front = q.popleft()
31         cur = front[0]
32         for j in range(front[1]):
33             if j == 0:
34                 cur.first_child = TreeNode(s[i])
35                 cur = cur.first_child
36             else:
37                 cur.next_sibling = TreeNode(s[i])
38                 cur = cur.next_sibling
39             q.append((cur, int(s[i+1])))
40             i += 2
41     postorder(root)
42 print(*ans)

```

练习24729: 括号嵌套树 (assign #7)

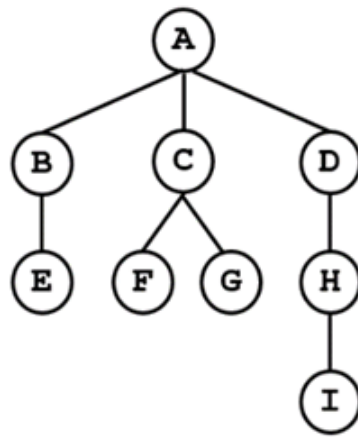
<http://cs101.openjudge.cn/practice/24729/>

可以用括号嵌套的方式来表示一棵树。表示方法如下：

1. 如果一棵树只有一个结点，则该树就用一个大写字母表示，代表其根结点。
2. 如果一棵树有子树，则用“树根(子树1,子树2,...,子树n)”的形式表示。树根是一个大写字母，子树之间用逗号隔开，没有空格。子树都是用括号嵌套法表示的树。

给出一棵不超过26个结点的树的括号嵌套表示形式，请输出其前序遍历序列和后序遍历序列。

输入样例代表的树如下图：



输入

一行，一棵树的括号嵌套表示形式

输出

两行。第一行是树的前序遍历序列，第二行是树的后序遍历序列

样例输入

```
1 | A(B(E),C(F,G),D(H(I)))
```

样例输出

```
1 | ABECFGDHI
2 | EBFGCIHDA
```

来源：Guo Wei

除了用邻接表实现，还可以用长子兄弟（Left-Child Right-Sibling, LCRS）法。

在LCRS表示法中，二叉树遍历与原多叉树遍历的关系：

1. **二叉树的前序遍历 (Root, Left, Right) = 原多叉树的前序遍历。**在 LCRS 中，先访问根，再访问第一个孩子，再访问兄弟，这恰好对应多叉树“根-子树1-子树2...”的顺序。
2. **二叉树的中序遍历 (Left, Root, Right) = 原多叉树的后序遍历。**这是最巧妙的一点。在二叉树中序遍历中，你会先遍历左子树（原多叉树的所有子孙），然后访问根节点，最后遍历右子树（原多叉树的后续兄弟节点）。对于多叉树的一个节点来说，它的“后序”意味着：先处理完它所有的子节点，再处理它自己。在二叉树结构中，处理完左子树（子孙）后紧接着访问根节点，正好完成了这个逻辑。
3. **递归构建逻辑：**多叉树转二叉树（左孩子右兄弟法）时，父节点只指向第一个子节点，而后续的兄弟节点应该链接在第一个子节点的右指针链上。

```
1 class Node:
2     def __init__(self, val):
3         self.val = val
4         self.left = None    # 指向第一个孩子
5         self.right = None   # 指向下一个兄弟
6
7
```

```

8  def buildTree(s):
9      i = 0
10     n = len(s)
11
12     def build():
13         nonlocal i
14         if i >= n:
15             return None
16
17         # 创建当前节点
18         node = Node(s[i])
19         i += 1
20
21         # 如果遇到 '(', 表示接下来是子节点列表
22         if i < n and s[i] == "(":
23             i += 1 # 跳过 '('
24
25             # 第一个子节点作为左孩子
26             node.left = build()
27             curr = node.left
28
29             # 剩下的子节点 (由逗号隔开) 作为前一个子节点的右兄弟
30             while i < n and s[i] == ",":
31                 i += 1 # 跳过 ','
32                 curr.right = build()
33                 curr = curr.right
34
35             # 处理完所有子节点, 跳过 ')'
36             if i < n and s[i] == ")":
37                 i += 1
38
39             return node
40
41     return build()
42
43
44 def Print(root):
45     pre_out = []
46     post_out = [] # 原多叉树的后序 = 二叉树的中序
47
48     def traverse(curr):
49         if not curr:
50             return
51         # 二叉树前序 = 原树前序
52         pre_out.append(curr.val)
53
54         # 递归左子树 (原树的孩子)
55         traverse(curr.left)
56
57         # 二叉树中序位置 = 原树后序
58         post_out.append(curr.val)
59
60         # 递归右子树 (原树的兄弟)
61         traverse(curr.right)
62
63     traverse(root)
64     print("".join(pre_out))
65     print("".join(post_out))

```

```
66
67
68 # 主程序
69 s = input().strip()
70 root = buildTree(s)
71 Print(root)
```

笔试填空@20240618

Q. 设森林 F 中有4棵树，第1、2、3、4棵树的结点个数分别为10、9、11、7，当把森林F转换成一棵二叉树后，其根结点的右子树中有多少个结点？ **27**

森林转换为二叉树

1. 第一棵树的根节点 成为二叉树的 根节点。
2. 第一棵树的所有子树 构成二叉树根节点的 左子树。
3. 森林中除第一棵树外，剩余的所有树 构成二叉树根节点的 右子树。

详细解析：

根据“左孩子右兄弟”（Left-Child Right-Sibling）的转换法则：

- 根节点的左孩子：指向第一棵树中根节点的第一个孩子。
- 根节点的右孩子：指向森林中第二棵树的根节点。
- 第二棵树根节点的右孩子：指向第三棵树的根节点，以此类推。

因此，转换后的二叉树结构如下：

- 根节点：由第1棵树的根节点担任（1个结点）。
- 左子树：由第1棵树中除去根节点以外的剩余结点组成（ $10 - 1 = 9$ 个结点）。
- 右子树：由第2、3、4棵树的**全部结点**组成。

计算过程：

根节点的右子树结点数 = 第2棵树的结点数 + 第3棵树的结点数 + 第4棵树的结点数
 $9 + 11 + 7 = 27$

答案：其根结点的右子树中有 **27** 个结点。

1.2 建立起“树=递归”的直觉

树结构的题目非常适合用递归来解决，因为树本身就是一个递归定义的数据结构——每个节点都可以看作是一个包含子节点的小树。使用递归方法来解决可以使代码更简洁、易读，并且通常更容易理解。以下是一些常见的树操作示例，以及如何使用递归优雅地实现它们。

使用类（class）来定义树的节点。每个节点包含三个属性：节点的值（val）、指向左子节点的引用（left）和指向右子节点的引用（right）。

```

1  # Definition for a binary tree node.
2  class TreeNode:
3      def __init__(self, val=0, left=None, right=None):
4          self.val = val
5          self.left = left
6          self.right = right

```

1. 遍历 (Traversal)

前序遍历 (Preorder Traversal)

先访问根节点，然后递归地前序遍历左子树，最后递归地前序遍历右子树。

```

1  def preorder_traversal(root):
2      if root:
3          print(root.val) # 访问根节点
4          preorder_traversal(root.left) # 递归遍历左子树
5          preorder_traversal(root.right) # 递归遍历右子树

```

中序遍历 (Inorder Traversal)

先递归地中序遍历左子树，然后访问根节点，最后递归地中序遍历右子树。

```

1  def inorder_traversal(root):
2      if root:
3          inorder_traversal(root.left) # 递归遍历左子树
4          print(root.val) # 访问根节点
5          inorder_traversal(root.right) # 递归遍历右子树

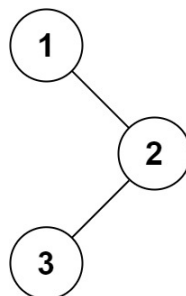
```

示例E94.二叉树的中序遍历 (assign#6)

<https://leetcode.cn/problems/binary-tree-inorder-traversal/>

给定一个二叉树的根节点 `root`，返回 它的 **中序** 遍历。

示例 1:



```

1  输入: root = [1,null,2,3]
2  输出: [1,3,2]

```

示例 2:

```
1 输入: root = []
2 输出: []
```

示例 3:

```
1 输入: root = [1]
2 输出: [1]
```

提示:

- 树中节点数目在范围 `[0, 100]` 内
- `-100 <= Node.val <= 100`

```
1 from typing import Optional, List
2
3 # Definition for a binary tree node.
4 class TreeNode:
5     def __init__(self, val=0, left=None, right=None):
6         self.val = val
7         self.left = left
8         self.right = right
9
10 class Solution:
11     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
12         result = []
13
14         def dfs(node: Optional[TreeNode]):
15             if not node:
16                 return
17             dfs(node.left)
18             result.append(node.val)
19             dfs(node.right)
20
21         dfs(root)
22         return result
```

用stack模拟的“颜色填充法”，和递归的思路其实很相似。

核心思想如下：

- 使用颜色标记节点的状态，新节点为白色，已访问的节点为灰色。
- 如果遇到的节点为白色，则将其标记为灰色，然后将其右子节点、自身、左子节点依次入栈。
- 如果遇到的节点为灰色，则将节点的值输出。

```
1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
```

```

5 #         self.left = left
6 #         self.right = right
7 class Solution:
8     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
9         white, gray = 0, 1
10        res = []
11        stack = [(white, root)]
12        while stack:
13            color, node = stack.pop()
14            if node is None: continue
15            if color == white:
16                stack.append((white, node.right))
17                stack.append((gray, node))
18                stack.append((white, node.left))
19            else:
20                res.append(node.val)
21        return res

```

二叉树的中序遍历顺序是：左子树 -> 根节点 -> 右子树。

这里提供两种主流写法：递归（最直观）和 迭代（进阶要求，使用栈）。

方法一：递归 (Recursion)

递归是最简单的方法。我们定义一个辅助函数，先访问左子树，再记录当前节点值，最后访问右子树。

```

1 class Solution:
2     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
3         res = []
4
5         def dfs(node):
6             if not node:
7                 return
8             # 1. 遍历左子树
9             dfs(node.left)
10            # 2. 访问根节点
11            res.append(node.val)
12            # 3. 遍历右子树
13            dfs(node.right)
14
15        dfs(root)
16        return res

```

- 时间复杂度： $O(n)$ ，其中 n 是节点数，每个节点访问一次。
- 空间复杂度： $O(n)$ ，最坏情况下（树呈链状）递归调用的栈深度为 n 。

方法二：迭代 (Iteration - 使用栈)

进阶要求使用迭代。我们可以利用显式栈来模拟递归的过程：

1. 一直向左走，将路径上的节点全部入栈，直到尽头。
2. 从栈中弹出一个节点（这通常是当前最左的节点）。
3. 记录该节点的值。
4. 转向该节点的右子树，重复步骤 1。


```

1 class Solution:
2     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
3         res = []
4         stack = []
5         curr = root
6
7         while curr or stack:
8             # 1. 尽可能向左走，并将沿途节点入栈
9             while curr:
10                stack.append(curr)
11                curr = curr.left
12
13            # 2. 当前节点为空，说明左边走到底了，弹出栈顶元素（最近的根节点）
14            curr = stack.pop()
15            res.append(curr.val)
16
17            # 3. 转向右子树
18            curr = curr.right
19
20        return res

```

- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(n)$ ，栈的大小在最坏情况下等于树的高度。

方法三：Morris 遍历 (进阶 - $O(1)$ 空间)

如果面试官要求 $O(1)$ 空间复杂度（不考虑结果数组），可以使用 Morris 遍历。它通过修改树的空闲指针，利用叶子节点的右指针指向其中序遍历的后继节点，从而避免了使用栈。

```

1 class Solution:
2     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
3         res = []
4         curr = root
5
6         while curr:
7             if not curr.left:
8                 # 如果没有左孩子，直接访问当前节点，并进入右孩子
9                 res.append(curr.val)
10                curr = curr.right
11            else:
12                # 找到左子树中序遍历的最后一个节点（最右侧节点）
13                pre = curr.left
14                while pre.right and pre.right != curr:
15                    pre = pre.right
16
17                if not pre.right:
18                    # 建立临时链接，指向后继节点（当前节点）
19                    pre.right = curr
20                    curr = curr.left
21                else:
22                    # 链接已存在，说明左子树已访问完，断开链接，访问当前节点
23                    pre.right = None
24                    res.append(curr.val)
25                    curr = curr.right
26        return res

```

- **时间复杂度：** $O(n)$ ，虽然有嵌套循环，但每条边最多被访问两次。
- **空间复杂度：** $O(1)$ （不计入结果列表）。

Morris 遍历的核心思想可以用一句话概括：**利用二叉树中大量空闲的 `right` 指针，建立指向“后继”节点的临时线索。**

1. 为什么需要栈？

在正常的遍历（递归或迭代）中，当我们从父节点进入左子树后，**必须记住回来的路**，否则等左子树遍历完了，我们就找不到父节点，也就无法进入右子树。

- **递归：**利用系统函数栈来保存返回地址。
- **迭代：**利用自定义的 `std::stack` 来保存父节点。

2. Morris 的天才构思：利用“前驱”指向“后继”

Morris 问：能不能不花额外空间就把“回来的路”存下来？

他观察到：在中序遍历中，当你遍历完左子树时，**最后访问的那个节点**（左子树中最右边的节点，称为“前驱节点”）的 `right` 指针是空的！

Morris 的做法：

让这个原本指向 `nullptr` 的 `right` 指针，临时指向当前的父节点。这样，当我们处理完左子树最末尾的节点后，顺着这个“后继线索”就能直接跳回父节点。

3. 图解原理

假设有这样一棵树：



第一步：建立线索

1. 当前在 **1**。它有左子树。
2. 找到 **1** 的左子树中最右的节点，即 **5**。
3. 发现 **5** 的 `right` 是空的，于是建立线索：`5->right = 1`。
4. 此时树变成了这样（虚线是线索）：



5. 向左移动：`curr = 2`。

第二步：继续向下

1. 当前在 **2**。它有左子树。
2. 找到 **2** 的左子树中最右的节点，即 **4**。

3. 建立线索: `4->right = 2`。
4. 向左移动: `curr = 4`。

第三步: 利用线索返回

1. 当前在 `4`。它没有左子树。
2. 打印 `4`。
3. 移动到 `curr->right`。注意! 此时 `4->right` 已经指向了 `2`。
4. 回到 `2` 了!

第四步: 消除线索 (恢复树结构)

1. 再次寻找 `2` 的前驱, 发现是 `4`。
2. 但此时 `4->right` 已经指向 `2` 了 (说明这是第二次来, 左子树已完)。
3. 打印 `2`。
4. 擦除线索: `4->right = nullptr`。
5. 移动到 `2` 的右子节点: `curr = 5`。

依此类推...

接下来会处理 `5`, 然后通过 `5->right` 回到 `1`, 打印 `1`, 最后进入 `3`。

4. 三个关键状态

在 Morris 遍历中, 对于每一个有左子树的节点, 我们会“路过”它两次:

1. 第一次经过:
 - 目的: 建立线索 (从左子树最右节点连回当前节点)。
 - 动作: 去左子树。
2. 第二次经过 (从左子树底端回来):
 - 目的: 销毁线索, 访问当前节点。
 - 动作: 去右子树。

5. 复杂度分析

- 时间复杂度 $O(n)$:
虽然看起来有嵌套循环, 但实际上每条边最多被走 3 次:
 1. 寻找前驱建立线索时。
 2. 遍历移动时。
 3. 寻找前驱销毁线索时。所以总时间是线性的。
- 空间复杂度 $O(1)$:
没有使用栈或递归, 只用了两个辅助指针 (`curr` 和 `pre`)。

总结

Morris 遍历就像是在森林里走路:

- 普通遍历: 随身带地图 (栈)。
- Morris 遍历: 每走到一个分叉路口, 先派人去左边小路的尽头, 在那棵树上刻个箭头指向这个路口, 然后再往左走。等以后顺着箭头走回来时, 再把箭头抹掉, 继续往右走。

添加一个辅助函数来根据列表创建二叉树，然后调用这个函数来生成 `root` 节点。

```
1  from typing import Optional, List
2
3
4  class TreeNode:
5      def __init__(self, val=0, left=None, right=None):
6          self.val = val
7          self.left = left
8          self.right = right
9
10
11 class Solution:
12     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
13         res = []
14         curr = root
15
16         while curr:
17             if not curr.left:
18                 # 如果没有左孩子，直接访问当前节点，并进入右孩子
19                 res.append(curr.val)
20                 curr = curr.right
21             else:
22                 # 找到左子树中序遍历的最后一个节点（最右侧节点）
23                 pre = curr.left
24                 while pre.right and pre.right != curr:
25                     pre = pre.right
26
27                 if not pre.right:
28                     # 建立临时链接，指向后继节点（当前节点）
29                     pre.right = curr
30                     curr = curr.left
31                 else:
32                     # 链接已存在，说明左子树已访问完，断开链接，访问当前节点
33                     pre.right = None
34                     res.append(curr.val)
35                     curr = curr.right
36         return res
37
38
39 def build_tree_from_list(values):
40     """
41     根据列表构建二叉树。
42     列表按层序遍历顺序表示二叉树，None 表示空节点。
43     """
44     if not values:
45         return None
46
47     # 创建根节点
48     root = TreeNode(values[0])
49     # 使用队列进行层序构建
50     queue = [root]
51     i = 1
52
53     while i < len(values):
54         current_node = queue.pop(0)
```

```

55
56     # 处理左子节点
57     if i < len(values) and values[i] is not None:
58         current_node.left = TreeNode(values[i])
59         queue.append(current_node.left)
60     i += 1
61
62     # 处理右子节点
63     if i < len(values) and values[i] is not None:
64         current_node.right = TreeNode(values[i])
65         queue.append(current_node.right)
66     i += 1
67
68     return root
69
70
71 if __name__ == "__main__":
72     sol = Solution()
73     # 1. 使用辅助函数将列表转换为 TreeNode 对象
74     #root = build_tree_from_list([1, None, 2, 3])
75     root = build_tree_from_list([1, 2, 3, 4, 5])
76     """
77         1
78        / \
79       2  3
80      / \
81     4  5
82     """
83     # 2. 将 TreeNode 对象传入方法
84     print(sol.inorderTraversal(root))
85
86     # [4, 2, 5, 1, 3]

```

Morris 遍历 (Morris Traversal) 的精髓在于：利用二叉树中大量的空闲指针（叶子节点的右孩子指针），来保存临时信息，从而省去栈的空间。

它的核心目标是：在左子树遍历完后，能通过某种方式“走回来”找到根节点。

1. 核心逻辑（直白版）

假设当前节点为 `curr`：

1. 如果 `curr` 没有左孩子：

- 说明左边没东西了，直接打印 `curr`。
- 然后去右边： `curr = curr.right`。

2. 如果 `curr` 有左孩子：

- 我们要找 `curr` 的前驱节点（即：左子树中最后被访问的那个点，也就是左子树里“最右边”的节点）。
- 情况 A（还没建立连接）：如果前驱节点的右孩子是空的，我们就把它连到 `curr` 上： `pre.right = curr`。然后继续去左子树： `curr = curr.left`。
- 情况 B（已经建立过连接）：如果前驱节点的右孩子指向 `curr`，说明这是第二次回来，左子树已经走完了。此时我们要断开连接（恢复树结构），打印 `curr`，然后去右边： `curr = curr.right`。

2. 详细图解示例

假设我们有这样一颗二叉树：



目标中序遍历结果：[4, 2, 5, 1, 3]

第一阶段：从根节点 1 开始

- `curr` 在 1。它有左孩子 (2)。
- 找到 1 的前驱 (左子树最右节点)：是 5。
- 5 的右孩子为空，建立连接：`5.right = 1`。
- `curr` 移向左孩子 2。
 - 此时树的样子 (逻辑上)：4 -> 2 -> 5 -> 1 -> 3

第二阶段：处理节点 2

- `curr` 在 2。它有左孩子 (4)。
- 找到 2 的前驱 (左子树最右节点)：是 4。
- 4 的右孩子为空，建立连接：`4.right = 2`。
- `curr` 移向左孩子 4。

第三阶段：处理节点 4

- `curr` 在 4。它没有左孩子。
- 打印 4。
- `curr` 移向 `curr.right`。由于刚才连了线，它回到了 2。

第四阶段：回到节点 2

- `curr` 在 2。它有左孩子 (4)。
- 找到 2 的前驱：还是 4。
- 发现 `4.right` 已经指向了 `curr(2)`。
- 说明左边全走完了！
- 断开连接：`4.right = None`。
- 打印 2。
- `curr` 移向右孩子 5。

第五阶段：处理节点 5

- `curr` 在 5。它没有左孩子。
- 打印 5。
- `curr` 移向 `curr.right`。由于第一阶段连了线，它回到了 1。

第六阶段：回到节点 1

- `curr` 在 1。它有左孩子 (2)。
- 找到 1 的前驱：是 5 (沿着 2 -> 5 找到)。

- 发现 `5.right` 已经指向了 `curr(1)`。
- 说明左边全走完了！
- 断开连接： `5.right = None`。
- 打印 1。
- `curr` 移向右孩子 3。

第七阶段：处理节点 3

- `curr` 在 3。没有左孩子。
- 打印 3。
- `curr` 移向 `curr.right` (None) 。
- 结束。

3. 总结

步骤	打印结果	说明
1. 遇到1		建立 5 -> 1 的线，去2
2. 遇到2		建立 4 -> 2 的线，去4
3. 遇到4	4	无左孩子，打印并沿线回2
4. 回到2	2	发现 4 -> 2 已存，拆线，打印，去5
5. 遇到5	5	无左孩子，打印并沿线回1
6. 回到1	1	发现 5 -> 1 已存，拆线，打印，去3
7. 遇到3	3	无左孩子，打印，结束

核心精髓：

1. 线索化：把原本是 `None` 的右指针利用起来，指向中序遍历的后继。
2. 原地修改，原地恢复：访问完后把线拆掉，不破坏原树结构。
3. 空间 $O(1)$ ：除了保存结果的数组，只用了两个辅助指针（`curr` 和 `pre`），没有用栈，也没有递归。

Q：通常是root先入栈，写成 `stack = [root]`，不用curr。这个代码为什么？：方法二：迭代 (Iteration - 使用栈)

进阶要求使用迭代。我们可以利用显式栈来模拟递归的过程：

1. 一直向左走，将路径上的节点全部入栈，直到尽头。
2. 从栈中弹出一个节点（这通常是当前最左的节点）。
3. 记录该节点的值。
4. 转向该节点的右子树，重复步骤 1。

```
1 class Solution:
2     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
3         res = []
4         stack = []
```

```

5     curr = root
6
7     while curr or stack:
8         # 1. 尽可能向左走，并将沿途节点入栈
9         while curr:
10            stack.append(curr)
11            curr = curr.left
12
13        # 2. 当前节点为空，说明左边走到底了，弹出栈顶元素（最近的根节点）
14        curr = stack.pop()
15        res.append(curr.val)
16
17        # 3. 转向右子树
18        curr = curr.right
19
20    return res

```

- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(n)$ ，栈的大小在最坏情况下等于树的高度。

这是一个非常经典的问题。你观察得很细致：在 **层序遍历 (BFS)** 或 **先序遍历 (Preorder)** 的迭代写法中，我们通常直接初始化 `stack = [root]`。

但在 **中序遍历 (Inorder)** 中，这种 `curr + stack` 的写法是最主流、最直观的。原因主要有以下几点：

1. 访问顺序与处理顺序的不一致

- **先序遍历/BFS**：当你到达一个节点时，立刻就可以处理它（记录它的值），然后把它的孩子入栈。所以 `stack = [root]` 很方便。
- **中序遍历**：当你到达根节点时，你不能立刻处理它。你必须先穿过它，一直走到最左边的子节点。
 - `curr` 的作用就是：负责“带路”去寻找最左侧的节点。
 - `stack` 的作用就是：记住回家的路（记录路径上的父节点），以便处理完左子树后能回来找根节点。

2. 模拟递归的真实过程

递归的代码是这样的：

```

1 def inorder(root):
2     if not root: return
3     inorder(root.left) # 这一步对应 while curr: stack.append(curr); curr = curr.left
4     print(root.val)    # 这一步对应 curr = stack.pop(); res.append(curr.val)
5     inorder(root.right) # 这一步对应 curr = curr.right

```

在递归中，程序会一直压入系统栈直到 `root.left` 为空。

在迭代写法中，`curr` 模拟的就是那个正在向下探索的指针。如果一开始就写 `stack = [root]`，逻辑上会变成“我已经把 `root` 放到待处理队列了”，但中序遍历要求我们必须先深入左子树，此时 `root` 并没有被“处理”，它只是在“排队等待”。

3. 避免重复访问（死循环）

如果你使用 `stack = [root]` 且不配合 `curr` 指针，你会遇到一个逻辑难题：

当你从栈中弹出 `root` 时，你怎么知道它的左子树是还没开始处理，还是已经处理完了？

- 如果没处理，你需要把左子树入栈。
- 如果处理完了，你需要记录 `root` 的值并转向右子树。

如果不引入额外的标志位（如颜色标记法），只用一个栈和 `stack = [root]` 很难区分这两个状态。而使用 `curr` 指针则非常清晰：

- 只要 `curr` 不为空，就是在“去左边”的路上。
- 只要 `curr` 为空开始从栈里弹元素，就是在“回程”或者说“处理根节点”的路上。

补充：如果你非要写 `stack = [root]`，代码会变成什么样？

有一种写法叫标记法（颜色标记法），它可以统一三种遍历，且初始状态是 `stack = [(WHITE, root)]`：

```
1 class Solution:
2     def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
3         WHITE, GRAY = 0, 1
4         res = []
5         stack = [(WHITE, root)]
6         while stack:
7             color, node = stack.pop()
8             if node is None: continue
9             if color == WHITE:
10                 # 中序遍历：右 -> 根 -> 左（入栈顺序反过来）
11                 stack.append((WHITE, node.right))
12                 stack.append((GRAY, node))
13                 stack.append((WHITE, node.left))
14             else:
15                 res.append(node.val)
16         return res
```

总结

你提供的这种写法之所以不用 `stack = [root]` 而是用 `curr`：

1. 逻辑解耦：`curr` 负责向下找左孩子，`stack` 负责回溯。
2. 状态明确：不需要额外的标记位就能知道左子树是否处理完毕。
3. 高度模拟：它是对计算机递归调用栈最纯粹的模拟。

所以，这种写法虽然初看有点别扭，但它是处理中序遍历最健壮、最标准的迭代模板。

后序遍历（Postorder Traversal）

先递归地后序遍历左子树，然后递归地后序遍历右子树，最后访问根节点。

```
1 def postorder_traversal(root):
2     if root:
3         postorder_traversal(root.left) # 递归遍历左子树
4         postorder_traversal(root.right) # 递归遍历右子树
5         print(root.val) # 访问根节点
```

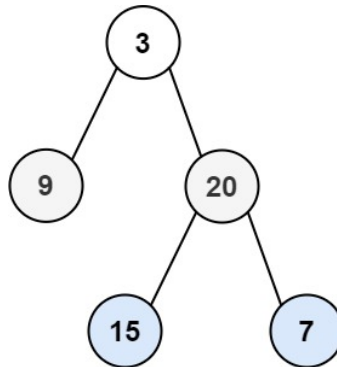
层序遍历：按层从左到右依次遍历

练习M102.二叉树的层序遍历（assign#6）

bfs, <https://leetcode.cn/problems/binary-tree-level-order-traversal/>

给你二叉树的根节点 `root`，返回其节点值的层序遍历。（即逐层地，从左到右访问所有节点）。

示例 1:



```
1 输入: root = [3,9,20,null,null,15,7]
2 输出: [[3],[9,20],[15,7]]
```

示例 2:

```
1 输入: root = [1]
2 输出: [[1]]
```

示例 3:

```
1 输入: root = []
2 输出: []
```

提示:

- 树中节点数目在范围 `[0, 2000]` 内
- `-1000 <= Node.val <= 1000`

```
1  # Definition for a binary tree node.
2  # class TreeNode:
3  #     def __init__(self, val=0, left=None, right=None):
4  #         self.val = val
5  #         self.left = left
6  #         self.right = right
7  class Solution:
8      def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
9          if not root:
10             return []
11
12             result = []
13             queue = deque([root])
14
15             while queue:
16                 level_size = len(queue)
17                 level = []
18
19                 for _ in range(level_size):
```

```

20         node = queue.popleft()
21         level.append(node.val)
22         if node.left:
23             queue.append(node.left)
24         if node.right:
25             queue.append(node.right)
26
27         result.append(level)
28
29     return result
30

```

序列化与反序列化 (Serialization)

序列化是将树转为线性字符串，反序列化是恢复过程。这是现代开发（如JSON、Protobuf）和高级算法题（LeetCode 297）的高频考点。

练习M297.二叉树的序列化与反序列化 (assign#7)

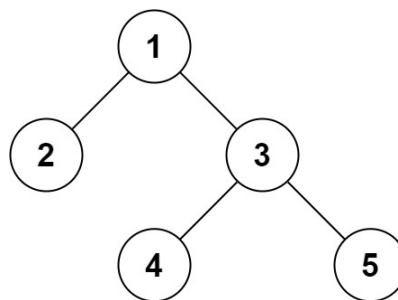
dfs, bfs, <https://leetcode.cn/problems/serialize-and-deserialize-binary-tree/>

序列化是将一个数据结构或者对象转换为连续的比特位的操作，进而可以将转换后的数据存储在一个文件或者内存中，同时也可以通过网络传输到另一个计算机环境，采取相反方式重构得到原数据。

请设计一个算法来实现二叉树的序列化与反序列化。这里不限定你的序列 / 反序列化算法执行逻辑，你只需要保证一个二叉树可以被序列化为一个字符串并且将这个字符串反序列化为原始的树结构。

提示: 输入输出格式与 LeetCode 目前使用的方式一致，详情请参阅 [LeetCode 序列化二叉树的格式](#)。你并非必须采取这种方式，你也可以采用其他的方法解决这个问题。

示例 1:



```

1  输入: root = [1,2,3,null,null,4,5]
2  输出: [1,2,3,null,null,4,5]

```

示例 2:

```

1  输入: root = []
2  输出: []

```

示例 3:

```
1 输入: root = [1]
2 输出: [1]
```

示例 4:

```
1 输入: root = [1,2]
2 输出: [1,2]
```

提示:

- 树中结点数在范围 `[0, 104]` 内
- `-1000 <= Node.val <= 1000`

这个问题要求实现二叉树的序列化 (`Tree -> String`) 与反序列化 (`String -> Tree`)。

最常用的两种方法是 **DFS (前序遍历)** 和 **BFS (层序遍历)**。

方法一: DFS (前序遍历)

这种方法比较直观, 序列化时按照“根-左-右”的顺序, 遇到空节点用特殊字符 (如 `#` 或 `None`) 表示。反序列化时, 同样按前序遍历的顺序递归构建。

代码实现:

```
1 class Codec:
2     def serialize(self, root):
3         """将二叉树编码为字符串。"""
4         res = []
5         def dfs(node):
6             if not node:
7                 res.append("None")
8                 return
9             res.append(str(node.val))
10            dfs(node.left)
11            dfs(node.right)
12
13        dfs(root)
14        return ",".join(res)
15
16    def deserialize(self, data):
17        """将字符串解码为二叉树。"""
18        data_list = data.split(",")
19        # 使用迭代器或全局索引来逐个读取列表元素
20        self.it = iter(data_list)
21
22        def build():
23            val = next(self.it)
24            if val == "None":
25                return None
26
27            node = TreeNode(int(val))
28            node.left = build() # 递归构建左子树
29            node.right = build() # 递归构建右子树
30            return node
```

```
31
32     return build()
```

方法二：BFS（层序遍历）

这种方法与 LeetCode 题目描述中展示的格式完全一致。序列化时使用队列进行层序遍历，包括空节点。反序列化时也使用队列，依次为队列中的节点挂载左右子节点。

代码实现：

```
1  from collections import deque
2
3  class Codec:
4      def serialize(self, root):
5          """将二叉树编码为字符串。"""
6          if not root: return ""
7          queue = deque([root])
8          res = []
9          while queue:
10             node = queue.popleft()
11             if node:
12                 res.append(str(node.val))
13                 queue.append(node.left)
14                 queue.append(node.right)
15             else:
16                 res.append("None")
17             return ",".join(res)
18
19     def deserialize(self, data):
20         """将字符串解码为二叉树。"""
21         if not data: return None
22         nodes = data.split(",")
23         root = TreeNode(int(nodes[0]))
24         queue = deque([root])
25         index = 1 # 用于遍历 nodes 列表
26
27         while queue:
28             node = queue.popleft()
29
30             # 处理左子节点
31             if nodes[index] != "None":
32                 node.left = TreeNode(int(nodes[index]))
33                 queue.append(node.left)
34             index += 1
35
36             # 处理右子节点
37             if nodes[index] != "None":
38                 node.right = TreeNode(int(nodes[index]))
39                 queue.append(node.right)
40             index += 1
41
42         return root
```

复杂度分析

1. 时间复杂度：

- 序列化： $O(N)$ ，其中 N 是节点数，每个节点访问一次。
- 反序列化： $O(N)$ ，每个节点被处理一次。

2. 空间复杂度：

- DFS**： $O(N)$ 。在最坏情况下（树呈链状），递归栈的深度为 N 。
- BFS**： $O(N)$ 。队列在最坏情况下（完全二叉树最后一层）存储约 $N/2$ 个节点。

总结

- DFS** 代码更简洁，利用递归天然满足“根左右”的顺序。
- BFS** 逻辑上更符合我们观察树的直觉（一层一层看），且生成的字符串格式通常更短（可以剪掉末尾不必要的空节点，虽然上面代码为了逻辑清晰没做特殊剪裁）。

2. 求树的高度/深度

```
1 def tree_height(root):
2     if not root: # 空树的高度为 0
3         return 0
4
5     left_height = tree_height(root.left) # 左子树的高度
6     right_height = tree_height(root.right) # 右子树的高度
7
8     return max(left_height, right_height) + 1 # 树的高度是左右子树最大高度加 1
```

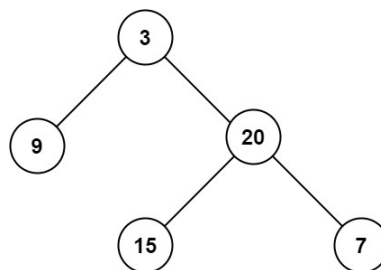
示例E104.二叉树的最大深度

tree, dfs, <https://leetcode.cn/problems/maximum-depth-of-binary-tree/>

给定一个二叉树 `root`，返回其最大深度。

二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

示例 1：



```
1 输入: root = [3,9,20,null,null,15,7]
2 输出: 3
```

示例 2:

```
1 输入: root = [1,null,2]
2 输出: 2
```

提示:

- 树中节点的数量在 $[0, 10^4]$ 区间内。
- $-100 \leq \text{Node.val} \leq 100$

已知树根root

```
1  # Definition for a binary tree node.
2  # class TreeNode:
3  #     def __init__(self, val=0, left=None, right=None):
4  #         self.val = val
5  #         self.left = left
6  #         self.right = right
7  class Solution:
8      def maxDepth(self, root: Optional[TreeNode]) -> int:
9          def tree_depth(node):
10             if node is None:
11                 return 0
12             left_depth = tree_depth(node.left)
13             right_depth = tree_depth(node.right)
14             return max(left_depth, right_depth) + 1
15
16         return tree_depth(root)
```

练习M06646: 二叉树的深度

<http://cs101.openjudge.cn/practice/06646/>

给定一棵二叉树，求该二叉树的深度

二叉树深度定义：从根结点到叶结点依次经过的结点（含根、叶结点）形成树的一条路径，最长路径的节点个数为树的深度

输入

第一行是一个整数n，表示二叉树的结点个数。二叉树结点编号从1到n，根结点为1， $n \leq 10$

接下来有n行，依次对应二叉树的n个结点。

每行有两个整数，分别表示该节点的左儿子和右儿子的节点编号。如果第一个（第二个）数为-1则表示没有左（右）儿子

输出

输出一个整型数，表示树的深度

样例输入

```
1 | 3
2 | 2 3
3 | -1 -1
4 | -1 -1
```

样例输出

```
1 | 2
```

推荐这种类的写法，在笔试中也常见。根据输入建树，知道树根位置。

```
1 class TreeNode:
2     def __init__(self, val=0, left=None, right=None):
3         self.val = val
4         self.left = left
5         self.right = right
6
7
8 def build_tree(nodes):
9     if not nodes:
10         return None
11
12     tree_nodes = [None] * (len(nodes) + 1)
13     for i in range(1, len(nodes) + 1):
14         tree_nodes[i] = TreeNode(i)
15
16     for i, (left, right) in enumerate(nodes, start=1):
17         if left != -1:
18             tree_nodes[i].left = tree_nodes[left]
19         if right != -1:
20             tree_nodes[i].right = tree_nodes[right]
21
22     return tree_nodes[1]
23
24
25 def tree_depth(root):
26     if not root:
27         return 0
28     left_depth = tree_depth(root.left)
29     right_depth = tree_depth(root.right)
30     return max(left_depth, right_depth) + 1
31
32
33 def main():
34     n = int(input())
35     nodes = []
36     index = 1
37     for _ in range(n):
38         left, right = map(int, input().split())
39         nodes.append((left, right))
40
41     root = build_tree(nodes)
42     depth = tree_depth(root)
43     print(depth)
44
```



```

45
46 if __name__ == "__main__":
47     main()
48

```

练习M27638: 求二叉树的高度和叶子数目

<http://cs101.openjudge.cn/practice/27638/>

给定一棵二叉树，求该二叉树的高度和叶子数目。二叉树高度定义：从根结点到叶结点依次经过的结点（含根、叶结点）形成树的一条路径，最长路径的结点数减1为树的高度。只有一个结点的二叉树，高度是0。

输入

第一行是一个整数n，表示二叉树的结点个数。二叉树结点编号从0到n-1，根结点n ≤ 100。接下来有n行，依次对应二叉树的编号为0,1,2,...,n-1的结点。每行有两个整数，分别表示该结点的左儿子和右儿子的编号。如果第一个（第二个）数为-1则表示没有左（右）儿子。

输出

在一行中输出2个整数，分别表示二叉树的高度和叶子结点数。

样例输入

```

1 | 3
2 | -1 -1
3 | 0 2
4 | -1 -1

```

样例输出

```

1 | 1 2

```

根据输入建树，找到树根。

```

1 class TreeNode:
2     def __init__(self):
3         self.left = None
4         self.right = None
5
6 def tree_height(node):
7     if node is None:
8         return -1 # 根据定义，空树高度为-1
9     return max(tree_height(node.left), tree_height(node.right)) + 1
10
11 def count_leaves(node):
12     if node is None:
13         return 0
14     if node.left is None and node.right is None:
15         return 1
16     return count_leaves(node.left) + count_leaves(node.right)
17
18 n = int(input()) # 读取节点数量

```

```

19 nodes = [TreeNode() for _ in range(n)]
20 has_parent = [False] * n # 用来标记节点是否有父节点
21
22 for i in range(n):
23     left_index, right_index = map(int, input().split())
24     if left_index != -1:
25         nodes[i].left = nodes[left_index]
26         has_parent[left_index] = True
27     if right_index != -1:
28         #print(right_index)
29         nodes[i].right = nodes[right_index]
30         has_parent[right_index] = True
31
32 # 寻找根节点, 也就是没有父节点的节点
33 root_index = has_parent.index(False)
34 root = nodes[root_index]
35
36 # 计算高度和叶子节点数
37 height = tree_height(root)
38 leaves = count_leaves(root)
39
40 print(f"{height} {leaves}")

```

3. 判断两棵树是否相同

```

1 def is_same_tree(p, q):
2     if not p and not q:
3         return True
4     if not p or not q:
5         return False
6     return (p.val == q.val and
7             is_same_tree(p.left, q.left) and
8             is_same_tree(p.right, q.right))

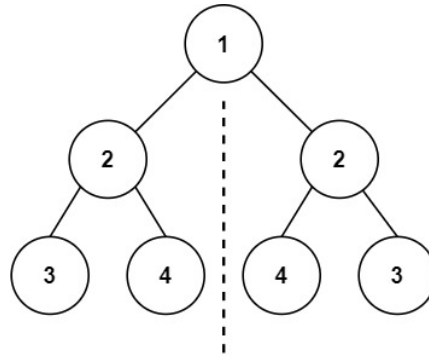
```

示例E101.对称二叉树

dfs, <https://leetcode.cn/problems/symmetric-tree/>

给你一个二叉树的根节点 `root` , 检查它是否轴对称。

示例 1:

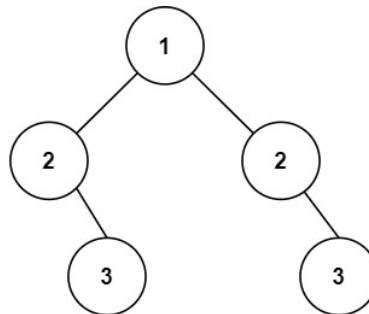


```

1 输入: root = [1,2,2,3,4,4,3]
2 输出: true

```

示例 2:



```

1 输入: root = [1,2,2,null,3,null,3]
2 输出: false

```

提示:

- 树中节点数目在范围 `[1, 1000]` 内
- `-100 <= Node.val <= 100`

```

1  # Definition for a binary tree node.
2  # class TreeNode:
3  #     def __init__(self, val=0, left=None, right=None):
4  #         self.val = val
5  #         self.left = left
6  #         self.right = right
7  class Solution:
8      def isSymmetric(self, root: Optional[TreeNode]) -> bool:
9          if not root:
10             return True
11
12         def isMirror(left: TreeNode, right: TreeNode) -> bool:
13             if not left and not right:
14                 return True
15             if not left or not right:
16                 return False
17             return (left.val == right.val) and isMirror(left.left, right.right) and
isMirror(left.right, right.left)

```

```
18  
19     return isMirror(root.left, root.right)
```

4. 翻转二叉树

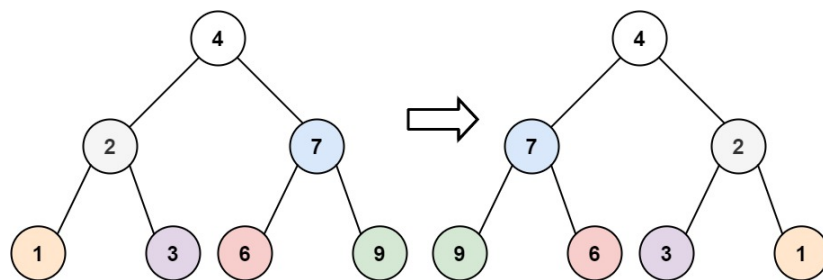
```
1  def invert_tree(root):  
2      if root:  
3          root.left, root.right = invert_tree(root.right), invert_tree(root.left)  
4      return root
```

示例E226.翻转二叉树

<https://leetcode.cn/problems/invert-binary-tree/>

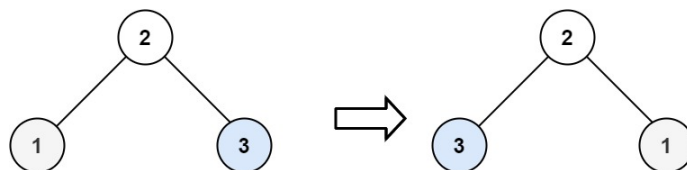
给你一棵二叉树的根节点 `root`，翻转这棵二叉树，并返回其根节点。

示例 1:



```
1 输入: root = [4,2,7,1,3,6,9]  
2 输出: [4,7,2,9,6,3,1]
```

示例 2:



```
1 输入: root = [2,1,3]  
2 输出: [2,3,1]
```

示例 3:

```
1 输入: root = []  
2 输出: []
```

提示:

- 树中节点数目范围在 `[0, 100]` 内

- `-100 <= Node.val <= 100`

```

1  # Definition for a binary tree node.
2  # class TreeNode:
3  #     def __init__(self, val=0, left=None, right=None):
4  #         self.val = val
5  #         self.left = left
6  #         self.right = right
7  class Solution:
8      def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
9          if not root:
10             return root
11
12         left = self.invertTree(root.left)
13         right = self.invertTree(root.right)
14         root.left, root.right = right, left
15         return root

```

5. 寻找二叉搜索树中的最小值/最大值

对于二叉搜索树（BST），最小值在最左边，最大值在最右边。BST各种翻译的名字都需要认识。

中文翻译	英文原名	使用场景
二叉搜索树	Binary Search Tree	最常见翻译，通用性强
二叉查找树	Binary Search Tree	强调查找功能
二叉排序树	Binary Search Tree	强调排序特性
二叉检索树	Binary Search Tree	强调检索功能
有序二叉树	Binary Search Tree	强调有序性
BST	Binary Search Tree	技术文档、代码注释中常用

```

1  def find_min(root):
2      if not root.left:
3          return root.val
4      return find_min(root.left)
5
6  def find_max(root):
7      if not root.right:
8          return root.val
9      return find_max(root.right)

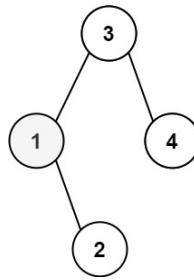
```

练习M230.二叉搜索树中第K小的元素

<https://leetcode.cn/problems/kth-smallest-element-in-a-bst/>

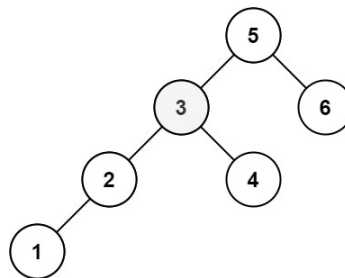
给定一个二叉搜索树的根节点 `root`，和一个整数 `k`，请你设计一个算法查找其中第 `k` 小的元素（从 1 开始计数）。

示例 1:



```
1 输入: root = [3,1,4,null,2], k = 1
2 输出: 1
```

示例 2:



```
1 输入: root = [5,3,6,2,4,null,null,1], k = 3
2 输出: 3
```

提示:

- 树中的节点数为 `n`。
- `1 <= k <= n <= 10^4`
- `0 <= Node.val <= 10^4`

进阶: 如果二叉搜索树经常被修改（插入/删除操作）并且你需要频繁地查找第 `k` 小的值，你将如何优化算法？

BST的中序遍历是升序的。只需要对BST进行 **中序遍历**，遍历到第 `k` 个节点时返回它即可。

```
1  # Definition for a binary tree node.
2  # class TreeNode:
3  #     def __init__(self, val=0, left=None, right=None):
4  #         self.val = val
5  #         self.left = left
6  #         self.right = right
7  class Solution:
8      def kthSmallest(self, root, k):
9          self.k = k
10         self.result = None
11
12         def inorder(node):
13             if not node or self.result is not None:
14                 return
```

```

15         inorder(node.left)
16         self.k -= 1
17         if self.k == 0:
18             self.result = node.val
19             return
20         inorder(node.right)
21
22     inorder(root)
23     return self.result
24
25

```

这个问题利用 **二叉搜索树 (BST)** 的中序遍历是升序序列 这一特性。只需要在遍历过程中计数，当计数达到 k 时，当前的节点值即为答案。

解法一：递归中序遍历

使用递归进行中序遍历，为了在找到结果后立即停止，我们可以使用一个实例变量来记录计数和结果。

```

1 class Solution:
2     def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
3         self.res = None
4         self.count = 0
5
6         def dfs(node):
7             if not node or self.res is not None:
8                 return
9
10            # 左
11            dfs(node.left)
12
13            # 根
14            self.count += 1
15            if self.count == k:
16                self.res = node.val
17                return
18
19            # 右
20            dfs(node.right)
21
22        dfs(root)
23        return self.res

```

解法二：迭代中序遍历（推荐）

迭代法使用栈来模拟中序遍历。它的优点是可以在找到第 k 个元素时**立即提前退出**，节省不必要的计算。

```

1 class Solution:
2     def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
3         stack = []
4         curr = root
5
6         while stack or curr:
7             # 1. 尽可能向左走
8             while curr:
9                 stack.append(curr)

```

```

10         curr = curr.left
11
12         # 2. 弹出并访问
13         curr = stack.pop()
14         k -= 1
15         if k == 0:
16             return curr.val
17
18         # 3. 转向右子树
19         curr = curr.right

```

进阶思考：频繁修改且频繁查询如何优化？

如果 BST 经常被修改（插入/删除），而我们需要频繁查询第 k 小的元素，那么每次 $O(H)$ 到 $O(N)$ 的遍历开销会很大。

优化方案：在节点中存储子树大小 (Size)

1. 修改数据结构：在 `TreeNode` 中增加一个属性 `size`，表示以该节点为根的子树的节点总数。
2. 维护 `size`：在每次执行 `insert` 或 `delete` 操作时，顺便更新路径上节点的 `size` 值（时间复杂度依然是 $O(H)$ ）。
3. 快速查询：
 - 令 `left_size = node.left.size`。
 - 如果 `k == left_size + 1`，则当前节点就是第 k 小。
 - 如果 `k <= left_size`，则在左子树中找第 k 小。
 - 如果 `k > left_size + 1`，则在右子树中找第 `k - left_size - 1` 小。

结果：查询操作的时间复杂度将从 $O(N)$ 降低到 $O(H)$ （ H 为树的高度）。如果配合平衡二叉树（如 AVL 或红黑树），查询复杂度可稳定在 $O(\log N)$ 。

6. 判断是否为平衡二叉树

一个二叉树是平衡的，如果它的左右两个子树的高度差不超过1。

```

1 class TreeNode:
2     def __init__(self, val=0, left=None, right=None):
3         self.val = val
4         self.left = left
5         self.right = right
6
7 def is_balanced(root):
8     def check_height(node):
9         if not node:
10             return 0
11
12         left_height = check_height(node.left)
13         if left_height == -1:
14             return -1 # Left subtree is unbalanced
15
16         right_height = check_height(node.right)
17         if right_height == -1:
18             return -1 # Right subtree is unbalanced
19
20         if abs(left_height - right_height) > 1:

```



```

21         return -1 # Current node is unbalanced
22
23         return max(left_height, right_height) + 1 # 返回当前节点所在子树的高度 (较大子树高度 + 1)
24
25     return check_height(root) != -1
26
27
28 # 构建如下所示的平衡二叉树:
29 #       3
30 #      / \
31 #     9  20
32 #    /  \
33 #   15   7
34
35 root_balanced = TreeNode(3)
36 root_balanced.left = TreeNode(9)
37 root_balanced.right = TreeNode(20, TreeNode(15), TreeNode(7))
38
39 print("Balanced Tree:", "Yes" if is_balanced(root_balanced) else "No")
40 # Balanced Tree: Yes
41
42 # 构建如下所示的不平衡二叉树:
43 #       1
44 #        \
45 #         2
46 #          \
47 #           3
48 #            \
49 #             4
50
51 root_unbalanced = TreeNode(1)
52 root_unbalanced.right = TreeNode(2)
53 root_unbalanced.right.right = TreeNode(3)
54 root_unbalanced.right.right.right = TreeNode(4)
55
56 print("Unbalanced Tree:", "Yes" if is_balanced(root_unbalanced) else "No")
57 # Unbalanced Tree: No

```

以上这些例子展示了如何通过递归优雅地解决一些常见的树问题。递归解法通常能够清晰地表达算法逻辑，使得代码更加容易理解和维护。不过，值得注意的是，在某些情况下，递归可能会导致栈溢出的问题，尤其是在处理非常深的树时。然而，通常树的题目不卡时间，因此递归写法通常是首选。

1.3 编程题目more

力扣热题100中，有15个题目是二叉树相关，是比重最大的一类题目。

二叉树	LeetCode 热题 15/100, https://leetcode.cn/studyplan/top-100-liked/
✔ 二叉树的中序遍历	简单
✔ 二叉树的最大深度	简单
✔ 翻转二叉树	简单
✔ 对称二叉树	简单
✔ 二叉树的直径	简单
✔ 二叉树的层序遍历	中等
✔ 将有序数组转换为二叉搜索树	简单
✔ 验证二叉搜索树	中等
✔ 二叉搜索树中第 K 小的元素	中等
✔ 二叉树的右视图	中等
☑ 二叉树展开为链表	中等
✔ 从前序与中序遍历序列构造二叉树	中等
✔ 路径总和 III	中等
✔ 二叉树的最近公共祖先	中等
✔ 二叉树中的最大路径和	困难

《算法笔记》配套晴问网站中，有46个题目是树相关，是比重最大的一类题目。

大纲

晴问编程题目

- › 入门模拟
- › 算法初步
- › 线性结构 (20题)
- › 搜索专题 (15题)
- › **树专题 (46题)**
 - › 1 树与二叉树 1题
 - › 2 二叉树的遍历 16题
 - › 3 树的遍历 7题
 - › 4 二叉查找树 (BST) 5题
 - › 5 平衡二叉树 (AVL树) 3题
 - › 6 并查集 5题
 - › 7 堆 6题
 - › 8 哈夫曼树 3题
- › 图算法专题 (33题)
- › 动态规划专题

sunnywhy_problems.md

树专题 (46题)

1 树与二叉树 1题

sy328: 树的判定

<https://sunnywhy.com/sfbj/9/1/328>

现有一个由个结点连接而成的连通结构，已知这个结构中存在的边数，问这个连通结构是否是一棵树。

输入

两个整数 n 、 m ($1 \leq n \leq 100, 0 \leq m \leq 100$)，分别表示结点数和边数。

输出

如果是一棵树，那么输出 **Yes**，否则输出 **No**。

样例1

输入

```
1 2 1
```

输出

```
1 Yes
```

2 表示法

https://blog.csdn.net/qq_41891805/article/details/104473065

2020-02-24

树是 n ($n \geq 0$) 个结点的有限集。在任意一棵非空树中：

- (1) 有且仅有一个根结点； (2) 除根结点外，其余的结点可分为 m ($m \geq 0$) 个互不相交的子树。

树的表示方法包括

2.1 嵌套括号表示法

Nested parentheses representation 是一种表示树结构的方法，通过括号的嵌套来表示树的层次关系。

先将根结点放入一对圆括号中，然后把它的子树按由左而右的顺序放入括号中，而对子树也采用同样方法处理：同层子树与它的根结点用圆括号括起来，同层子树之间用逗号隔开，最后用闭括号括起来。例如下图可写成如下形式

$(a(b, c, d, e))$

```
a
/ | | \
b c d e
```

练习M27637:括号嵌套二叉树

dfs, stack, <http://cs101.openjudge.cn/practice/27637>

可以用括号嵌套的方式来表示一棵二叉树。

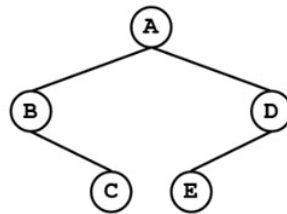
方法如下：

* 表示空的二叉树。如果一棵二叉树只有一个结点，则该树就用一个非* 字符表示，代表其根结点。

如果一棵二叉左右子树都非空，则用“树根(左子树,右子树)”的形式表示。树根是一个非'*'字符，左右子树之间用逗号隔开，没有空格。左右子树都用括号嵌套法表示。

如果左子树非空而右子树为空，则用 树根(左子树,*) 形式表示；如果左子树为空而右子树非空，则用 树根(*,右子树) 形式表示。

给出一棵树的括号嵌套表示形式，请输出其前序遍历序列、中序遍历序列、后序遍历序列。例如，A(B(*,C),D(E)) 表示的二叉树如图所示



输入

第一行是整数n表示有n棵二叉树(n<100) 接下来有n行，每行是1棵二叉树的括号嵌套表示形式

输出

对每棵二叉树，输出其前序遍历序列和中序遍历序列

样例输入

```
1 2
2 A
3 A(B(*,C),D(E,*))
```

样例输出

```
1 A
2 A
3 ABCDE
4 BCAED
```

来源：改编自 <http://dsbpython.openjudge.cn/dspythonbook/P0680/>

将输入的括号嵌套形式转换成二叉树，然后实现了前序和中序遍历。

```
1 class TreeNode:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6
7
8 def parse_tree(s):
9     """ 解析括号嵌套格式的二叉树 """
10    if s == '*': # 处理空树
11        return None
12    if '(' not in s: # 只有单个根节点
```

```

13         return TreeNode(s)
14
15     root_value = s[0] # 根节点值
16     subtrees = s[2:-1] # 去掉根节点和外层括号
17
18     # 使用栈找到逗号位置
19     stack = []
20     comma_index = None
21     for i, char in enumerate(subtrees):
22         if char == '(':
23             stack.append(char)
24         elif char == ')':
25             stack.pop()
26         elif char == ',' and not stack:
27             comma_index = i
28             break
29
30     left_subtree = subtrees[:comma_index] if comma_index is not None else subtrees
31     right_subtree = subtrees[comma_index + 1:] if comma_index is not None else None
32
33     root = TreeNode(root_value)
34     root.left = parse_tree(left_subtree) # 解析左子树
35     root.right = parse_tree(right_subtree) if right_subtree else None # 解析右子树
36     return root
37
38
39 def preorder_traversal(root):
40     """前序遍历: 根 -> 左 -> 右"""
41     return root.value + preorder_traversal(root.left) + preorder_traversal(root.right) if root
42     else ""
43
44 def inorder_traversal(root):
45     """中序遍历: 左 -> 根 -> 右"""
46     return inorder_traversal(root.left) + root.value + inorder_traversal(root.right) if root else
47     ""
48
49 # 读取输入
50 n = int(input().strip())
51 results = []
52
53 for _ in range(n):
54     tree_string = input().strip().replace(" ", "") # 去掉可能的空格
55     tree = parse_tree(tree_string)
56     results.append(preorder_traversal(tree))
57     results.append(inorder_traversal(tree))
58
59 print("\n".join(results)) # 按格式输出
60

```

优化后的版本（支持无括号单节点、自动跳过空树、递归清晰且无多余判断）。

优化思路

1. 字符流式解析

不再多次切片字符串，而是用一个指针（索引）在字符串中递归推进。

这样可以避免反复创建子串（ $O(n^2)$ → $O(n)$ ），在长输入下效率明显提升。

2. 结构清晰化

将解析逻辑封装成 `parse()` 内嵌函数（带索引控制），更直观。

3. 遍历统一处理

避免递归时频繁拼接字符串，用 `list` 收集结果后 `''.join()` 输出，更快。

4. 容错改进

自动忽略右子树缺省、星号、空括号等特殊情况。

优化版代码（精简高效 + $O(n)$ 解析）

```
1 class TreeNode:
2     __slots__ = ("val", "left", "right")
3     def __init__(self, val):
4         self.val = val
5         self.left = None
6         self.right = None
7
8
9 def parse_tree(s):
10     """将括号嵌套形式解析为二叉树，使用流式指针解析避免重复切片"""
11     i = 0
12     n = len(s)
13
14     def parse():
15         nonlocal i
16         if i >= n or s[i] == '*':
17             i += 1 # 跳过空树符号
18             return None
19
20         node = TreeNode(s[i])
21         i += 1
22
23         if i < n and s[i] == '(':
24             i += 1 # 跳过 '('
25             node.left = parse() # 左子树
26             if i < n and s[i] == ',':
27                 i += 1 # 跳过 ','
28                 node.right = parse() # 右子树
29             if i < n and s[i] == ')':
30                 i += 1 # 跳过 ')'
31         return node
32
33     return parse()
34
35
36 def preorder(root):
37     res = []
38     def dfs(node):
39         if not node: return
40         res.append(node.val)
41         dfs(node.left)
42         dfs(node.right)
43     dfs(root)
44     return ''.join(res)
45
46
47 def inorder(root):
48     res = []
49     def dfs(node):
```

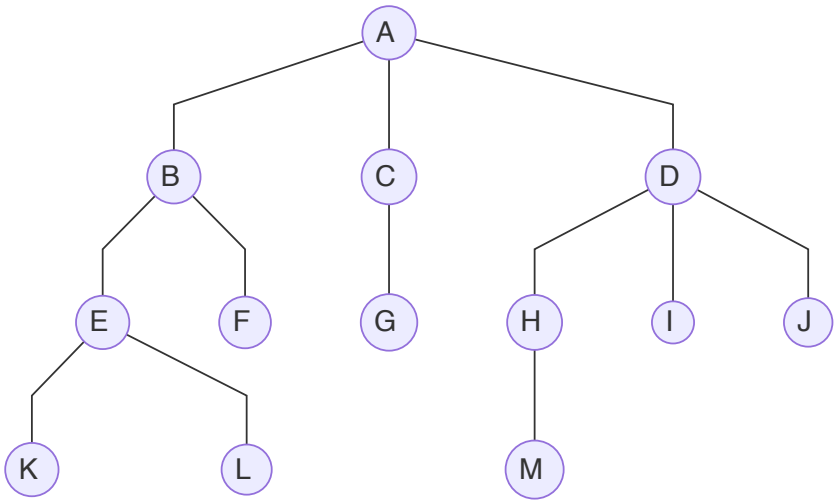
```
50     if not node: return
51     dfs(node.left)
52     res.append(node.val)
53     dfs(node.right)
54 dfs(root)
55 return ''.join(res)
56
57
58 # 主程序
59 if __name__ == "__main__":
60     n = int(input().strip())
61     for _ in range(n):
62         s = input().strip().replace(" ", "")
63         tree = parse_tree(s)
64         print(preorder(tree))
65         print(inorder(tree))
```

优点总结

优化项	原实现	优化后
解析复杂度	$O(n^2)$ (切片多次)	$O(n)$ (指针推进)
递归逻辑	需要查找逗号、维护栈	自动语法驱动，简洁直观
拼接性能	字符串相加	<code>list.append + ''.join()</code>
可读性	多层嵌套逻辑	层次清晰、容易扩展

2.2 树形表示

Node-Based or Node-Link structure
In computer science, a general tree is typically represented using this data structure. Each node in the tree contains information and references (links) to its child nodes.

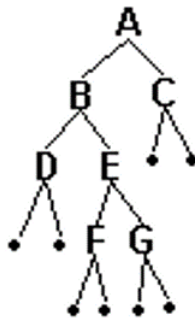


它的嵌套括号表示为：
`(A(B(E(K,L),F),C(G),D(H(M),I,J)))`

练习M08581: 扩展二叉树

<http://cs101.openjudge.cn/practice/08581/>

由于先序、中序和后序序列中的任一个都不能唯一确定一棵二叉树，所以对二叉树做如下处理，将二叉树的空结点用·补齐，如图所示。我们把这样处理后的二叉树称为原二叉树的扩展二叉树，扩展二叉树的先序和后序序列能唯一确定其二叉树。现给出扩展二叉树的先序序列，要求输出其中序和后序序列。



输入

扩展二叉树的先序序列（全部都由大写字母或者·组成）

输出

第一行：中序序列

第二行：后序序列

样例输入

```
1 ABD..EF..G..C..
```

样例输出

```
1 DBFEGAC
2 DFGEBCA
```

通过递归方法解析扩展二叉树的先序序列，并输出其中序和后序序列：

```
1 class Node:
2     def __init__(self, val):
3         self.val = val
4         self.left = None
5         self.right = None
6
7 def build_tree(s, index):
8     # 如果当前字符为 '.', 表示空结点, 返回None, 并将索引后移一位
9     if s[index] == '.':
10         return None, index + 1
11     # 否则创建一个结点
12     node = Node(s[index])
13     index += 1
14     # 递归构造左子树
```



```

15     node.left, index = build_tree(s, index)
16     # 递归构造右子树
17     node.right, index = build_tree(s, index)
18     return node, index
19
20 def inorder(node, res):
21     if node is None:
22         return
23     inorder(node.left, res)
24     res.append(node.val)
25     inorder(node.right, res)
26
27 def postorder(node, res):
28     if node is None:
29         return
30     postorder(node.left, res)
31     postorder(node.right, res)
32     res.append(node.val)
33
34 def main():
35     s = input().strip()
36     root, _ = build_tree(s, 0)
37
38     in_res = []
39     inorder(root, in_res)
40
41     post_res = []
42     postorder(root, post_res)
43
44     print("".join(in_res))
45     print("".join(post_res))
46
47 if __name__ == '__main__':
48     main()

```

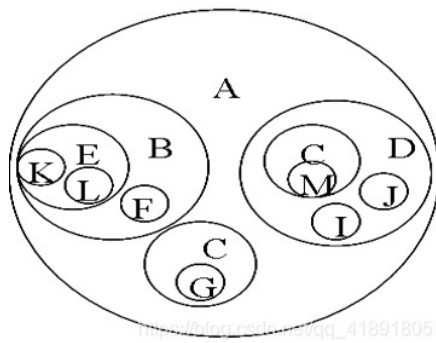
代码说明

- **build_tree 函数**
递归地根据扩展二叉树的先序序列构造二叉树：
 - 当遇到 '.' 时表示空结点，直接返回 None。
 - 否则以当前字符创建一个结点，然后递归构造其左子树和右子树。
- **inorder 和 postorder 函数**
分别实现中序遍历（左-根-右）和后序遍历（左-右-根）。
- **main 函数**
读取输入字符串，构造树后计算中序和后序遍历结果，并输出。

2.3 文氏图

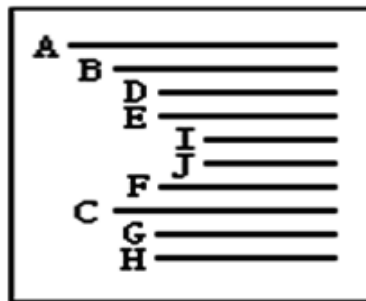
Venn diagram

Venn图是在所谓的集合论（或者类的理论）数学分支中，在不太严格的意义下用以表示集合（或类）的一种草图。



2.4 凹入表

Indented Tree Representation



练习T02775: 文件结构“图”

<http://cs101.openjudge.cn/practice/02775/>

在计算机上看到文件系统的结构通常很有用。Microsoft Windows上面的"explorer"程序就是这样的一个例子。但是在有图形界面之前，没有图形化的表示方法的，那时候最好的方式是把目录和文件的结构显示成一个"图"的样子，而且使用缩排的形式来表示目录的结构。比如：

```

1  ROOT
2  |   dir1
3  |   file1
4  |   file2
5  |   file3
6  |   dir2
7  |   dir3
8  |   file1
9  file1
10 file2

```

这个图说明：ROOT目录包括三个子目录和两个文件。第一个子目录包含3个文件，第二个子目录是空的，第三个子目录包含一个文件。

输入

你的任务是写一个程序读取一些测试数据。每组测试数据表示一个计算机的文件结构。每组测试数据以 `*` 结尾，而所有合理的输入数据以 `#` 结尾。一组测试数据包括一些文件和目录的名字（虽然在输入中我们没有给出，但是我们总假设ROOT目录是最外层的目录）。在输入中,以 `]` 表示一个目录的内容的结束。目录名字的第一个字母是'd'，文件名字的第一个字母是 `f`。文件名可能有扩展名也可能没有（比如 `fmyfile.dat` 和 `fmyfile`）。文件和目录的名字中都不包括空格,长度都不超过30。一个目录下的子目录个数和文件个数之和不超过30。

输出

在显示一个目录中内容的时候，先显示其中的子目录（如果有的话），然后再显示文件（如果有的话）。文件要求按照名字的字母表的顺序显示（目录不用按照名字的字母表顺序显示，只需要按照目录出现的先后显示）。对每一组测试数据，我们要先输出 `DATA SET x:`，这里 `x` 是测试数据的编号（从1开始）。在两组测试数据之间要输出一个空行来隔开。

你需要注意的是，我们使用一个 `|` 和5个空格来表示出缩排的层次。

样例输入

```
1 file1
2 file2
3 dir3
4 dir2
5 file1
6 file2
7 ]
8 ]
9 file4
10 dir1
11 ]
12 file3
13 *
14 file2
15 file1
16 *
17 #
```

样例输出

```
1 DATA SET 1:
2 ROOT
3 |   dir3
4 |   |   dir2
5 |   |   file1
6 |   |   file2
7 |   dir1
8 file1
9 file2
10 file3
11 file4
12
13 DATA SET 2:
14 ROOT
15 file1
16 file2
```

提示

- 一个目录和它的子目录处于不同的层次
- 一个目录和它的里面的文件处于同一层次

来源

翻译自 Pacific Northwest 1998 的试题

比较复杂的一道题目，首先是输入数据的处理就比较困难，需要先全部存储到tests中再逐行处理。构建文件树的过程中，需要一个stack存储待处理的目录，遇到“]”就弹出栈顶取出下一个待处理目录，file直接存入当前目录，dir则存入当前目录的同时压入栈中。输出时，则需要记录当前层数，按照层数输出前缀，并且目录理论上比文件要更深一层。

标准的 DFS 树结构构建 + 递归打印的写法。

```
1 class Node:
2     __slots__ = ('name', 'dirs', 'files')
3     def __init__(self, name):
4         self.name = name
5         self.dirs = []
6         self.files = []
7
8     def print_tree(node, depth=0):
9         prefix = '| ' * depth
10        print(prefix + node.name)
11        for d in node.dirs:          # 子目录按出现顺序
12            print_tree(d, depth + 1)
13        for f in sorted(node.files):  # 文件按字母序
14            print(prefix + f)
15
16    def parse_dataset(lines):
17        root = Node('ROOT')
18        stack = [root]
19        for s in lines:
20            if s[0] == 'd':          # 目录
21                new_dir = Node(s)
22                stack[-1].dirs.append(new_dir)
23                stack.append(new_dir)
24            elif s[0] == 'f':        # 文件
25                stack[-1].files.append(s)
26            elif s == ']':          # 目录结束
27                stack.pop()
28        return root
29
30    def main():
31        datasets, current = [], []
32        while True:
33            try:
34                s = input().strip()
35            except EOFError:
36                break
37            if s == '#':
38                if current:
39                    datasets.append(current)
40                break
41            elif s == '*':
42                datasets.append(current)
43                current = []
44            else:
45                current.append(s)
46
47        for i, data in enumerate(datasets, 1):
```

```

48         print(f'DATA SET {i}:')
49         print_tree(parse_dataset(data))
50         print()
51
52     if __name__ == "__main__":
53         main()

```

更轻量的类定义

- 使用 `__slots__` 节省内存，避免动态属性创建。

2.5 邻接表表示法

邻接表表示法（Adjacency List Representation）是一种常见的树的表示方法，特别适用于表示稀疏树（树中节点的度数相对较小）。适用于多叉树。

在邻接表表示法中，使用一个字典（或二维数组）来存储每个节点及其子节点列表，字典的键表示节点，对应的值是一个列表，存储节点的所有子节点。

下面是一个示例，展示了如何使用邻接表表示法表示一个树：

假设我们有以下树的结构：

```

1      A
2     / | \
3    B  C  D
4   / \   \
5  E  F   G
6     \
7      H

```

使用邻接表表示法，我们可以得到如下的表示：

```

1  A: ['B', 'C', 'D']
2  B: ['E', 'F']
3  E: []
4  F: ['H']
5  H: []
6  C: []
7  D: ['G']
8  G: []

```

这种表示方法可以方便地查询任意节点的子节点，并且结构清晰、存储高效。

```

1  class TreeNode:
2      def __init__(self, value):
3          self.value = value
4          self.children = []
5
6  def build_tree():
7      # 创建树节点
8      root = TreeNode('A')
9      node_b = TreeNode('B')
10     node_c = TreeNode('C')
11     node_d = TreeNode('D')

```

```

12     node_e = TreeNode('E')
13     node_f = TreeNode('F')
14     node_g = TreeNode('G')
15     node_h = TreeNode('H')
16
17     # 构建树的结构
18     root.children.extend([node_b, node_c, node_d])
19     node_b.children.extend([node_e, node_f])
20     node_d.children.append(node_g)
21     node_f.children.append(node_h)
22
23     return root
24
25 def print_tree_adjacency_list(root):
26     adjacency_list = {}
27
28     # 递归构建邻接表
29     def build_adjacency_list(node):
30         adjacency_list[node.value] = [child.value for child in node.children]
31         for child in node.children:
32             build_adjacency_list(child)
33
34     # 构建并打印邻接表
35     build_adjacency_list(root)
36     for node, children in adjacency_list.items():
37         print(f"{node}: {children}")
38
39 # 构建树
40 root_node = build_tree()
41
42 # 打印邻接表表示的树
43 print_tree_adjacency_list(root_node)
44
45 """
46 A: ['B', 'C', 'D']
47 B: ['E', 'F']
48 E: []
49 F: ['H']
50 H: []
51 C: []
52 D: ['G']
53 G: []
54 """

```

邻接表，不陌生。

18182: 打怪兽，data structures, <http://cs101.openjudge.cn/practice/18182/>

用字典模拟邻接表，存储相同时刻的技能。

E23555: 节省存储的矩阵乘法

matrices, <https://cs101.openjudge.cn/pctbook/E23555/>

练习M07161: 森林的带度数层次序列存储 (assign#6)

dfs + bfs, <http://cs101.openjudge.cn/practice/07161/>

07161: 森林的带度数层次序列存储是 DFS + BFS 结合的题目。

利用队列根据“带度数的层次序列”构造树，然后进行后根遍历。代码如下：

```
1  from collections import deque
2  import sys
3
4  # 定义树的结点
5  class Node:
6      def __init__(self, value, degree):
7          self.value = value
8          self.degree = degree
9          self.children = []
10
11 # 根据带度数的层次序列构造树
12 def build_tree(tokens):
13     # tokens 的格式: [字母, 度数, 字母, 度数, ...]
14     # 第一个结点为根
15     root = Node(tokens[0], int(tokens[1]))
16     queue = deque([root])
17     index = 2 # 下一个待处理的token索引
18     while queue and index < len(tokens):
19         current = queue.popleft()
20         # current.degree 个孩子依次出现在 tokens 中
21         for _ in range(current.degree):
22             # 每个孩子由两个元素构成: 字母和度数
23             child = Node(tokens[index], int(tokens[index+1]))
24             current.children.append(child)
25             queue.append(child)
26             index += 2
27     return root
28
29 # 后根遍历 (后序遍历): 先遍历所有子树, 再访问根节点
30 def postorder(node, output):
31     for child in node.children:
32         postorder(child, output)
33     output.append(node.value)
34
35 def main():
36     input_lines = sys.stdin.read().splitlines()
37     if not input_lines:
38         return
39     n = int(input_lines[0].strip())
40     result = []
41     # 对于每一棵树进行构造并后序遍历
42     for i in range(1, n+1):
43         # 将一行的内容按空格分割
44         tokens = input_lines[i].split()
45         if not tokens:
46             continue
47         root = build_tree(tokens)
48         temp = []
49         postorder(root, temp)
50         result.extend(temp)
51     # 输出时空格分隔各个结点
52     print(" ".join(result))
53
```

```
54 if __name__ == "__main__":
55     main()
```

练习M24729: 括号嵌套树 (assign#7)

<http://cs101.openjudge.cn/practice/24729/>

邻接表实现，主要步骤：

1. 解析括号嵌套表示：用栈来构建树结构，每次遇到字母创建节点，遇到 `)` 弹出栈顶节点表示子树结束。
2. 前序遍历：根节点 -> 依次遍历子节点。
3. 后序遍历：依次遍历子节点 -> 根节点。

下面两个代码。先给出用类表示node

```
1 class TreeNode:
2     def __init__(self, value): #类似字典
3         self.value = value
4         self.children = []
5
6 def parse_tree(s):
7     stack = []
8     node = None
9     for char in s:
10        if char.isalpha(): # 如果是字母，创建新节点
11            node = TreeNode(char)
12            if stack: # 如果栈不为空，把节点作为子节点加入到栈顶节点的子节点列表中
13                stack[-1].children.append(node)
14        elif char == '(': # 遇到左括号，当前节点可能会有子节点
15            if node:
16                stack.append(node) # 把当前节点推入栈中
17                node = None
18        elif char == ')': # 遇到右括号，子节点列表结束
19            if stack:
20                node = stack.pop() # 弹出当前节点
21        return node # 根节点
22
23
24 def preorder(node):
25     output = [node.value]
26     for child in node.children:
27         output.extend(preorder(child))
28     return ''.join(output)
29
30 def postorder(node):
31     output = []
32     for child in node.children:
33         output.extend(postorder(child))
34     output.append(node.value)
35     return ''.join(output)
36
37 # 主程序
38 def main():
39     s = input().strip()
```



```

40     s = ''.join(s.split()) # 去掉所有空白字符
41     root = parse_tree(s) # 解析整棵树
42     if root:
43         print(preorder(root)) # 输出前序遍历序列
44         print(postorder(root)) # 输出后序遍历序列
45     else:
46         print("input tree string error!")
47
48 if __name__ == "__main__":
49     main()

```

用字典表示node

```

1 def parse_tree(s):
2     stack = []
3     node = None
4     for char in s:
5         if char.isalpha(): # 如果是字母, 创建新节点
6             node = {'value': char, 'children': []}
7             if stack: # 如果栈不为空, 把节点作为子节点加入到栈顶节点的子节点列表中
8                 stack[-1]['children'].append(node)
9             elif char == '(': # 遇到左括号, 当前节点可能会有子节点
10                 if node:
11                     stack.append(node) # 把当前节点推入栈中
12                     node = None
13             elif char == ')': # 遇到右括号, 子节点列表结束
14                 if stack:
15                     node = stack.pop() # 弹出当前节点
16     return node # 根节点
17
18
19 def preorder(node):
20     output = [node['value']]
21     for child in node['children']:
22         output.extend(preorder(child))
23     return ''.join(output)
24
25 def postorder(node):
26     output = []
27     for child in node['children']:
28         output.extend(postorder(child))
29     output.append(node['value'])
30     return ''.join(output)
31
32 # 主程序
33 def main():
34     s = input().strip()
35     s = ''.join(s.split()) # 去掉所有空白字符
36     root = parse_tree(s) # 解析整棵树
37     if root:
38         print(preorder(root)) # 输出前序遍历序列
39         print(postorder(root)) # 输出后序遍历序列
40     else:
41         print("input tree string error!")
42
43 if __name__ == "__main__":

```

练习M27928:遍历树 (assign#6)

dfs + sortings, <http://cs101.openjudge.cn/practice/27928/>

请你对输入的树做遍历。遍历的规则是：遍历到每个节点时，按照该节点和所有子节点的值从小到大进行遍历，例如：

```
1 |      7
2 |  /   |   \
3 | 10   3   6
```

对于这个树，你应该先遍历值为3的子节点，然后是值为6的子节点，然后是父节点7，最后是值为10的子节点。

本题中每个节点的值互不相同的正整数，最大不超过9999999。

输入

第一行：节点个数n (n<500)

接下来的 n 行：第一个数是此节点的值，之后的数分别表示它的所有子节点的值。每个数之间用空格隔开。如果没有子节点，该行便只有一个数。

输出

输出遍历结果，一行一个节点的值。

样例输入

```
1 | sample1 input:
2 | 4
3 | 7 10 3 6
4 | 10
5 | 6
6 | 3
7 |
8 | sample1 output:
9 | 3
10 | 6
11 | 7
12 | 10
```

样例输出

```
1 | sample2 input:
2 | 6
3 | 10 3 1
4 | 7
5 | 9 2
6 | 2 10
7 | 3 7
8 | 1
9 |
10 | sample2 output:
11 | 2
12 | 1
13 | 3
```

```
14 | 7
15 | 10
16 | 9
```

来源：2024spring zht

总体思路分为三步：1.通过字典建立输入数据的父子关系；2.找到树的根（这里我将父节点和子节点分别用两个列表记录，最后使用集合减法）；3.通过递归实现要求的从小到大遍历。

```
1  from collections import defaultdict
2  import sys
3  sys.setrecursionlimit(10000)
4
5  def main():
6      n = int(sys.stdin.readline())
7      tree = defaultdict(list)
8      all_nodes = set()
9      child_nodes = set()
10
11     for _ in range(n):
12         parts = list(map(int, sys.stdin.readline().split()))
13         parent, *children = parts
14         tree[parent].extend(children)
15         all_nodes.add(parent)
16         all_nodes.update(children)
17         child_nodes.update(children)
18
19     # 根节点 = 出现在 all_nodes 但没出现在 child_nodes 的那个
20     root = (all_nodes - child_nodes).pop()
21
22     def traverse(u):
23         # 把 u 自己和它的所有直接孩子放一起排序
24         group = tree[u] + [u]
25         group.sort()
26         for x in group:
27             if x == u:
28                 print(u)
29             else:
30                 traverse(x)
31
32     traverse(root)
33
34 if __name__ == "__main__":
35     main()
36
```

Height of a generic tree from parent array

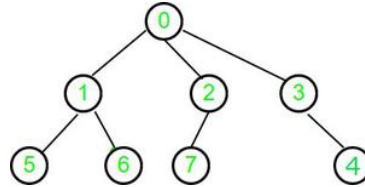
<https://www.geeksforgeeks.org/height-generic-tree-parent-array/>

Last Updated : 01 Nov, 2024

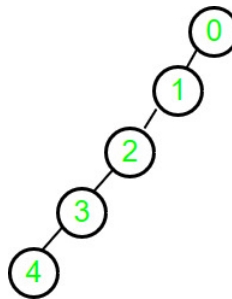
给定一棵大小为 n 的树，以数组 `parent[0..n-1]` 的形式表示，其中 `parent[i]` 中的每个索引 i 代表一个节点，而 i 处的值表示该节点的直接父节点。对于根节点，其值为 `-1`。根据给定的父节点链接，求这棵通用树的高度。

Examples:

```
1 Input : parent[] = [-1, 0, 0, 0, 3, 1, 1, 2]
2 Output : 2
```



```
1 Input : parent[] = [-1, 0, 1, 2, 3]
2 Output : 4
```



Here, a **generic tree** is sometimes also called an N-ary tree or N-way tree where N denotes the maximum number of child a node can have. In this problem, the array represents n number of nodes in the tree.

朴素的方法是从节点向上遍历树，直到到达值为 `-1` 的根节点。在遍历每个节点时，记录最大路径长度。该解决方案的时间复杂度为 $O(n^2)$ 。

[Expected Approach - 1] Using BFS - $O(n)$ Time and $O(n)$ Space

Build **graph** for **N-ary Tree** in $O(n)$ time and apply **BFS** on the stored graph in $O(n)$ time and while doing BFS store **maximum reached level**. This solution does **two iterations** to find the **height** of N-ary tree.

Implementation:

```
1 # find height of N-ary tree in O(n)
2 from collections import deque
3
4 MAX = 1001
5 adj = [[] for i in range(MAX)] # Adjacency list to store N-ary tree
6
7 def build_tree(arr, n): # Build tree in tree in O(n)
8     root_index = 0
9
10    for i in range(n):
11        if (arr[i] == -1): # if root node, store index
```

```

12         root_index = i
13     else:
14         adj[i].append(arr[i]) # 子 -> 父
15         adj[arr[i]].append(i) # 父 -> 子
16
17     return root_index
18
19
20 def BFS(start):
21     vis = {} # map is used as visited array
22
23     q = deque()
24     max_level_reached = 0
25
26     q.append([start, 0]) # height of root node is zero
27
28     # p[0] denotes node in adjacency list
29     # p[1] denotes level of p[0]
30     p = []
31
32     while (len(q) > 0):
33         p = q.popleft()
34         vis[p[0]] = 1
35
36         max_level_reached = max(max_level_reached, p[1])
37
38         for i in range(len(adj[p[0]])):
39             if (adj[p[0]][i] not in vis):
40                 q.append([adj[p[0]][i], p[1] + 1]) # adding 1 to pre_level
41
42     return max_level_reached
43
44
45 # Driver code
46 if __name__ == '__main__':
47     parent = [-1, 0, 1, 2, 3] # node 0 to node n-1
48     n = len(parent) # Number of nodes in tree
49
50     root_index = build_tree(parent, n)
51     ma = BFS(root_index)
52     print("Height of N-ary Tree =", ma)
53
54 # output: Height of N-ary Tree = 4
55

```

建无向边是为了“懒”——省去显式找子节点的步骤，靠 BFS 的 `vis` 来控制方向。

Time Complexity: $O(n)$ which converges to $O(n)$ for very large n .

Auxiliary Space: $O(n)$, we are using an **adjacency list** to store the tree in memory. The size of the adjacency list is proportional to the number of nodes in the tree, so the space complexity of the algorithm is $O(n)$.

[Expected Approach – 2] Without using map – $O(n)$ Time and $O(n)$ Space

我们可以在一次迭代中找到 **N 叉树的高度**。我们以迭代方式访问从 **0 到 n-1** 的所有节点，并递归地标记那些尚未被访问过的**祖先节点**，直到到达一个已被访问过的节点，或者到达根节点。如果在通过父节点链接向上遍历树时遇到一个已访问过的节点，那么我们会使用它的高度，并停止进一步的递归。

```

1  # find height of N-ary tree in O(n) (Efficient Approach)
2
3  # Recur For Ancestors of node and store height of node at last
4  def fillHeight(p, node, visited, height):
5      if (p[node] == -1): # If root node
6          visited[node] = 1 # mark root node as visited
7          return 0
8
9      if (visited[node]):
10         return height[node]
11
12     visited[node] = 1
13
14     # recur for the parent node
15     height[node] = 1 + fillHeight(p, p[node], visited, height)
16
17     # return calculated height for node
18     return height[node]
19
20
21 def findHeight(parent):
22     n = len(parent)
23     maxHeight = 0
24     visited = [0] * n
25     height = [0] * n
26
27     for i in range(n):
28         if (not visited[i]):
29             height[i] = fillHeight(parent, i, visited, height)
30
31             maxHeight = max(maxHeight, height[i])
32
33     return maxHeight
34
35
36 if __name__ == '__main__':
37     parent = [-1, 0, 0, 0, 3, 1, 1, 2]
38     n = len(parent)
39
40     print(findHeight(parent))
41
42 # 2
43

```

Time Complexity: $O(n)$

Auxiliary Space: $O(n)$, this is because we need to store the visited and height arrays which are of size n .

General Tree Level Order Traversal层序遍历

<https://www.geeksforgeeks.org/generic-tree-level-order-traversal/>

Last Updated : 14 Mar, 2023

树的遍历，除了前序、中序、后序，还有一个是层序遍历，就是使用队列的bfs方式。

Given a generic tree, perform a Level order traversal and print all of its nodes

Examples:

```

1 Input :          10
2           /   /   \   \
3          2  34  56  100
4         / \      |  / | \
5        77 88      1  7  8  9
6
7 Output : 10
8          2 34 56 100
9          77 88 1 7 8 9
10
11 Input :          1
12           /   /   \   \
13          2  3  4   5
14         / \      |  / | \
15        6  7      8 9 10 11
16 Output : 1
17          2 3 4 5
18          6 7 8 9 10 11

```

The approach to this problem is similar to [Level Order traversal in a binary tree](#). We Start with pushing root node in a queue and for each node we pop it, print it and push all its child in the queue.

In case of a generic tree we store child nodes in a vector. Thus we put all elements of the vector in the queue.

实现代码，我做了部分调整，主要是用deque

```

1 # do level order traversal of a generic tree
2 from collections import deque
3
4 # Represents a node of an n-ary tree
5 class Node:
6     def __init__(self, key):
7         self.key = key
8         self.children = []
9
10
11 def create_node(key):
12     """Helper function to create a new node."""
13     return Node(key)
14
15
16 def level_order_traversal(root):
17     """
18     Perform level order traversal (BFS) of a generic n-ary tree.
19     Prints nodes level by level.
20     """
21     if root is None:
22         return
23
24     queue = deque([root]) # Initialize the queue with the root node
25
26     while queue:
27         level_size = len(queue) # Number of nodes at the current level
28
29         # Process all nodes in the current level
30         for _ in range(level_size):
31             current_node = queue.popleft() # Dequeue the front node
32             print(current_node.key, end=' ') # Print the node's key

```

```

33
34         # Enqueue all children of the current node
35         for child in current_node.children:
36             queue.append(child)
37
38         print() # Separate levels with a newline
39
40
41 # Driver program
42 if __name__ == '__main__':
43     '''
44     Create the following n-ary tree:
45           10
46        /  / \ \
47       2 34 56 100
48      / \      | / | \
49     77 88     1 7 8 9
45
50     '''
51     root = create_node(10)
52     root.children.append(create_node(2))
53     root.children.append(create_node(34))
54     root.children.append(create_node(56))
55     root.children.append(create_node(100))
56
57     root.children[0].children.append(create_node(77))
58     root.children[0].children.append(create_node(88))
59
60     root.children[2].children.append(create_node(1))
61
62     root.children[3].children.append(create_node(7))
63     root.children[3].children.append(create_node(8))
64     root.children[3].children.append(create_node(9))
65
66     print("Level order traversal of the tree:")
67     level_order_traversal(root)
68
69     """
70     Level order traversal Before Mirroring
71     10
72     2 34 56 100
73     77 88 1 7 8 9
74     """
75

```

Time Complexity: $O(n)$ where n is the number of nodes in the n -ary tree.

Auxiliary Space: $O(n)$

Q: 本周题目我认为最有学习价值的一点在于，可以通过邻接表模拟树，这样可以节省很多内存。在python里，邻接表就是字典或者二维数组吧？

你的理解非常准确！在 Python 中，实现邻接表（Adjacency List）最常用的方式确实是列表（嵌套列表）或字典。

相比于邻接矩阵（Adjacency Matrix，即 $n \times n$ 的二维数组），邻接表在处理树结构或稀疏图时，内存优势极其巨大。

以下是 Python 中几种常见的邻接表实现方式及其对比：

1. 使用嵌套列表（最常用）

如果你的节点编号是连续的整数（例如 0 到 $n - 1$ ），这是最简单且运行速度最快的方法。

```
1 # 假设有 n 个节点
2 n = 5
3 adj = [[] for _ in range(n)]
4
5 # 添加一条边 (u, v)
6 adj[u].append(v)
7 adj[v].append(u) # 如果是无向图/树
```

- 优点：访问速度极快 ($O(1)$ 索引)，内存连续。
- 缺点：节点编号必须是整数。

2. 使用 `collections.defaultdict(list)`（最推荐，最灵活）

当你不知道节点具体数量，或者节点编号不连续、甚至节点是字符串时，这是最“Pythonic”的做法。

```
1 from collections import defaultdict
2
3 adj = defaultdict(list)
4
5 # 添加边，无需提前初始化大小
6 adj[u].append(v)
7 adj['A'].append('B') # 节点可以是任意不可变类型
```

- 优点：不需要预先知道 n 的大小；不会因为访问不存在的键而报错（自动创建空列表）。
- 缺点：比纯列表稍微慢一点点（微乎其微）。

3. 使用字典（普通 `dict`）

```
1 adj = {}
2 # 需要手动判断键是否存在
3 if u not in adj:
4     adj[u] = []
5 adj[u].append(v)
```

- 特点：和 `defaultdict` 类似，但代码会略显繁琐。

为什么邻接表比邻接矩阵省钱（内存）？

假设我们要存一棵有 10^5 个节点的树：

1. 邻接矩阵 (2D Array)：

- 你需要创建一个 $10^5 \times 10^5$ 的二维数组。
- 即便每个元素只占 1 字节，也需要大约 **10GB** 的内存。
- 但在树中，其实只有 $n - 1$ （即 99,999）条边是有意义的，其余 9,999,900,001 个位置全是 0（浪费）。

2. 邻接表：

- 你只存储存在的边。
- 树有 n 个节点和 $n - 1$ 条边。
- 空间复杂度是 $O(V + E)$ ，即 $O(n)$ 。
- 对于 10^5 个节点的树，内存占用通常只有 **几 MB 到几十 MB**。

总结

在算法面试或竞赛中：

- 如果节点是 $0 \sim n-1$ ，直接用 `adj = [[] for _ in range(n)]`。
- 如果节点编号分散，用 `adj = defaultdict(list)`。
- 永远不要在节点数超过 5000 的时候尝试用 `matrix = [[0]*n for _ in range(n)]` 来模拟树。

你提到的“2D数组”：

需要注意的是，在 Python 中 `adj = [[...] for _ in range(n)]` 严格来说是列表的列表。由于内层列表的长度是动态的（这被称为 **Jagged Array**，参差阵），所以它只存储了实际存在的边，而不是一个规整的矩形。这正是它节省内存的精髓所在！

二、构建、遍历、哈夫曼、解析树（性质和二叉树应用）

3 树的基本性质

3.1 m -叉树的性质

1. **总结点数**：设树中总的节点数为 N ，则：

$$N = N_0 + N_1 + N_2 + \cdots + N_m \quad (1)$$

其中 N_0 表示度数为 0 的节点数（即叶结点）， $N_1, N_2, \dots, N_m$ 分别表示度数为 1, 2, ..., m 的节点数。

2. **总分支数**：在树中，每个度数为 k 的节点贡献了 k 条分支。因此，树中的总分支数（边数）可以表示为：

$$\text{总分支数} = 1 \cdot N_1 + 2 \cdot N_2 + \cdots + m \cdot N_m \quad (2)$$

3. **树的总分支数与总结点数的关系**：对于一棵树，总分支数等于总结点数减去 1（因为除了根节点外，每个节点都有一条分支连接到其父节点）。因此：

$$\text{总分支数} = N - 1 \quad (3)$$

将以上关系结合起来，可以得到关于 N_0 的方程。

对任何一棵非空二叉树，如果叶节点数 n_0 ，度为 2 的非叶节点数 n_2 ，则：

$$n_0 = n_2 + 1$$

在一棵二叉树中，除了度为 0 的叶子节点，就是度为 1 的节点和度为 2 的节点，则树的总节点数为：

$$N = n_0 + n_1 + n_2$$

在二叉树中节点总数为 N 和边数之间的关系是：**边数 = $N - 1$** ，所以有：

$$n_0 + n_1 + n_2 - 1 = 2n_2 + n_1$$

最后得到： $n_0 = n_2 + 1$

笔试填空@20240618

设一棵 m 叉树中有 N_1 个度数为 1 的节点（度数表示子节点个数）， N_2 个度数为 2 的节点，.....， N_m 个度数为 m 的节点，则该 m 叉树中共有 _____ 个终端结点（即叶结点）。 **$1 + \sum_{i=2}^m (i-1)N_i$**

从树的总结点数和总分支数两条性质出发：

1. 总分支数的两种表达式相等：

$$1 \cdot N_1 + 2 \cdot N_2 + \cdots + m \cdot N_m = N - 1 \quad (4)$$

2. 将 $N = N_0 + N_1 + N_2 + \cdots + N_m$ 代入上式:

$$1 \cdot N_1 + 2 \cdot N_2 + \cdots + m \cdot N_m = (N_0 + N_1 + N_2 + \cdots + N_m) - 1 \quad (5)$$

3. 化简后得到:

$$N_0 = 1 + (0 \cdot N_0) + (1 \cdot N_1) + (2 \cdot N_2) + \cdots + (m \cdot N_m) - (N_1 + N_2 + \cdots + N_m) \quad (6)$$

4. 合并同类项:

$$N_0 = 1 + \sum_{k=1}^m (k-1)N_k \quad (7)$$

最终公式

一棵 m -叉树中, 叶结点的数量 N_0 可以表示为:

$$N_0 = 1 + \sum_{k=1}^m (k-1)N_k \quad (8)$$

其中:

- N_k 是度数为 k 的结点数;
- $k-1$ 表示每个度数为 k 的结点对叶结点数量的贡献。

3.2 二叉树的性质

1. **每个节点最多有两个子节点**: 二叉树中每个节点最多有两个子节点, 分别是“左子节点”和“右子节点”。
2. **二叉树的高度与深度**:
 - **树的高度**: 从根节点到叶子节点的最长路径的长度 (即经过的边的数量)。空树的高度为-1, 只有一个节点时树的高度为0。
 - **节点的深度**: 从根节点到该节点的路径长度 (即经过的边的数量)。根节点的深度为0, 其他节点的深度是其父节点的深度加1。
3. **二叉树的节点数**: 对于一般二叉树, 节点数目是有限的, 可以通过遍历计算。
4. **满二叉树**: 满二叉树是一种特殊的二叉树, 其中每一层的节点数都达到最大值 (即每个非叶子节点都有两个子节点)。假设树的高度为 h , 则满二叉树的节点数为 $2^{(h+1)} - 1$ 。
5. **完全二叉树**: 完全二叉树是一种特殊的二叉树, 其中每一层的节点都被完全填满, 且最底层的节点集中在左边。如果一个完全二叉树有 n 个节点, 那么它的高度为 $\lfloor \log_2(n) \rfloor$ (向下取整)。
6. **叶子节点和非叶子节点**:
 - **叶子节点**: 没有子节点的节点。
 - **非叶子节点**: 有一个或两个子节点的节点。
7. **二叉树的遍历方式**:
 - **前序遍历 (Pre-order)**: 根节点 -> 左子树 -> 右子树
 - **中序遍历 (In-order)**: 左子树 -> 根节点 -> 右子树
 - **后序遍历 (Post-order)**: 左子树 -> 右子树 -> 根节点

- **层序遍历 (Level-order)**：从上到下、从左到右逐层访问节点。

8. **二叉树的平衡性**：如果一个二叉树的左右子树的高度差的绝对值大于1，那么这个二叉树就是**不平衡的**。如果一个二叉树在任何时刻都满足左右子树高度差不超过1，那么它就是**平衡二叉树**。

这些性质是理解二叉树结构的基础，有助于设计和优化基于二叉树的算法，比如二叉查找树、平衡树等。

笔试选择@20240618

Q. 定义一棵没有1度结点的二叉树为满二叉树。对于一棵包含k个结点的满二叉树，其叶子结点的个数为（**C**）。

A. $\lfloor k/2 \rfloor$ B. $\lfloor k/2 \rfloor - 1$ C. $\lfloor k/2 \rfloor + 1$ D. 以上三个都有可能

笔试填空@20230620

Q. 定义二叉树中一个结点的度数为其子结点的个数。现有一棵结点总数为 101 的二叉树，其中度数为 1 的结点数有 30 个，则度数为 0 结点有 ____ 个。

推导过程

1. 根据总结点数公式：

$$N = N_0 + N_1 + N_2$$

将已知值代入：

$$101 = N_0 + 30 + N_2$$

化简得：

$$N_0 + N_2 = 71 \quad (1)$$

2. 根据总分支数公式：

$$N_1 + 2N_2 = N - 1$$

将已知值代入：

$$30 + 2N_2 = 101 - 1$$

化简得：

$$2N_2 = 70 \Rightarrow N_2 = 35 \quad (2)$$

3. 将 $N_2 = 35$ 代入 (1) 式：

$$N_0 + 35 = 71$$

化简得：

$$N_0 = 36$$

36

笔试填空@20230620

Q. 定义完全二叉树的根结点所在层为第一层。如果一个完全二叉树的第六层有 23 个叶结点，则它的总结点数可能为 ____（请填写所有 3 个可能的结点数，写对 1 个得 1 分，2 个得 1.5 分，写错 1 个不得分）。

54，是 $1+2+4+8+16+23=54$

在完全二叉树中，除了最底层，其他每一层的节点都是满的。在最底层，从左往右填充节点。

可以是第六层是满的。但是叶节点只有23个，还有9个节点是非叶节点。这样第七层要么是17个节点要么是18个。

$(1+2+4+8+16+32) + 17$ 或 $18 = 80$ ，或 81

54,80,81

4 二叉树的应用

4.1 解析树/抽象语法树AST

现在来看看如何用树解决一些实际问题。本节介绍解析树，可以用它来表示现实世界中像句子（如图1所示）或数学表达式这样的构造。

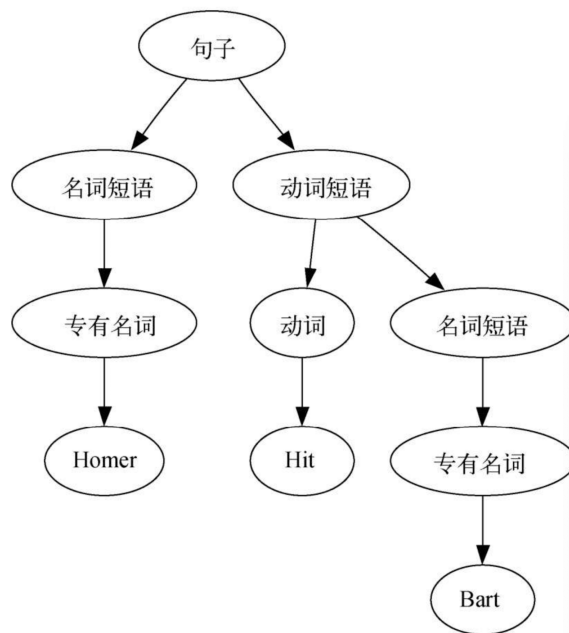


图1 一个简单句子的解析树

图1展示了一个简单句子的层次结构。用树状结构表示句子可以使用子树处理句子的独立部分。

也可以将 $((7 + 3) * (5 - 2))$ 这样的数学表达式表示成解析树，如图2所示。这是完全括号表达式，乘法的优先级高于加法和减法，但因为括号，所以在做乘法前必须先做括号内的加法和减法。树的层次性有助于理解整个表达式的计算次序。在计算顶层的乘法前，必须先计算子树中的加法和减法。加法（左子树）的结果是10，减法（右子树）的结果是3。利用树的层次结构，在计算完子树的表达式后，只需用一个节点代替整棵子树即可。应用这个替换过程后，便得到如图3所示的简化树。

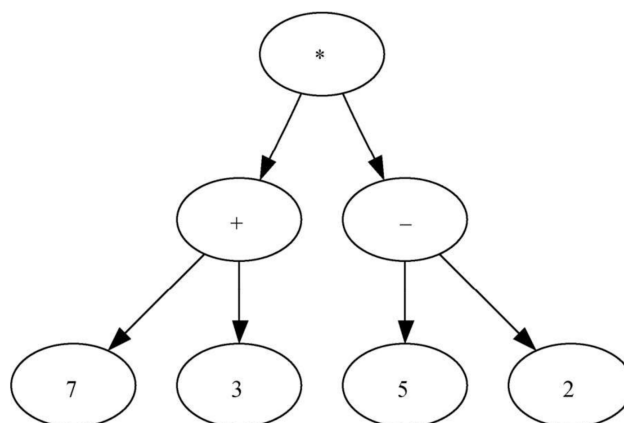


图2 $((7 + 3) * (5 - 2))$ 的解析树

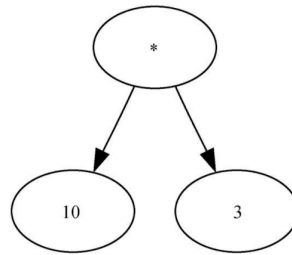


图3 $((7 + 3) * (5 - 2))$ 的简化解析树

本节的剩余部分将仔细考察解析树，重点如下：

- 如何根据完全括号表达式构建解析树；
- 如何计算解析树中的表达式；
- 如何将解析树还原成最初的数学表达式。

Q: 想确认下根据完全括号表达式构建解析式是否有一个算法名称？

gpt：完全括号表达式，通常称为“完全圆括号表达式”或“完全括号化表达式”，是数学和计算机科学中的一个概念，用于明确无误地表示带有括号的算术表达式或其他任何需要操作符优先级的表达式。这种表达式形式消除了操作符优先规则的歧义，因为所有的操作都明确地通过括号来指示其顺序。

构建解析树（或抽象语法树，Abstract Syntax Tree，AST）的过程，从给定的完全括号表达式中提取结构信息，并创建相应的树形结构，这在计算机科学中通常是语法分析（Parsing）的一部分。针对此过程的算法没有特定的通用名称，但它是编译器和解释器中使用的语法分析或解析算法的一部分。

树形结构能够体现操作符和操作数之间的层级关系，而表达式的解析过程通常遵循以下步骤：

1. **词法分析（Lexical Analysis）**：此步骤中，输入的字符序列（代码、表达式等）被转换成一系列标记（tokens），例如数字、操作符、括号等。
2. **语法分析（Syntactic Analysis）**：在该阶段，标记序列根据语法规则被组织成解析树。语法分析器会使用诸如递归下降、LL 解析、LR 解析等算法来构造树。
3. **语义分析（Semantic Analysis）**：构建的解析树可能会进一步经过语义检查，以确保表达式的意义是合理的，比如类型检查等。

对于构建解析树的具体算法，没有一个统一的名称来描述这个过程，因为它可能会涉及不同的解析策略和技术。在编译器设计中，常见的解析算法有LL解析器、LR解析器、递归下降解析器等。这些算法都可以用来从完全括号表达式中构建出解析树。

Gemini: 根据完全括号表达式构建解析树，最准确的术语描述是：**基于栈的表达式树构建（Stack-based Expression Tree Construction）**。

该算法的核心逻辑通常如下：

1. 遇到 **(**：新建左子节点，入栈，下降到左子。
2. 遇到 **数字**：填充当前节点值，返回父节点（出栈）。
3. 遇到 **运算符**：填充当前节点值，新建右子节点，入栈，下降到右子。
4. 遇到 **)**：返回父节点（出栈）。

构建解析树的第一步是将表达式字符串拆分成标记列表，即**词法分析**。需要考虑4种标记：**左括号、右括号、运算符和操作数**。左括号代表新表达式的起点，所以应该创建一棵对应该表达式的新树。反之，遇到右括号则意味着到达该表达式的终点。操作数既是叶子节点，也是其运算符的子节点。此外，每个运算符都有左右子节点。

有了上述信息，便可以定义以下**4条规则**：

- (1) 如果当前标记是 `(`，就为当前节点添加一个左子节点，入栈，并下沉至该子节点；
- (2) 如果当前标记在列表 `['+', '-', '/', '*']` 中，就将当前节点的值设为当前标记对应的运算符；为当前节点添加一个右子节点，入栈，并下沉至该子节点；
- (3) 如果当前标记是数字，就将当前节点的值设为这个数，并返回至父节点（出栈）；
- (4) 如果当前标记是 `)`，就跳到当前节点的父节点（出栈）。

编写代码前，先通过一个例子来理解上述规则。将表达式 `(3 + (4 * 5))` 拆分成标记列表 `['(', '3', '+', '(', '4', '*', '5', ')', ')', ')']`。起初，解析树只有一个空的根节点，随着对每个标记的处理，解析树的结构和内容逐渐充实，如图6所示。

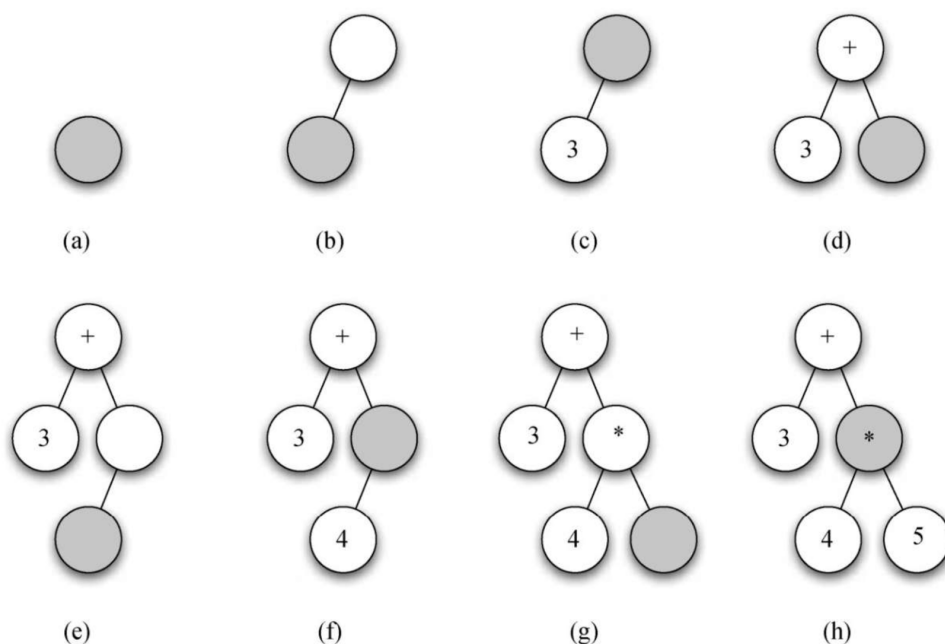


图4 将 `(3 + (4 * 5))` 一步步地构建解析树

以图4为例，我们来一步步地构建解析树。

- (a) 创建一棵空树。
- (b) 读入第一个标记 `(`。根据规则1，为根节点添加一个左子节点。
- (c) 读入下一个标记 `3`。根据规则3，将当前节点的值设为3，并回到父节点。
- (d) 读入下一个标记 `+`。根据规则2，将当前节点的值设为+，并添加一个右子节点。新节点成为当前节点。
- (e) 读入下一个标记 `(`。根据规则1，为当前节点添加一个左子节点，并将其作为当前节点。
- (f) 读入下一个标记 `4`。根据规则3，将当前节点的值设为4，并回到父节点。
- (g) 读入下一个标记 `*`。根据规则2，将当前节点的值设为*，并添加一个右子节点。新节点成为当前节点。
- (h) 读入下一个标记 `5`。根据规则3，将当前节点的值设为5，并回到父节点。
- (i) 读入下一个标记 `)`。根据规则4，将*的父节点作为当前节点。
- (j) 读入下一个标记 `)`。根据规则4，将+的父节点作为当前节点。因为+没有父节点，所以工作完成。

本例表明，在构建解析树的过程中，需要追踪当前节点及其父节点。可以通过 `getLeftChild` 与 `getRightChild` 获取子节点，但如何追踪父节点呢？一个简单的办法就是在遍历这棵树时使用栈记录父节点。每当要下沉至当前节点的子节点时，先将当前节点压到栈中。当要返回到当前节点的父节点时，就将父节点从栈中弹出来。

利用前面描述的规则以及 `Stack` 和 `BinaryTree`，就可以编写创建解析树的Python函数。

4.1.1 实现：解析树构建器

```
1 class Stack(object):
2     def __init__(self):
3         self.items = []
```

```

4         self.stack_size = 0
5
6     def isEmpty(self):
7         return self.stack_size == 0
8
9     def push(self, new_item):
10        self.items.append(new_item)
11        self.stack_size += 1
12
13    def pop(self):
14        self.stack_size -= 1
15        return self.items.pop()
16
17    def peek(self):
18        return self.items[self.stack_size - 1]
19
20    def size(self):
21        return self.stack_size
22
23
24    class BinaryTree:
25        def __init__(self, rootObj):
26            self.key = rootObj
27            self.leftChild = None
28            self.rightChild = None
29
30        def insertLeft(self, newNode):
31            if self.leftChild == None:
32                self.leftChild = BinaryTree(newNode)
33            else: # 已经存在左子节点。此时，插入一个节点，并将已有的左子节点降一层。
34                t = BinaryTree(newNode)
35                t.leftChild = self.leftChild
36                self.leftChild = t
37
38        def insertRight(self, newNode):
39            if self.rightChild == None:
40                self.rightChild = BinaryTree(newNode)
41            else:
42                t = BinaryTree(newNode)
43                t.rightChild = self.rightChild
44                self.rightChild = t
45
46        def getRightChild(self):
47            return self.rightChild
48
49        def getLeftChild(self):
50            return self.leftChild
51
52        def setRootVal(self, obj):
53            self.key = obj
54
55        def getRootVal(self):
56            return self.key
57
58        def traversal(self, method="preorder"):
59            if method == "preorder":
60                print(self.key, end=" ")
61            if self.leftChild != None:

```



```

62         self.leftChild.traversal(method)
63     if method == "inorder":
64         print(self.key, end=" ")
65     if self.rightChild != None:
66         self.rightChild.traversal(method)
67     if method == "postorder":
68         print(self.key, end=" ")
69
70
71 def buildParseTree(fpexp):
72     fplist = fpexp.split()
73     pStack = Stack()
74     eTree = BinaryTree('')
75     pStack.push(eTree)
76     currentTree = eTree
77
78     for i in fplist:
79         if i == '(':
80             currentTree.insertLeft('')
81             pStack.push(currentTree)
82             currentTree = currentTree.getLeftChild()
83         elif i not in '+-*/':
84             currentTree.setRootVal(int(i))
85             parent = pStack.pop()
86             currentTree = parent
87         elif i in '+-*/':
88             currentTree.setRootVal(i)
89             currentTree.insertRight('')
90             pStack.push(currentTree)
91             currentTree = currentTree.getRightChild()
92         elif i == ')':
93             currentTree = pStack.pop()
94         else:
95             raise ValueError("Unknown Operator: " + i)
96     return eTree
97
98
99 exp = "( ( 7 + 3 ) * ( 5 - 2 ) )"
100 pt = buildParseTree(exp)
101 for mode in ["preorder", "postorder", "inorder"]:
102     pt.traversal(mode)
103     print()
104
105 """
106 * + 7 3 - 5 2
107 7 3 + 5 2 - *
108 7 + 3 * 5 - 2
109 """
110
111 # 代码清单6-10
112 import operator
113
114 def evaluate(parseTree):
115     ops = {'+':operator.add, '-':operator.sub, '*':operator.mul, '/':operator.truediv}
116
117     leftC = parseTree.getLeftChild()
118     rightC = parseTree.getRightChild()
119

```

```

120     if leftC and rightC:
121         fn = opers[parseTree.getRootVal()]
122         return fn(evaluate(leftC),evaluate(rightC))
123     else:
124         return parseTree.getRootVal()
125
126 print(evaluate(pt))
127 # 30
128
129 #代码清单6-14 后序求值
130 def postordereval(tree):
131     opers = {'+':operator.add, '-':operator.sub,
132             '*':operator.mul, '/':operator.truediv}
133     res1 = None
134     res2 = None
135     if tree:
136         res1 = postordereval(tree.getLeftChild())
137         res2 = postordereval(tree.getRightChild())
138         if res1 and res2:
139             return opers[tree.getRootVal()](res1,res2)
140         else:
141             return tree.getRootVal()
142
143 print(postordereval(pt))
144 # 30
145
146 #代码清单6-16 中序还原完全括号表达式
147 def printexp(tree):
148     sVal = ""
149     if tree:
150         sVal = '(' + printexp(tree.getLeftChild())
151         sVal = sVal + str(tree.getRootVal())
152         sVal = sVal + printexp(tree.getRightChild()) + ')'
153     return sVal
154
155 print(printexp(pt))
156 # ((7)+3)*((5)-2))

```

在代码清单中，第79、83、87和92行的 `if` 语句体现了构建解析树的4条规则，其中每条语句都通过调用 `BinaryTree` 和 `Stack` 的方法实现了前面描述的规则。这个函数中唯一的错误检查在 `else` 从句中，如果遇到一个不能识别的标记，就抛出一个 `ValueError` 异常。

有了一棵解析树之后，我们能对它做些什么呢？作为第一个例子，我们可以写一个函数 **计算解析树**，并返回计算结果。要写这个函数，我们将利用树的层次性。针对图2中的解析树，可以用图3中的简化解析树替换。由此可见，可以写一个算法，通过递归计算每棵子树得到整棵解析树的结果。

因为两个图距离目前文字太远，复制粘贴

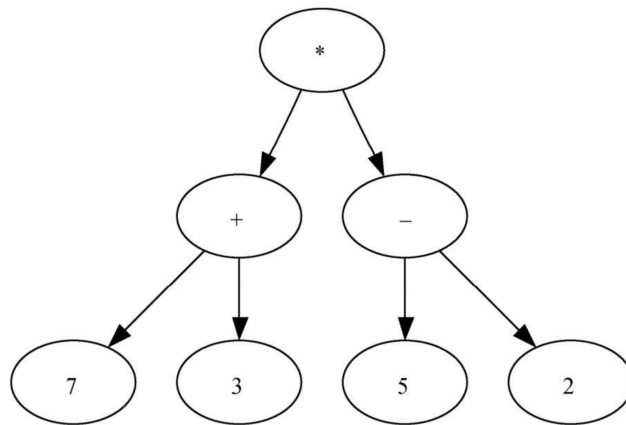


图2 `((7 + 3) \* (5 - 2))` 的解析树

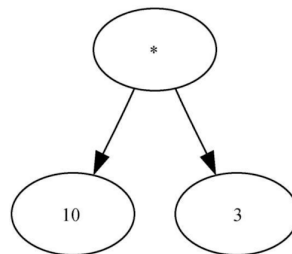


图3 `((7 + 3) \* (5 - 2))` 的简化解析树

和之前编写递归函数一样，设计递归计算函数要从确定基本情况开始。就针对树进行操作的递归算法而言，一个很自然的基本情况就是检查叶子节点。解析树的叶子节点必定是操作数。由于像整数和浮点数这样的数值对象不需要进一步翻译，因此evaluate函数可以直接返回叶子节点的值。为了向基本情况靠近，算法将执行递归步骤，即对当前节点的左右子节点调用evaluate函数。递归调用可以有效地沿着各条边往叶子节点靠近。

若要结合两个递归调用的结果，只需将父节点中存储的运算符应用于子节点的计算结果即可。从图3中可知，根节点的两个子节点的计算结果就是它们自身，即10和3。应用乘号，得到最后的结果30。

递归函数evaluate的实现如代码清单所示。首先，获取指向当前节点的左右子节点的引用。如果左右子节点的值都是None，就说明当前节点确实是叶子节点。第9行执行这项检查。如果当前节点不是叶子节点，则查看当前节点中存储的运算符，并将其应用于左右子节点的递归计算结果。

4.1.2 实现：计算二叉解析树的递归函数

代码清单6-10。

```

1 import operator
2
3 def evaluate(parseTree):
4     opers = {'+':operator.add, '-':operator.sub, '*':operator.mul, '/':operator.truediv}
5
6     leftC = parseTree.getLeftChild()
7     rightC = parseTree.getRightChild()
8
9     if leftC and rightC:
10         fn = opers[parseTree.getRootVal()]
11         return fn(evaluate(leftC),evaluate(rightC)) #先计算左子树，再计算右子树，最后通过根节点运算符的函数
            调用将两个结果结合起来。
12     else:
13         return parseTree.getRootVal()

```

我们使用具有键 +、-、* 和 / 的字典实现。字典中存储的值是 `operator` 模块的函数。该模块提供了常用运算符的函数版本。在字典中查询运算符时，对应的函数对象被取出。既然取出的对象是函数，就可以用普通的方式 `function(param1, param2)` 调用。因此，`opers['+'](2, 2)` 等价于 `operator.add(2, 2)`。

最后，让我们通过 `(3 + (4 * 5))` 的解析树构建过程来理解 `evaluate` 函数。第一次调用 `evaluate` 函数时，将整棵树的根节点作为参数 `parseTree` 传入。然后，获取指向左子节点的引用，检查它们是否存在。第11行进行递归调用。从查询根节点的运算符开始，该运算符是+，对应 `operator.add` 函数，要传入两个参数。和普通的Python函数调用一样，Python做的第一件事是计算入参的值。本例中，两个入参都是对 `evaluate` 函数的递归调用。由于入参的计算顺序是从左到右，因此第一次递归调用是在左边。对左子树递归调用 `evaluate` 函数，发现节点没有左右子节点，所以这是一个叶子节点。处于叶子节点时，只需返回叶子节点的值作为计算结果即可。本例中，返回整数3。

至此，已经为顶层的 `operator.add` 调用计算出一个参数的值了，但还没完。继续从左到右的参数计算过程，现在进行一个递归调用，计算根节点的右子节点。发现该节点不仅有左子节点，还有右子节点，所以检查节点存储的运算符——是*，将左右子节点作为参数调用函数。这时可以看到，两个调用都已到达叶子节点，计算结果分别是4和5。算出参数之后，返回 `operator.mul(4, 5)` 的结果。至此，已经算出了顶层运算符 (+) 的操作数，剩下的工作就是完成对 `operator.add(3, 20)` 的调用。因此，表达式 `(3 + (4 * 5))` 的计算结果就是23。

4.1.3 编程题目

示例M24591:中序表达式转后序表达式

stack, ast, <http://cs101.openjudge.cn/practice/24591/>

中序表达式是运算符放在两个数中间的表达式。乘、除运算优先级高于加减。可以用"()"来提升优先级 --- 就是小学生写的四则算术运算表达式。中序表达式可用如下方式递归定义：

- 1) 一个数是一个中序表达式。该表达式的值就是数的值。
2. 若a是中序表达式，则"(a)"也是中序表达式(引号不算)，值为a的值。
3. 若a,b是中序表达式，c是运算符，则"acb"是中序表达式。"acb"的值是对a和b做c运算的结果，且a是左操作数，b是右操作数。

输入一个中序表达式，要求转换成一个后序表达式输出。

输入

第一行是整数n(n<100)。接下来n行，每行一个中序表达式，数和运算符之间没有空格，长度不超过700。

输出

对每个中序表达式，输出转成后序表达式后的结果。后序表达式的数之间、数和运算符之间用一个空格分开。

样例输入

```
1 | 3
2 | 7+8.3
3 | 3+4.5*(7+2)
4 | (3)*((3+4)*(2+3.5)/(4+5))
```

样例输出

```
1 | 7 8.3 +
2 | 3 4.5 7 2 + * +
3 | 3 3 4 + 2 3.5 + * 4 5 + / *
```

思路：利用 Python 的自带的语法解析模块 `ast` 模块把中序表达式解析成抽象语法树（AST，Abstract Syntax Tree），把原本需要自己写“运算符优先级”和“栈”的问题，交给 Python 的语法分析器自动完成。

然后递归遍历语法树，按照“左子树 → 右子树 → 根节点”的顺序输出，这正是 **后序遍历** 的顺序，也正对应 **后缀表达式** 的结构。

`mode='eval'` 让 `ast.parse()` 解析的是“单个表达式”，不会把它当成程序。

返回的 `tree` 是 `ast.Expression` 对象；

真正的表达式节点（`BinOp` / `Constant`）在 `tree.body` 里，所以我们要从 `tree.body` 开始递归遍历。

`ast` 中的运算符是类（如 `ast.Add`, `ast.Mult`），需要映射成我们要打印的符号 `“+”`、`“-”`、`“*”`、`“/”`。

```
1 | import ast
2 |
3 | # 定义运算符到字符串的映射
4 | operator_to_str = {
5 |     ast.Add: '+',
6 |     ast.Sub: '-',
7 |     ast.Mult: '*',
8 |     ast.Div: '/'
9 | }
10 |
11 | def postfix(node):
12 |     # 如果节点是常量，则直接返回其值
13 |     if isinstance(node, ast.Constant):
14 |         return str(node.value)
15 |     # 如果节点是二元运算符，则递归处理左右子节点并拼接运算符
16 |     elif isinstance(node, ast.BinOp):
17 |         return f'{postfix(node.left)} {postfix(node.right)} {operator_to_str[type(node.op)]}'
18 |
19 | # 主程序部分
20 | n = int(input())
21 | for i in range(n):
22 |     expr = input()
23 |     tree = ast.parse(expr, mode='eval')
24 |     print(postfix(tree.body))
```

对照示意图：

```
1 | tree = ast.parse("3+4.5*(7+2)", mode='eval')
```

结构：

```
1 Expression
2   └─ body → BinOp(+)
3       └─ left: Constant(3)
4       └─ right: BinOp(*)
5           └─ left: Constant(4.5)
6           └─ right: BinOp(+)
7               └─ left: Constant(7)
8               └─ right: Constant(2)
```

调用：

```
1 postfix(tree.body)
```

从 `BinOp(+)` 开始递归，就能正确生成：

```
1 3 4.5 7 2 + * +
```

练习M25140: 根据后序表达式建立队列表达式

<http://cs101.openjudge.cn/practice/25140/>

后序算术表达式可以通过栈来计算其值，做法就是从左到右扫描表达式，碰到操作数就入栈，碰到运算符，就取出栈顶的2个操作数做运算（先出栈的是第二个操作数，后出栈的是第一个），并将运算结果压入栈中。最后栈里只剩下一个元素，就是表达式的值。

有一种算术表达式不妨叫做“队列表达式”，它的求值过程和后序表达式很像，只是将栈换成了队列：从左到右扫描表达式，碰到操作数就入队列，碰到运算符，就取出队头2个操作数做运算（先出队的是第2个操作数，后出队的是第1个），并将运算结果加入队列。最后队列里只剩下一个元素，就是表达式的值。

给定一个后序表达式，请转换成等价的队列表达式。例如，`3 4 + 6 5 * -`的等价队列表达式就是`5 6 4 3 * + -`。

输入

第一行是正整数 n ($n < 100$)。接下来是 n 行，每行一个由字母构成的字符串，长度不超过100,表示一个后序表达式，其中小写字母是操作数，大写字母是运算符。运算符都是需要2个操作数的。

输出

对每个后序表达式，输出其等价的队列表达式。

样例输入

```
1 2
2 xyPzwIM
3 abcABdefgCDEF
```

样例输出

```
1 wzyxIPM
2 gfCecbDdAaEBF
```

提示

建立起表达式树，按层次遍历表达式树的结果前后颠倒就得到队列表达式

来源：Guo Wei modified from Ulm Local 2007

问题是要求将一个后缀表达式转换为等价的“队列表达式”。这个“队列表达式”是通过由后缀表达式构建出的表达式树进行层序遍历的逆序得到的。

步骤如下：

1. 创建一个 `TreeNode` 类来表示树中的每个节点。
2. 创建一个 `build_tree` 函数，接收后缀表达式作为输入，并返回构建好的表达式树的根节点。
 - 使用栈来存储节点。
 - 遍历后缀表达式的每一个字符：
 - 如果字符是操作数 (operand)，创建一个新节点并压入栈中。
 - 如果字符是操作符 (operator)，从栈中弹出两个节点，作为新操作符节点的左右子节点，并将该新节点压入栈中。
3. 创建一个 `level_order_traversal` 函数，接收树的根节点，返回其层序遍历的结果。
 - 使用一个队列 `traversal` 来存储待访问的节点。
 - 当队列非空时，出队一个节点，访问它，并将其左右子节点依次入队。
4. 对每一个后缀表达式，先构造树，再进行层序遍历，然后将结果反转，输出最终字符串。

```
1 class TreeNode:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6
7 def build_tree(postfix):
8     stack = []
9     for char in postfix:
10         node = TreeNode(char)
11         if char.isupper():
12             node.right = stack.pop()
13             node.left = stack.pop()
14             stack.append(node)
15     return stack[0]
16
17 def level_order_traversal(root):
18     queue = [root]
19     traversal = []
20     while queue:
21         node = queue.pop(0)
22         traversal.append(node.value)
23         if node.left:
24             queue.append(node.left)
25         if node.right:
26             queue.append(node.right)
27     return traversal
28
29 n = int(input().strip())
30 for _ in range(n):
31     postfix = input().strip()
32     root = build_tree(postfix)
```

```

33     queue_expression = level_order_traversal(root)[::-1]
34     print(''.join(queue_expression))

```

4.2 遍历及应用

我们已经了解了树的基本功能，现在是时候看看一些附加的使用模式了。这些使用模式可以按节点的访问方式分为3种。我们将对所有节点的访问称为“遍历”，共有3种遍历方式，分别为前序遍历、中序遍历和后序遍历。接下来，我们先仔细地定义这3种遍历方式，然后通过一些例子看看它们的用法。

前序遍历

在前序遍历中，先访问根节点，然后递归地前序遍历左子树，最后递归地前序遍历右子树。

中序遍历

在中序遍历中，先递归地中序遍历左子树，然后访问根节点，最后递归地中序遍历右子树。

后序遍历

在后序遍历中，先递归地后序遍历左子树，然后递归地后序遍历右子树，最后访问根节点。

让我们通过几个例子来理解这3种遍历方式。首先看看前序遍历。将一本书的内容结构表示为一棵树，整本书是根节点，每一章是根节点的子节点，每一章中的每一节是这章的子节点，每小节又是这节的子节点，依此类推。图8展示了一本书的树状结构，它包含两章。注意，遍历算法对每个节点的子节点数没有要求，但本例只针对二叉树。

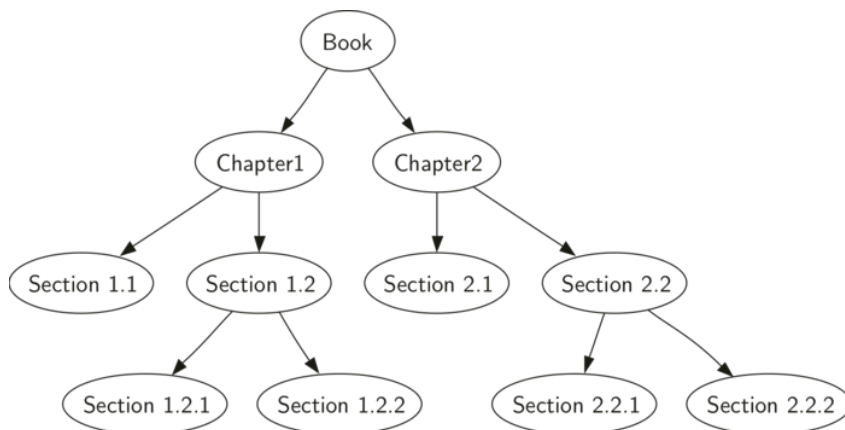


图8 一本书的树状结构

假设我们从前往后阅读这本书，那么阅读顺序就符合前序遍历的次序。从根节点“书”开始，遵循前序遍历指令，对左子节点“第1章”递归调用preorder函数。然后，对“第1章”的左子节点递归调用preorder函数，得到节点“1.1节”。由于该节点没有子节点，因此不必再进行递归调用。沿着树回到节点“第1章”，接下来访问它的右子节点，即“1.2节”。和前面一样，先访问左子节点“1.2.1节”，然后访问右子节点“1.2.2节”。访问完“1.2节”之后，回到“第1章”。接下来，回到根节点，以同样的方式访问节点“第2章”。

遍历树的代码格外简洁，这主要是因为遍历是递归的。

你可能会想，前序遍历算法的最佳实现方式是什么呢？是一个将树用作数据结构的函数，还是树本身的一个方法？代码清单6-11给出了前序遍历算法的外部函数版本，该函数将二叉树作为参数，其代码尤为简洁，这是因为算法的基本情况仅仅是检查树是否存在。如果参数tree是None，函数直接返回。

代码清单6-11 将前序遍历算法实现为外部函数

```

1 def preorder(tree):
2     if tree:
3         print(tree.getRootVal())
4         preorder(tree.getLeftChild())
5         preorder(tree.getRightChild())

```


也可以将preorder实现为BinaryTree类的方法，如代码清单6-12所示。请留意将代码从外部移到内部后有何变化。通常来说，不仅需要用self代替tree，还需要修改基本情况。**内部方法必须在递归调用preorder前，检查左右子节点是否存在。**

代码清单6-12 将前序遍历算法实现为BinaryTree类的方法

```
1 def preorder(self):
2     print(self.key)
3     if self.leftChild:
4         self.leftChild.preorder()
5     if self.rightChild:
6         self.rightChild.preorder()
```

Q: 为什么内部方法必须在递归调用preorder前，检查左右子节点是否存在？

它触及了外部函数与类的内部方法在设计递归算法（如树的遍历）时的核心差异。

我们来深入分析为什么——如原文所说——**内部方法必须检查左右节点是否存在**，而外部函数版本则不需要显式检查子节点，只需要检查传入的树对象本身是否存在。

一、外部函数版本为何“简洁”？

```
1 def preorder(tree):
2     if tree:
3         print(tree.getRootVal())
4         preorder(tree.getLeftChild())
5         preorder(tree.getRightChild())
```

这个版本之所以“格外简洁”，是因为它把 **树的节点是否存在** 的判断统一交给了函数入口的 `if tree:`。

- 每次递归调用 `preorder(...)` 时，参数可能是 `None`（比如某个节点没有左孩子）。
- 但函数一开始就检查 `if tree:`，所以一旦传入的是 `None`，函数直接返回，不会继续执行。
- 因此，即使我们“无脑”地把 `tree.getLeftChild()`（可能为 `None`）传进去，也能安全处理。

优点：逻辑集中，代码简洁，递归调用无需前置判断。

二、内部方法为何必须检查子节点？

再看代码：

```
1 def preorder(self):
2     print(self.key)
3     if self.leftChild:
4         self.leftChild.preorder()
5     if self.rightChild:
6         self.rightChild.preorder()
```

这里的关键在于：`preorder` 是 `BinaryTree` 类的一个实例方法。这意味着：

- 它只能被一个有效的 `BinaryTree` 实例调用（即 `self` 是一个真实存在的节点对象）。
- 但 `self.leftChild` 和 `self.rightChild` 可能是 `None`。
- 如果你尝试对 `None` 调用 `.preorder()`，会引发 `AttributeError: 'NoneType' object has no attribute`

'preorder'。

所以你不能像外部函数那样直接递归调用：

```
1 | self.leftChild.preorder() # 如果 leftChild 是 None, 就会出错!
```

因此必须先判断：

```
1 | if self.leftChild:
2 |     self.leftChild.preorder()
```

这一步是为了防止调用不存在对象的方法，属于安全性检查。

哪种实现方式更好呢？在本例中，将preorder实现为外部函数可能是更好的选择。原因在于，很少会仅执行遍历操作，在大多数情况下，还要通过基本的遍历模式实现别的目标。在下一个例子中，就会通过后序遍历来计算解析树。所以，在此采用外部函数版本。

后序遍历函数postorder与前序遍历函数preorder几乎相同，只不过对print的调用被移到了函数的末尾。

代码清单6-13 后序遍历函数

```
1 | def postorder(tree):
2 |     if tree != None:
3 |         postorder(tree.getLeftChild())
4 |         postorder(tree.getRightChild())
5 |         print(tree.getRootVal())
```

我们已经见识过后序遍历的一个常见用途，那就是计算解析树。回顾代码清单6-10，所做的就是先计算左子树，再计算右子树，最后通过根节点运算符的函数调用将两个结果结合起来。假设二叉树只存储一个表达式的数据。让我们来重写计算函数，使之更接近于代码清单6-13中的后序遍历函数。

4.2.1 后序求值

代码清单6-14

```
1 | def postordereval(tree):
2 |     ops = {'+':operator.add, '-':operator.sub,
3 |           '*':operator.mul, '/':operator.truediv}
4 |     res1 = None
5 |     res2 = None
6 |     if tree:
7 |         res1 = postordereval(tree.getLeftChild())
8 |         res2 = postordereval(tree.getRightChild())
9 |         if res1 and res2:
10 |             return ops[tree.getRootVal()](res1,res2)
11 |         else:
12 |             return tree.getRootVal()
```

注意，代码清单6-14与代码清单6-13在形式上很相似，只不过求值函数最后不是打印节点，而是返回节点。这样一来，就可以保存从第7行和第8行的递归调用返回的值，然后在第10行使用这些值和运算符进行计算。

最后了解中序遍历。中序遍历的访问顺序是左子树、根节点、右子树。代码给出了中序遍历函数的代码。注意，3个遍历函数的区别仅在于print语句与递归调用语句的相对位置。

代码清单6-15 中序遍历函数

```
1 def inorder(tree):
2     if tree != None:
3         inorder(tree.getLeftChild())
4         print(tree.getRootVal())
5         inorder(tree.getRightChild())
```

通过中序遍历解析树，可以还原不带括号的表达式。接下来修改中序遍历算法，以得到完全括号表达式。唯一要做的修改是：在递归调用左子树前打印一个左括号，在递归调用右子树后打印一个右括号。代码清单6-16是修改后的函数。

4.2.2 中序还原完全括号表达式

代码清单6-16

```
1 def printexp(tree):
2     sVal = ""
3     if tree:
4         sVal = '(' + printexp(tree.getLeftChild())
5         sVal = sVal + str(tree.getRootVal())
6         sVal = sVal + printexp(tree.getRightChild()) + ')'
7     return sVal
8
9 print(printexp(pt))
10 # ((7)+3)*((5)-2))
```

注意，printexp函数给每个数字都加上了括号。尽管不能算错误，但这些括号显然是多余的。请修改printexp函数，移除这些括号。

4.2.3 编程题目

“抽象结构还原—遍历—输出规约”；关键在于先从题面抽出可验证的不变量（父子出现顺序、括号文法、目录与文件的层级与排序、BST 的区间收敛），再用最小而稳定的模板承载它们：用指针推进和栈完成结构构建，用递归把状态沿路径精准传递（如路径数的十进制累积），用局部排序与稳定顺序控制输出；每次编码都先画出数据流与控制流，明确“递归终止—状态转移—输出时机”的三件事，这比单纯背模板更可靠也更可迁移。

① Note

M07161: 森林的带度数层次序列存储 (assign#6)

tree, <http://cs101.openjudge.cn/practice/07161/>

M27928: 遍历树

adjacency list, dfs, <http://cs101.openjudge.cn/practice/27928/>

思路：重点是根结点的求取

M129.求根节点到叶节点数字之和 (assign#7)

dfs, <https://leetcode.cn/problems/sum-root-to-leaf-numbers/>

思路：不断扩充， $\times 10$ +新的个位数字

M24729: 括号嵌套树 (assign#7)

dfs, stack, <http://cs101.openjudge.cn/practice/24729/>

思路：使用之前的括号策略，把第一个字母作为根，之后如果有父节点，加入stack，当括号结束后再移出stack。然后形成节点树。使用dfs输出preorder和postorder的序列结果。

T02775: 文件结构“图”

tree, <http://cs101.openjudge.cn/practice/02775/>

思路：要特别区别区分directions和files的差别化处理，本题特别复杂，先生成树，再进行输出，files要按照排列顺序，directions按照按照目录出现的先后显示。

遍历关系的“唯一性结论”

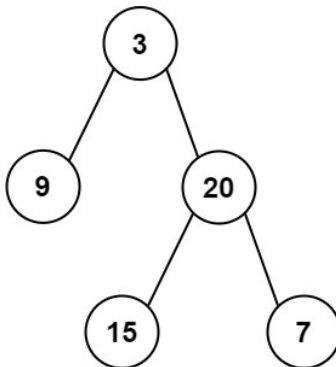
- 已知 [前序+中序] 或 [后序+中序] 可唯一确定一棵树。
- 注意：已知 [前序+后序] 不能唯一确定一棵树。

练习M105.从前序与中序遍历序列构造二叉树

<https://leetcode.cn/problems/construct-binary-tree-from-preorder-and-inorder-traversal/>

给定两个整数数组 `preorder` 和 `inorder`，其中 `preorder` 是二叉树的先序遍历，`inorder` 是同一棵树的中序遍历，请构造二叉树并返回其根节点。

示例 1:



```
1 输入: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
2 输出: [3,9,20,null,null,15,7]
```

示例 2:

```
1 输入: preorder = [-1], inorder = [-1]
2 输出: [-1]
```

提示:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` 和 `inorder` 均无重复元素
- `inorder` 均出现在 `preorder`
- `preorder` 保证为二叉树的前序遍历序列

- `inorder` 保证 为二叉树的中序遍历序列

```
1 from typing import List, Optional
2 # Definition for a binary tree node.
3 # class TreeNode:
4 #     def __init__(self, val=0, left=None, right=None):
5 #         self.val = val
6 #         self.left = left
7 #         self.right = right
8 class Solution:
9     def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
10         if not preorder or not inorder:
11             return None
12
13         # The first element in preorder is the root
14         root_val = preorder[0]
15         root = TreeNode(root_val)
16
17         # Find the index of the root in inorder
18         root_index = inorder.index(root_val)
19
20         # Recursively build the left and right subtrees
21         root.left = self.buildTree(preorder[1:1 + root_index], inorder[:root_index])
22         root.right = self.buildTree(preorder[1 + root_index:], inorder[root_index + 1:])
23
24         return root
25
26 if __name__ == '__main__':
27     solution = Solution()
28     preorder = [3, 9, 20, 15, 7]
29     inorder = [9, 3, 15, 20, 7]
30     root = solution.buildTree(preorder, inorder)
31     # The output tree is [3, 9, 20, None, None, 15, 7]
32
```

以下是与力扣 105 类似的3个题目

1) 练习M24750: 根据二叉树中后序序列建树

<http://cs101.openjudge.cn/practice/24750/>

假设二叉树的节点里包含一个大写字母，每个节点的字母都不同。

给定二叉树的中序遍历序列和后序遍历序列（长度均不超过26），请输出该二叉树的前序遍历序列。

输入

2行，均为大写字母组成的字符串，表示一棵二叉树的中序遍历序列与后序遍历排列。

输出

表示二叉树的前序遍历序列。

样例输入

```
1 | BADC
2 | BDCA
```

样例输出

```
1 | ABCD
```

来源

Lou Yuke

```
1 """
2 后序遍历的最后一个元素是树的根节点。然后，在中序遍历序列中，根节点将左右子树分开。
3 可以通过这种方法找到左右子树的中序遍历序列。然后，使用递归地处理左右子树来构建整个树。
4 """
5
6 def build_tree(inorder, postorder):
7     if not inorder or not postorder:
8         return []
9
10    root_val = postorder[-1]
11    root_index = inorder.index(root_val)
12
13    left_inorder = inorder[:root_index]
14    right_inorder = inorder[root_index + 1:]
15
16    left_postorder = postorder[:len(left_inorder)]
17    right_postorder = postorder[len(left_inorder):-1]
18
19    root = [root_val]
20    root.extend(build_tree(left_inorder, left_postorder))
21    root.extend(build_tree(right_inorder, right_postorder))
22
23    return root
24
25
26 def main():
27     inorder = input().strip()
28     postorder = input().strip()
29     preorder = build_tree(inorder, postorder)
30     print(' '.join(preorder))
31
32
33 if __name__ == "__main__":
34     main()
35
```

```
1 """
2 定义一个递归函数。在这个递归函数中，我们将后序遍历的最后一个元素作为当前的根节点，然后在中序遍历序列中找到这个根节点的位置，
3 这个位置将中序遍历序列分为左子树和右子树。
4 """
```

```

5 class TreeNode:
6     def __init__(self, x):
7         self.val = x
8         self.left = None
9         self.right = None
10
11
12 def buildTree(inorder, postorder):
13     if not inorder or not postorder:
14         return None
15
16     # 后序遍历的最后一个元素是当前的根节点
17     root_val = postorder.pop()
18     root = TreeNode(root_val)
19
20     # 在中序遍历中找到根节点的位置
21     root_index = inorder.index(root_val)
22
23     # 构建右子树和左子树
24     root.right = buildTree(inorder[root_index + 1:], postorder)
25     root.left = buildTree(inorder[:root_index], postorder)
26
27     return root
28
29
30 def preorderTraversal(root):
31     result = []
32     if root:
33         result.append(root.val)
34         result.extend(preorderTraversal(root.left))
35         result.extend(preorderTraversal(root.right))
36     return result
37
38
39 # 读取输入
40 inorder = input().strip()
41 postorder = input().strip()
42
43 # 构建树
44 root = buildTree(list(inorder), list(postorder))
45
46 # 输出前序遍历序列
47 print(''.join(preorderTraversal(root)))

```

24行, 25行递归调用顺序: 先右后左

```

1 root.right = buildTree(..., postorder) # 先构建右子树
2 root.left = buildTree(..., postorder)  # 再构建左子树

```

这个顺序至关重要!

- 当你 `pop()` 出当前根后, 下一个要构建的是右子树。
- 而 `postorder` 剩下的末尾元素, 正好是右子树的根。
- 构建完右子树后, 再构建左子树, 此时 `postorder` 剩下的末尾又是左子树的根。

所以, 递归顺序与 `pop()` 顺序完美匹配。

2) 练习M22158: 根据二叉树前中序序列建树 (assign#7)

<http://cs101.openjudge.cn/practice/22158/>

假设二叉树的节点里包含一个大写字母，每个节点的字母都不同。

给定二叉树的前序遍历序列和中序遍历序列(长度均不超过26)，请输出该二叉树的后序遍历序列

输入

多组数据

每组数据2行，第一行是前序遍历序列，第二行是中序遍历序列

输出

对每组序列建树，输出该树的后序遍历序列

样例输入

```
1 | DURPA
2 | RUDPA
3 | XTCNB
4 | CTBNX
```

样例输出

```
1 | RUAPD
2 | CBNTX
```

来源：郭炜

问题是要求根据给定的前序遍历和中序遍历序列来构造一棵二叉树，然后输出该构造树的后序遍历序列。

以下是分步计划：

1. 创建一个 `TreeNode` 类来表示树中的每个节点。
2. 创建一个函数 `build_tree`，它接收前序和中序遍历序列作为输入，并返回所构造树的根节点。
 - o 前序遍历序列的第一个字符是树的根节点。
 - o 在中序遍历序列中找到根节点的位置。
 - o 使用中序序列的左半部分和对应的前序序列部分递归地构造左子树。
 - o 使用中序序列的右半部分和对应的前序序列部分递归地构造右子树。
3. 创建一个函数 `postorder_traversal`，它接收树的根节点作为输入，并返回树的后序遍历序列。
4. 对于每一对前序和中序遍历序列，构造树并输出其后序遍历序列。

```
1 | class TreeNode:
2 |     def __init__(self, value):
3 |         self.value = value
4 |         self.left = None
5 |         self.right = None
6 |
7 | def build_tree(preorder, inorder):
```



```

8     if not preorder or not inorder:
9         return None
10    root_value = preorder[0]
11    root = TreeNode(root_value)
12    root_index_inorder = inorder.index(root_value)
13    root.left = build_tree(preorder[1:1+root_index_inorder], inorder[:root_index_inorder])
14    root.right = build_tree(preorder[1+root_index_inorder:], inorder[root_index_inorder+1:])
15    return root
16
17    def postorder_traversal(root):
18        if root is None:
19            return ''
20        return postorder_traversal(root.left) + postorder_traversal(root.right) + root.value
21
22    while True:
23        try:
24            preorder = input().strip()
25            inorder = input().strip()
26            root = build_tree(preorder, inorder)
27            print(postorder_traversal(root))
28        except EOFError:
29            break

```

3) 练习M25145: 猜二叉树（按层次遍历）

<http://cs101.openjudge.cn/practice/25145/>

一棵二叉树，结点都是大写英文字母，且不重复。

给出它的中序遍历序列和后序遍历序列，求其按层次遍历的序列。

输入

第一行是整数n, n <= 30，表示有n棵二叉树

接下来每两行代表一棵二叉树，第一行是其中序遍历序列，第二行是后序遍历序列

输出

对每棵二叉树输出其按层次遍历序列

样例输入

```

1  2
2  LZGD
3  LGDZ
4  BKTVPQ
5  TPQVKB

```

样例输出

```

1  ZLDG
2  BKVTQP

```

来源: Guo Wei

```

1 from collections import deque
2
3 class Node:
4     def __init__(self, data):
5         self.data = data
6         self.left = None
7         self.right = None
8
9 def build_tree(inorder, postorder):
10     if inorder:
11         root = Node(postorder.pop())
12         root_index = inorder.index(root.data)
13         root.right = build_tree(inorder[root_index+1:], postorder)
14         root.left = build_tree(inorder[:root_index], postorder)
15         return root
16
17 def level_order_traversal(root):
18     if root is None:
19         return []
20     result = []
21     queue = deque([root])
22     while queue:
23         node = queue.popleft()
24         result.append(node.data)
25         if node.left:
26             queue.append(node.left)
27         if node.right:
28             queue.append(node.right)
29     return result
30
31 n = int(input())
32 for _ in range(n):
33     inorder = list(input().strip())
34     postorder = list(input().strip())
35     root = build_tree(inorder, postorder)
36     print(''.join(level_order_traversal(root)))

```

练习T20576: printExp (逆波兰表达式建树)

<http://cs101.openjudge.cn/practice/20576/>

输出中缀表达式（去除不必要的括号）

输入

一个字串

输出

一个字串

样例输入

```
1 | ( not ( True or False ) ) and ( False or True and True )
```

样例输出

```
1 | not ( True or False ) and ( False or True and True )
```

题意：这三个操作符：not：优先级最高，and：其次，or：优先级最低。手写一个完整的Shunting Yard + AST 构造 + 打印

思路：基于优先级的通用中缀打印

核心思想：

- 给每个节点定义一个优先级值（not=3, and=2, or=1）。
- 打印时，仅当子节点的优先级低于父节点时加括号。
- not 是单目操作符，只处理一个子节点。
- True、False 优先级设为 4（最高，永远不加括号）。

```
1 class BinaryTree:
2     def __init__(self, root, left=None, right=None):
3         self.root = root
4         self.leftChild = left
5         self.rightChild = right
6
7     def postorder(string): # 中缀改后缀 (Shunting Yard)
8         opStack, postList = [], []
9         inList = string.split()
10        prec = {'(': 0, 'or': 1, 'and': 2, 'not': 3}
11        for word in inList:
12            if word == '(':
13                opStack.append(word)
14            elif word == ')':
15                while opStack and opStack[-1] != '(':
16                    postList.append(opStack.pop())
17                opStack.pop()
18            elif word in ('True', 'False'):
19                postList.append(word)
20            else: # operator
21                while opStack and prec[word] <= prec[opStack[-1]]:
22                    postList.append(opStack.pop())
23                opStack.append(word)
24        while opStack:
25            postList.append(opStack.pop())
26        return postList
27
28    def buildParseTree(infix):
29        postList = postorder(infix)
30        stack = []
31        for word in postList:
32            if word == 'not':
33                child = stack.pop()
34                stack.append(BinaryTree('not', child))
35            elif word in ('True', 'False'):
36                stack.append(BinaryTree(word))
37            else:
38                right, left = stack.pop(), stack.pop()
39                stack.append(BinaryTree(word, left, right))
```

```

40     return stack[-1]
41
42 # 定义运算符优先级
43 priority = {'or': 1, 'and': 2, 'not': 3, 'True': 4, 'False': 4}
44
45 def printTree(tree):
46     """返回 token 列表"""
47     root = tree.root
48     if root in ('True', 'False'):
49         return [root]
50
51     if root == 'not':
52         child = tree.leftChild
53         # 若子优先级更低则加括号
54         child_tokens = printTree(child)
55         if priority[child.root] < priority[root]:
56             child_tokens = ['('] + child_tokens + [')']
57         return ['not'] + child_tokens
58
59     # 二元操作符 and/or
60     left, right = tree.leftChild, tree.rightChild
61     left_tokens = printTree(left)
62     right_tokens = printTree(right)
63     if priority[left.root] < priority[root]:
64         left_tokens = ['('] + left_tokens + [')']
65     if priority[right.root] < priority[root]:
66         right_tokens = ['('] + right_tokens + [')']
67     return left_tokens + [root] + right_tokens
68
69 def main():
70     infix = input().strip()
71     Tree = buildParseTree(infix)
72     print(' '.join(printTree(Tree)))
73
74 if __name__ == "__main__":
75     main()
76

```

直接利用 Python 自带的 `ast` 模块来完成解析与遍历，无需手写语法分析器。

因为 `ast.parse()` 能正确识别 Python 布尔表达式（`and/or/not/True/False`）的语法树。

改写思路（使用 `ast`）

1. 直接用 `ast.parse(expr, mode='eval')` 将输入的布尔表达式解析为抽象语法树（AST）。
2. 递归遍历 AST，根据节点类型（`BoolOp`, `UnaryOp`, `NameConstant` 等）输出字符串。
3. 根据运算符优先级，**仅在必要时加括号**。

优先级设定为：

```

1 not: 3
2 and: 2
3 or : 1

```

当子表达式优先级低于父表达式时，才加括号。

```

1 import ast
2 import sys
3
4 # 优先级: or=1, and=2, not=3
5 PREC = {
6     ast.Or: 1,
7     ast.And: 2,
8     ast.Not: 3
9 }
10
11 def get_prec(node):
12     """返回节点的优先级（数字越大优先级越高）。默认常量/名称优先级最高。"""
13     if isinstance(node, ast.BoolOp):
14         return PREC[type(node.op)]
15     if isinstance(node, ast.UnaryOp):
16         return PREC[type(node.op)]
17     # 常量或其它原子表达式
18     return 4
19
20 def to_str(node, parent_prec=0):
21     """将 AST 节点转为 token 风格的字符串（带空格），根据 parent_prec 决定是否需要外层括号。"""
22     # BoolOp: values 列表 (and/or)
23     if isinstance(node, ast.BoolOp):
24         op = 'and' if isinstance(node.op, ast.And) else 'or'
25         prec = PREC[type(node.op)]
26         parts = [to_str(v, prec) for v in node.values]
27         s = f' {op} '.join(parts)
28         if prec < parent_prec:
29             return f'({s})'
30         return s
31
32     # UnaryOp: only support Not here
33     if isinstance(node, ast.UnaryOp) and isinstance(node.op, ast.Not):
34         prec = PREC[type(node.op)]
35         # 直接让子节点以 prec 作为 parent_prec 决定是否加括号（避免重复加括号）
36         operand_str = to_str(node.operand, prec)
37         return f'not {operand_str}'
38
39     # Constant (Python 3.8+)
40     if isinstance(node, ast.Constant):
41         # 确保 True/False 以首字母大写形式输出
42         val = node.value
43         if val is True:
44             return 'True'
45         if val is False:
46             return 'False'
47         # 一般不会出现其他常量，但保底
48         return str(val)
49
50     # 兼容旧版本 (ast.NameConstant)
51     if hasattr(ast, 'NameConstant') and isinstance(node, ast.NameConstant):
52         if node.value is True:
53             return 'True'
54         if node.value is False:
55             return 'False'
56         return str(node.value)
57
58     # 也兼容像直接写 True/False 作为 Name（极少见）

```

```

59     if isinstance(node, ast.Name):
60         return node.id
61
62     # 比较/其他表达式: 简单用 ast.unparse (若可用) 或回退到手动
63     # 为了本题只需支持布尔表达式 True/False/and/or/not, 所以其余类型抛错
64     raise TypeError(f"Unsupported node type: {type(node)}")
65
66 def main():
67     data = sys.stdin.read().strip()
68     if not data:
69         return
70     # ast.parse 解析为 Expression
71     tree = ast.parse(data, mode='eval')
72     out = to_str(tree.body)
73     print(out)
74
75 if __name__ == '__main__':
76     main()
77

```

思路：只依赖运算符优先级，不建AST，一遍扫描、直接根据运算符优先级输出去除不必要括号的中缀表达式。

这其实就是一个 **Shunting Yard** 的变体：仍然利用两个栈（操作数、操作符），但在“弹出并组合”时直接构造字符串，而不是树。
关键是：在构造时根据优先级动态决定是否加括号。

最简洁、可通过、仅用优先级控制的版本

```

1  # T20576: printExp. 不建AST, 直接基于运算符优先级一遍扫描输出最简括号中缀表达式
2
3  def priority(op):
4      if op == 'not':
5          return 3
6      if op == 'and':
7          return 2
8      if op == 'or':
9          return 1
10     return 0
11
12 def is_op(x):
13     return x in ('not', 'and', 'or')
14
15 expr = input().strip()
16 tokens = expr.replace('(', ' ( ').replace(')', ' ) ').split()
17
18 vals, ops = [], []
19
20 def apply():
21     op = ops.pop()
22     if op == 'not':
23         a, pa = vals.pop()
24         # 右操作数优先级 < 当前优先级 才加括号
25         if pa < priority(op):
26             a = f"({a})"
27         vals.append((f"not {a}", priority(op)))

```

```

28     else:
29         b, pb = vals.pop()
30         a, pa = vals.pop()
31         # 左右侧根据优先级判断是否加括号
32         if pa < priority(op):
33             a = f"({a})"
34         if pb <= priority(op):
35             b = f"({b})"
36         vals.append((f"{a} {op} {b}", priority(op)))
37
38 for t in tokens:
39     if t == '(':
40         ops.append(t)
41     elif t == ')':
42         while ops and ops[-1] != '(':
43             apply()
44         ops.pop()
45     elif is_op(t):
46         while ops and is_op(ops[-1]) and \
47             ((t != 'not' and priority(ops[-1]) >= priority(t)) or
48              (t == 'not' and priority(ops[-1]) > priority(t))):
49             apply()
50         ops.append(t)
51     else:
52         vals.append((t, 4)) # 操作数优先级最高 (常数4)
53
54 while ops:
55     apply()
56
57 print(vals[-1][0])

```

说明

- 两个栈：
 - `vals` 存 (表达式字符串, 优先级)；
 - `ops` 存操作符。
- `apply()`：
每次弹出一个操作符，根据优先级决定是否给子表达式加括号。
- 优先级规则：

```
1 | not > and > or
```

`not` 是右结合（所以当后面的操作符优先级相等时不提前计算）。

- 括号控制逻辑：
 - 左子式：若优先级 < 当前操作符，则加括号。
 - 右子式：若优先级 ≤ 当前操作符，则加括号。
 - 单目 `not`：若子式优先级 < 当前操作符，则加括号。

解读：

```

1 elif is_op(t):
2     while ops and is_op(ops[-1]) and \
3         ((t != 'not' and priority(ops[-1]) >= priority(t)) or
4          (t == 'not' and priority(ops[-1]) > priority(t))):
5         apply()
6     ops.append(t)

```

非常好的问题

这几行是整个算法的**核心关键**，控制了**操作符出栈的时机**（即：当前要进来的运算符 `t`，是否要先让栈顶运算符计算完）。

我们来逐句完全拆解说明。

背景：Shunting Yard 的“运算符优先级处理规则”

Shunting Yard（调度场算法）：

当读到一个新的运算符时，要决定是否把**栈顶运算符**先弹出（并计算），还是**暂时压栈**等待以后处理。

逻辑：

- 如果**栈顶运算符的优先级 $\geq$ 当前运算符**，并且两者是左结合 $\rightarrow$ 弹出。
- 如果**栈顶运算符的优先级 $>$ 当前运算符**，并且两者是右结合 $\rightarrow$ 弹出。
- 否则，直接把当前运算符压栈。

我们的代码片段

```

1 elif is_op(t):
2     while ops and is_op(ops[-1]) and \
3         ((t != 'not' and priority(ops[-1]) >= priority(t)) or
4          (t == 'not' and priority(ops[-1]) > priority(t))):
5         apply()
6     ops.append(t)

```

一行一行解释

1 `elif is_op(t):`

如果当前 token 是运算符（`not`，`and`，`or`）。

2 `while ops and is_op(ops[-1]) and ...`

只要：

- 栈中还有运算符；
- 栈顶也是运算符（而不是括号）；
- 满足**优先级出栈条件**（下一步解释），就先执行 `apply()`。

也就是说，我们在进入新的运算符前，会清理掉该弹出的旧操作符。

3 出栈条件（最重要）


```
1 | ((t != 'not' and priority(ops[-1]) >= priority(t)) or
2 | (t == 'not' and priority(ops[-1]) > priority(t)))
```

意思是：

当前运算符	栈顶运算符	何时弹出栈顶？
and / or (左结合)	优先级 $\geq$ 当前	弹出
not (右结合)	优先级 $>$ 当前	弹出

解释：

- **左结合运算符** (and, or) → 如果栈顶优先级相同或更高，就该先算；
- **右结合运算符** (not) → 只在栈顶优先级更高时才弹出，
避免例如 not not True 被提前计算成 (not not) True 这样的错误。

4 apply()

执行一次实际的计算（在我们这里，是拼接表达式字符串）。
比如遇到：

```
1 | not True or False
```

当处理 or 时，
会先弹出 not（因为 not 优先级更高），执行 apply() 变为 "not True"。

5 ops.append(t)

最后把当前运算符压栈（因为还没执行完）。

举个例子理解整个 while 循环

输入：

```
1 | not True or False
```

扫描到每个 token：

token	ops	vals	动作
not	[] → ['not']	[]	压入 not
True	['not']	[('True',4)]	压入值
or	['not'] → [] → ['or']	[('not True',3)]	因为栈顶 not 优先级高，先 apply 一次，然后压 or
False	['or']	[('not True',3), ('False',4)]	压入值

最终 apply 'or'：

→ 输出 not True or False

对比右结合例子

输入：

```
1 | not not True
```

当处理第二个 `not` 时：

- 栈顶运算符也是 `not`
- 由于 `not` 是右结合
- 出栈条件是 `priority(ops[-1]) > priority(t)` （严格大于）
- 这时两者优先级相等 → 不会弹出

所以两个 `not` 会顺序压栈：

```
1 | ops = ['not', 'not']
```

等遇到 `True` 后再按顺序 `apply` → 正确输出 `not not True`。

如果我们用 `>=`，就会错误地提前算出 `(not not) True`。

小结表格

运算符	结合性	出栈条件	解释
<code>not</code>	右结合	栈顶优先级 > 当前	保证 <code>not not x</code> 不被提前执行
<code>and, or</code>	左结合	栈顶优先级 ≥ 当前	保证 <code>A and B and C</code> 从左往右算

4.3 哈夫曼编码Huffman code

2013-book-Data Structures And Algorithms In Python

在这一节中，考虑一个重要的文本处理任务：文本压缩。在这个问题中，我们得到一个在某些字母表（如ASCII字符集）上定义的字符串 X ，并希望将其高效地编码为一个较小的二进制字符串 Y （仅使用0和1字符）。文本压缩在任何希望减少数字通信带宽的情况下都非常有用，以便最小化传输文本所需的时间。同样，文本压缩对于更高效地存储大型文档也很有用，从而允许固定容量的存储设备包含尽可能多的文档。

本节探讨的文本压缩方法是**Huffman code**。像ASCII这样的标准编码方案使用固定长度的二进制字符串来编码字符（在传统的或扩展的ASCII系统中分别为7位或8位）。Huffman code通过使用较短的码字code-word 字符串编码高频字符和较长的码字字符串编码低频字符来节省空间。此外，Huffman code使用专门针对给定字母表上的任何字符串 x 优化的可变长度编码variable-length encoding。这种优化基于字符频率的使用，对于每个字符 c ，其在字符串 X 中出现次数的计数为 $f(c)$ 。

为了编码字符串 x ，将其中的每个字符转换为可变长度的码字，并按顺序连接所有这些码字以生成 x 的编码 y 。为了避免歧义，在编码中没有任何码字是另一个码字的前缀。这样的编码被称为**前缀编码prefix code**，它简化了将 Y 解码回 X 的过程。（参见图13.9。）即使有这个限制，使用可变长度前缀码所能达到的节省也是非常显著的，特别是如果字符频率存在较大的差异时（几乎每种书面语言的自然语言文本都是这种情况）。

用于为 x 生成最优可变长度前缀码的 Huffman's algorithm 基于构造表示该代码的二叉树 T 。 T 中的每条边代表码字中的一位，到左孩子的边代表“0”，到右孩子的边代表“1”。每个叶子 v 与特定字符关联，该字符的码字由从 T 的根到 v 路径上的边所关联的比特序列定义。（参见图13.9。）每个叶子 v 都有一个频率 $f(v)$ ，即与 v 关联的字符在 x 中的频率。此外，我们给 T 中的每个内部节点 v 赋予一个频率 $f(v)$ ，它是以 v 为根的子树中所有叶子的频率之和。

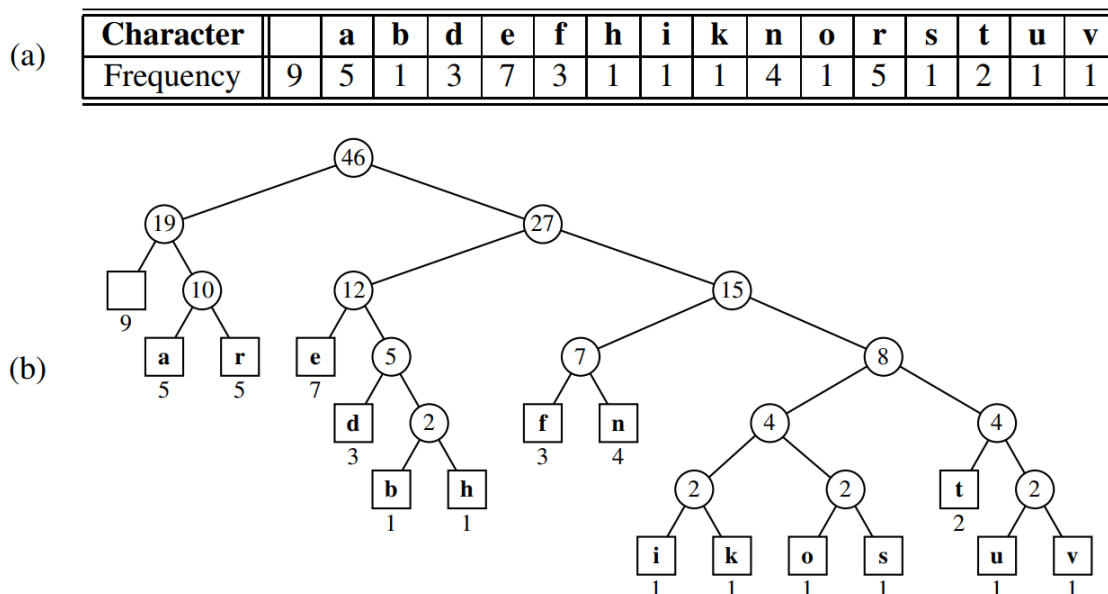


图13.9：输入字符串 $X = \text{"a fast runner need never be afraid of the dark"}$ 的一个Huffman code示例说明：(a) X 中每个字符的频率；(b) 字符串 X 的哈夫曼树 T 。字符 c 的编码是通过从 T 的根节点追踪到存储 c 的叶子节点的路径获得的，其中左孩子边用0表示，右孩子边用1表示。例如，字符 r 的编码是011，而字符 h 的编码是10111。

哈夫曼树通常是一棵“正则二叉树”（也叫满二叉树，Strictly Binary Tree）。

为了让你彻底理清，我们需要区分三个容易混淆的概念：

1.概念辨析

- **完全二叉树 (Complete Binary Tree):**
 - 除了最后一层外，每一层都是满的。
 - 最后一层的节点都靠左排列。
 - 哈夫曼树并不保证这一点，它的形状完全由权值决定。
- **正则二叉树 / 严格二叉树 (Full/Strict Binary Tree):**
 - 每个节点要么没有子节点（叶子节点），要么有两个子节点。
 - 哈夫曼树一定是正则二叉树。因为它通过每次合并“两个”节点形成的，不存在只有一个子节点的中间节点。
- **满二叉树 (Perfect Binary Tree):**
 - 所有层全满。哈夫曼树只有在节点数恰好是 2^k 且权值全部相等时才可能是满二叉树。

2.为什么哈夫曼树不是完全二叉树?

完全二叉树追求的是结构的紧凑性（高度最小），而哈夫曼树追求的是带权路径长度（WPL）最小。

案例 A：权值差距巨大（会生成“斜树”）

假设权值为：{1, 10, 100, 1000}

1. 合并 1 和 10 $\rightarrow$ 11
2. 合并 11 和 100 $\rightarrow$ 111
3. 合并 111 和 1000 $\rightarrow$ 1111

形状：这棵树会像一个长长的梯子（高度为 3），非常不平衡。而 4 个叶子节点的完全二叉树高度仅为 2。

案例 B：权值非常接近（会趋近“完全二叉树”）

假设权值为：{5, 5, 5, 5}

1. 合并 5 和 5 $\rightarrow$ 10
2. 合并 5 和 5 $\rightarrow$ 10
3. 合并 10 和 10 $\rightarrow$ 20

形状：这是一棵完美的平衡树，此时它既是哈夫曼树，也是完全二叉树。

3. 一个重要的结论

在哈夫曼算法的上下文中，我们通常称它为“扩充二叉树”（Extended Binary Tree）。

- 外部节点（External Nodes）：就是你输入的 n 个权值，最终成为叶子节点。
- 内部节点（Internal Nodes）：是合并过程中产生的节点，总共有 $n - 1$ 个。
- 总节点数： $n + (n - 1) = 2n - 1$ 。

总结记忆：

- 哈夫曼树的形状是“按权值定制”的。
- 权值越平均，哈夫曼树越像完全二叉树（矮胖）。
- 权值越悬殊，哈夫曼树越偏离完全二叉树，甚至变成斜树（瘦高）。
- 但无论如何，它绝不会出现“只有一个孩子”的节点。

4.3.1 哈夫曼编码算法 (The Huffman Coding Algorithm)

哈夫曼算法的起点是字符串 X 中的 d 个唯一字符，每个字符最初都被视为一棵单节点二叉树。算法通过一系列迭代步骤完成：在每一轮中，算法选取两棵频率最低的二叉树，将其合并为一棵新树。重复此过程，直至所有节点最终合并为一棵树。

在实现上，哈夫曼算法的 **while** 循环可以利用基于堆的优先队列，在 $O(\log d)$ 时间内完成每次迭代。算法共需进行 $d - 1$ 次迭代（每次取出两个节点并插入一个新节点），直至队列中只剩一个节点。因此，该算法的总运行时间为 $O(n + d \log d)$ ，其中 n 为字符串长度。

n 的来源（频率统计）和 $d \log d$ 的来源（堆操作）

核心直觉源于一个简单的推论：在任何最优编码方案中，频率最低的两个字符（设为 a 和 b ）的码字应当仅在最后一位比特有所不同。通过将 a, b 合并为一个新字符 c ，并在规模缩减后的字符集上重复这一逻辑，即可推导出最优性：

命题 (Proposition)：对于长度为 n 、包含 d 个不同字符的字符串，哈夫曼算法可以在 $O(n + d \log d)$ 时间内构造出一棵最优前缀码（Optimal Prefix Code）树。

4.3.2 哈夫曼编码：贪心策略的深度解析

1. 贪心法 (The Greedy Method)

哈夫曼算法是贪心法的典型应用。贪心法的核心是通过一系列局部最优选择（Local Optimal Choice）来构建解，并期望这些选择能导向全局最优解（Global Optimal Solution）。

一个问题能够使用贪心法解决，通常需要具备两个性质：

1. 贪心选择性质：可以通过局部最优选择达到全局最优。
2. 最优子结构性质：原问题的最优解包含其子问题的最优解。

2. 深度思考：为什么不能直接“排序”？

在处理 WPL（带权路径长度） $\sum W_i \times L_i$ 时，直觉可能会告诉我们：让权值大的节点深度小，权值小的节点深度大。

关于斜线树 vs. 平衡树的对比，非常有助于理解哈夫曼树的本质。

- **错误猜想**：构造一棵“斜树”，按权值排序，最大权值深度为 1，次大为 2.....
- **反例推翻**：若所有权值相等（如 $n = 8$, 权值均为 t ）。
 - 斜树 WPL: $(1 + 2 + 3 + 4 + 5 + 6 + 7 + 7) \times t = 35t$
 - 完全二叉树 WPL: $8 \times 3 \times t = 24t$
- **结论**：哈夫曼树的形态（深度分配）不仅取决于权值的**相对顺序**，还取决于权值的**具体数值累加情况**。

3. 核心证明：哈夫曼算法的正确性

我们要证明：哈夫曼算法每次合并两个权值最小的节点，最终生成的树 WPL 最小。

性质一：贪心选择性质 (Greedy Choice Property)

命题：对于给定的 n 个权值，一定存在一棵最优二叉树，其中权值最小的两个节点（设为 x, y ）是深度最深的兄弟。

证明（交换论证法, Exchange Argument）：

1. 假设一棵最优树 T 中，深度最深的两个兄弟节点是 a 和 b 。
2. 不妨设权值顺序为 $x \leq y \leq \dots \leq a \leq b$ (x, y 是最小的两个)。
3. 如果在 T 中， x, y 不是 a, b ，我们交换 x 与 a ， y 与 b 得到新树 T' 。
4. 比较 WPL 的变化：
$$\Delta WPL = WPL(T') - WPL(T) = (x \cdot d_a + a \cdot d_x) - (x \cdot d_x + a \cdot d_a)$$

(其中 d 为深度) 整理得：
$$\Delta WPL = (a - x)(d_x - d_a)$$
5. 因为 d_a 是最大深度，所以 $d_x \leq d_a \implies (d_x - d_a) \leq 0$ 。又因为 x 是最小权值，所以 $(a - x) \geq 0$ 。
6. 因此 $\Delta WPL \leq 0$ 。这说明交换后的树 T' 不会比原最优树 T 更差。
7. 同理交换 y 与 b 依然成立。由此证明：**让最小的两个权值在一起作为最深叶子，是通往全局最优的正确第一步。**

性质二：最优子结构 (Optimal Substructure)

命题：将 x, y 合并为新节点 $z = x + y$ 后，归约为 $n - 1$ 个规模的问题，新问题的最优解包含原问题的最优解。

证明：

1. 设树 T 是由 n 个节点构成的，其中 x, y 是兄弟，其父亲为 z 。
2. $WPL(T)$ 可以表示为： $WPL(T) = WPL(T - \{x, y\} \cup \{z\}) + x + y$ 。
3. 因为 $(x + y)$ 是常数，要使 $WPL(T)$ 最小，只需使 $n - 1$ 个权值（原 $n - 2$ 个加上合并后的 z ）构成的树 T' 的 WPL 最小。
4. 这证明了我们可以通过递归/迭代的方式，不断缩小问题规模。

4.3.3 哈夫曼编码实现

要构建一个最优的哈夫曼编码树，首先需要对给定的字符及其权值进行排序。然后，通过重复合并权值最小的两个节点（或子树），直到所有节点都合并为一棵树为止。

下面是用 Python 实现的代码：

```

1 import heapq
2
3 class Node:
4     def __init__(self, weight, char=None):
5         self.weight = weight
6         self.char = char
7         self.left = None
8         self.right = None
9
10    def __lt__(self, other):
11        if self.weight == other.weight:
12            return self.char < other.char
13        return self.weight < other.weight
14
15    def huffman_encoding(char_freq):
16        heap = [Node(freq, char) for char, freq in char_freq.items()]
17        heapq.heapify(heap)
18
19        while len(heap) > 1:
20            left = heapq.heappop(heap)
21            right = heapq.heappop(heap)
22            merged = Node(left.weight + right.weight, min(left.char, right.char))
23            merged.left = left
24            merged.right = right
25            heapq.heappush(heap, merged)
26
27        return heap[0]
28
29    def external_path_length(node, depth=0):
30        if node is None:
31            return 0
32        if node.left is None and node.right is None:
33            return depth * node.weight
34        return (external_path_length(node.left, depth + 1) +
35                external_path_length(node.right, depth + 1))
36
37    def main():
38        char_freq = {'a': 3, 'b': 4, 'c': 5, 'd': 6, 'e': 8, 'f': 9, 'g': 11, 'h': 12}
39        huffman_tree = huffman_encoding(char_freq)
40        external_length = external_path_length(huffman_tree)
41        print("The weighted external path length of the Huffman tree is:", external_length)
42
43    if __name__ == "__main__":
44        main()
45
46    # Output:
47    # The weighted external path length of the Huffman tree is: 169

```

这段代码首先定义了一个 `Node` 类来表示哈夫曼树的节点。然后，使用最小堆来构建哈夫曼树，每次从堆中取出两个频率最小的节点进行合并，直到堆中只剩下一个节点，即哈夫曼树的根节点。接着，使用递归方法计算哈夫曼树的带权外部路径长度（weighted external path length）。最后，输出计算得到的带权外部路径长度。

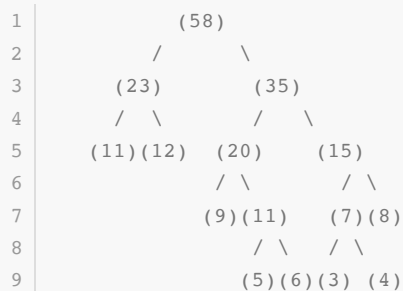
可以运行这段代码来得到该最优二叉编码树的带权外部路径长度。

Q: 用 Huffman 算法构造一个最优二叉编码树，待编码的字符权值分别为{3, 4, 5, 6, 8, 9, 11, 12}，请问该最优二叉编码树的带权外部路径长度为（ B ）。（补充说明：树的带权外部路径长度定义为树中所有叶子结点的带权路径长度之和；其中，结点的带权路径长度定义为该结点到树根之间的路径长度与该结点权值的乘积）

A: 58 B: 169 C: 72 D: 18

解释：为了构造哈夫曼树，我们遵循一个重复的选择过程，每次选择两个最小的权值创建一个新的节点，直到只剩下一个节点为止。我们可以按照以下步骤操作：

1. 将给定的权值排序：{3, 4, 5, 6, 8, 9, 11, 12}。
2. 选择两个最小的权值：3 和 4，将它们组合成一个新的权值为 7 的节点。
现在权值变为：{5, 6, 7, 8, 9, 11, 12}。
3. 再次选择两个最小的权值：5 和 6，将它们组合成一个新的权值为 11 的节点。
现在权值变为：{7, 8, 9, 11, 11, 12}。
4. 选择两个最小的权值：7 和 8，将它们组合成一个新的权值为 15 的节点。
现在权值变为：{9, 11, 11, 12, 15}。
5. 选择两个最小的权值：9 和 11，将它们合并成一个新的权值为 20 的节点。
现在权值变为：{11, 12, 15, 20}。
6. 选择两个最小的权值：11 和 12，合并成一个新的权值为 23 的节点。
现在权值变为：{15, 20, 23}。
7. 选择两个最小的权值：15 和 20，合并成一个新的权值为 35 的节点。
现在权值变为：{23, 35}。
8. 最后，合并这两个节点得到根节点，权值为 $23 + 35 = 58$ 。
现在我们可以计算哈夫曼树的带权外部路径长度（WPL）。



现在让我们计算每个叶子节点的带权路径长度：

- 权值 3 的节点路径长度为 4，WPL 部分为 $3 * 4 = 12$ 。
- 权值 4 的节点路径长度为 4，WPL 部分为 $4 * 4 = 16$ 。
- 权值 5 的节点路径长度为 4，WPL 部分为 $5 * 4 = 20$ 。
- 权值 6 的节点路径长度为 4，WPL 部分为 $6 * 4 = 24$ 。
- 权值 9 的节点路径长度为 3，WPL 部分为 $9 * 3 = 27$ 。
- 权值 8 的节点路径长度为 3，WPL 部分为 $8 * 3 = 24$ 。
- 权值 11 的节点路径长度为 2，WPL 部分为 $11 * 2 = 22$ 。
- 权值 12 的节点路径长度为 2，WPL 部分为 $12 * 2 = 24$ 。

将所有部分的 WPL 相加，我们得到整棵哈夫曼树的 WPL：

$$WPL = 12 + 16 + 20 + 24 + 27 + 24 + 22 + 24 = 169$$

4.3.4 编程题目

示例04080:Huffman编码树

greedy, <http://cs101.openjudge.cn/practice/04080/>

构造一个具有n个外部节点的扩充二叉树，每个外部节点 K_i 有一个 W_i 对应，作为该外部节点的权。使得这个扩充二叉树的叶节点带权外部路径长度总和最小：

$$\text{Min}(W_1 * L_1 + W_2 * L_2 + W_3 * L_3 + \dots + W_n * L_n)$$

W_i :每个节点的权值。

L_i :根节点到第i个外部叶子节点的距离。

编程计算最小外部路径长度总和。

输入

第一行输入一个整数n，外部节点的个数。第二行输入n个整数，代表各个外部节点的权值。

$2 \leq n \leq 100$

输出

输出最小外部路径长度总和。

样例输入

```
1 | 4
2 | 1 1 3 5
```

样例输出

```
1 | 17
```

这个问题其实是一个经典的 **最优二叉树** 构造问题，也叫做 **霍夫曼编码 (Huffman Coding)**。要构造一棵 **扩充二叉树**（即每个非叶子节点都有两个子节点），使得所有 **叶子节点的带权路径长度和最小**。

思路简述（霍夫曼算法）：

通过以下贪心策略构造一棵最优二叉树：

1. 将所有权值作为初始节点，放入一个最小堆中。
2. 重复执行以下操作直到只剩一个节点：
 - 从堆中取出两个最小权值节点 a 和 b
 - 合并为一个新节点，权值为 $a + b$
 - 把这个新节点的权值加入堆
 - 这次合并会产生一个代价： $a + b$ ，将其加入总路径代价中
3. 最终累加的合并代价即为 **最小外部带权路径长度总和**。

示例解释

输入：

```
1 | 4
2 | 1 1 3 5
```

构造过程：

- $1 + 1 \rightarrow$ 新节点权值 2，总成本 $+= 2$
- $2 + 3 \rightarrow$ 新节点权值 5，总成本 $+= 5$
- $5 + 5 \rightarrow$ 新节点权值 10，总成本 $+= 10$
- 最终成本 $= 2 + 5 + 10 = 17$

5. Python 实现（优化版）

代码逻辑已经很简洁，利用 `heapq` 实现 $\mathcal{O}(n \log n)$ 。

```
1 import heapq
2
3 def solve():
4     try:
5         line1 = input().split()
6         if not line1: return
7         n = int(line1[0])
8         weights = list(map(int, input().split()))
9     except EOFError:
10         return
11
12     # 转化为最小堆
13     heapq.heapify(weights)
14
15     total_wpl = 0
16     # 贪心选择：每次弹出两个最小的
17     while len(weights) > 1:
18         first = heapq.heappop(weights)
19         second = heapq.heappop(weights)
20
21         combined = first + second
22         # 这里的 total_wpl 累加实际上等同于计算所有非叶节点的权值和
23         # 数学上：WPL = 所有非叶节点的权值之和
24         total_wpl += combined
25
26         heapq.heappush(weights, combined)
27
28     print(total_wpl)
29
30 if __name__ == "__main__":
31     solve()
```

在哈夫曼树问题中，“所有非叶节点的权值之和”恰好等于“所有叶子节点的带权路径长度之和”。这是一个非常实用的数学结论，简化了代码实现的逻辑。

greedy, <http://cs101.openjudge.cn/practice/22161/>

根据字符使用频率(权值)生成一棵唯一的哈夫曼编码树。生成树时需要遵循以下规则以确保唯一性:

选取最小的两个节点合并时, 节点比大小的规则是:

1. 权值小的节点算小。权值相同的两个节点, 字符集里最小字符小的, 算小。

例如 $(\{'c','k'\}, 12)$ 和 $(\{'b','z'\}, 12)$, 后者小。

2. 合并两个节点时, 小的节点必须作为左子节点

3. 连接左子节点的边代表0, 连接右子节点的边代表1

然后对输入的串进行编码或解码

输入

第一行是整数n, 表示字符集有n个字符。

接下来n行, 每行是一个字符及其使用频率(权重)。字符都是英文字母。

再接下来是若干行, 有的是字母串, 有的是01编码串。

输出

对输入中的字母串, 输出该字符串的编码

对输入中的01串, 将其解码, 输出原始字符串

样例输入

```
1 3
2 g 4
3 d 8
4 c 10
5 dc
6 110
```

样例输出

```
1 110
2 dc
```

提示: 数据规模很小, 不用在乎效率

来源: 郭炜

建树: 主要利用最小堆, 每次取出weight最小的两个节点, weight相加后创建节点, 连接左右孩子, 再入堆, 直至堆中只剩一个节点。

编码: 跟踪每一步走的是左还是右, 用0和1表示, 直至遇到有char值的节点, 说明到了叶子节点, 将01字符串添加进字典。

解码: 根据01字符串决定走左还是右, 直至遇到有char值的节点, 将char值取出。

```
1 import heapq
2
3 class Node:
4     def __init__(self, weight, char=None):
5         self.weight = weight
6         self.char = char
7         self.left = None
```

```

8         self.right = None
9
10    def __lt__(self, other):
11        if self.weight == other.weight:
12            return self.char < other.char
13        return self.weight < other.weight
14
15    def build_huffman_tree(characters):
16        heap = []
17        for char, weight in characters.items():
18            heapq.heappush(heap, Node(weight, char))
19
20        while len(heap) > 1:
21            left = heapq.heappop(heap)
22            right = heapq.heappop(heap)
23            #merged = Node(left.weight + right.weight) #note: 合并后, char 字段默认值是空
24            merged = Node(left.weight + right.weight, min(left.char, right.char))
25            merged.left = left
26            merged.right = right
27            heapq.heappush(heap, merged)
28
29        return heap[0]
30
31    def encode_huffman_tree(root):
32        codes = {}
33
34        def traverse(node, code):
35            #if node.char:
36            if node.left is None and node.right is None:
37                codes[node.char] = code
38            else:
39                traverse(node.left, code + '0')
40                traverse(node.right, code + '1')
41
42        traverse(root, '')
43        return codes
44
45    def huffman_encoding(codes, string):
46        encoded = ''
47        for char in string:
48            encoded += codes[char]
49        return encoded
50
51    def huffman_decoding(root, encoded_string):
52        decoded = ''
53        node = root
54        for bit in encoded_string:
55            if bit == '0':
56                node = node.left
57            else:
58                node = node.right
59
60            #if node.char:
61            if node.left is None and node.right is None:
62                decoded += node.char
63                node = root
64        return decoded
65

```

```

66 # 读取输入
67 n = int(input())
68 characters = {}
69 for _ in range(n):
70     char, weight = input().split()
71     characters[char] = int(weight)
72
73 #string = input().strip()
74 #encoded_string = input().strip()
75
76 # 构建哈夫曼编码树
77 huffman_tree = build_huffman_tree(characters)
78
79 # 编码和解码
80 codes = encode_huffman_tree(huffman_tree)
81
82 strings = []
83 while True:
84     try:
85         line = input()
86         strings.append(line)
87
88     except EOFError:
89         break
90
91 results = []
92 #print(strings)
93 for string in strings:
94     if string[0] in ('0', '1'):
95         results.append(huffman_decoding(huffman_tree, string))
96     else:
97         results.append(huffman_encoding(codes, string))
98
99 for result in results:
100     print(result)

```

三、堆, BST, AVL, 并查集

5 利用二叉堆实现优先级队列

我们学习过队列这一先进先出的数据结构。队列有一个重要的变体，叫作优先级队列。和队列一样，优先级队列从头部移除元素，不过元素的逻辑顺序是由优先级决定的。优先级最高的元素在最前，优先级最低的元素在最后。因此，当一个元素入队时，它可能直接被移到优先级队列的头部。后续学习图相关内容时会看到，对于一些图算法来说，优先级队列是一个有用的数据结构。

你或许可以想到一些使用排序函数和列表实现优先级队列的简单方法。但是，就时间复杂度而言，列表的插入操作是 $O(n)$ ，排序操作是 $O(n \log n)$ 。其实，效率可以更高。实现优先级队列的经典方法是使用叫作二叉堆的数据结构。二叉堆的入队操作和出队操作均可达到 $O(\log n)$ 。

二叉堆学起来很有意思，它画出来很像一棵树，但实现时只用一个列表作为内部表示。我们将实现最小堆（最小的元素一直在队首）。

5.1 二叉堆的实现

1. 结构性

为了使二叉堆能高效地工作，我们利用树的对数性质来表示它。为了保证对数性能，必须维持树的平衡。平衡的二叉树是指，其根节点的左右子树含有数量大致相等的节点。在实现二叉堆时，我们通过创建一棵**完全二叉树complete binary tree**来维持树的平衡。在完全二叉树中，除了最底层，其他每一层的节点都是满的。在最底层，我们从左往右填充节点。图 1 展示了完全二叉树的一个例子。

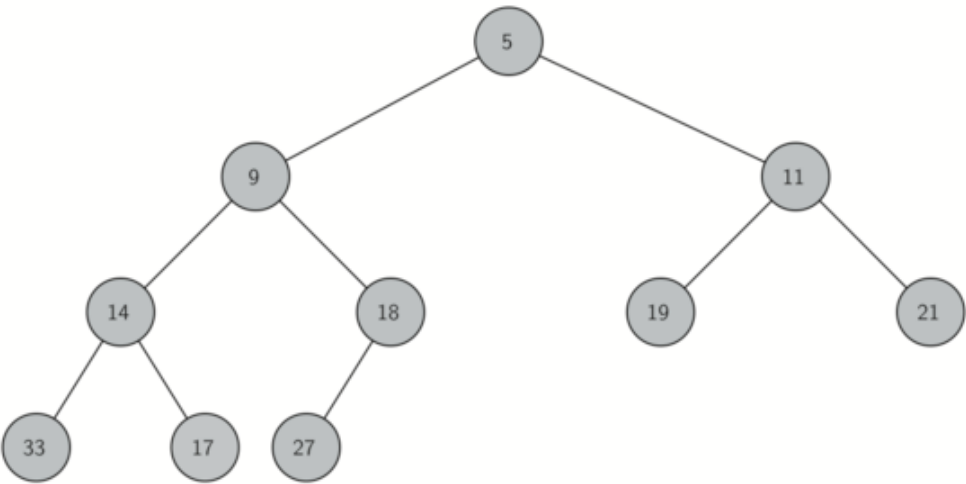


图 1 完全二叉树

完全二叉树的另一个有趣之处在于，**可以用一个列表来表示它**，而不需要采用“列表之列表”或“节点与引用”表示法。由于树是完全的，因此对于在列表中处于位置 p 的节点来说，它的左子节点正好处于位置 $2p+1$ ；同理，右子节点处于位置 $2p+2$ 。若要找到树中任意节点的父节点，只需使用 Python 的整数除法即可。给定列表中位置 n 处的节点，其父节点的位置就是 $(n-1)//2$ 。图 2 展示了一棵完全二叉树，并给出了列表表示。**树的列表表示**——加上这个“完全”的结构性质——让我们得以通过一些简单的数学运算遍历完全二叉树。会看到，这也有助于高效地实现二叉堆。

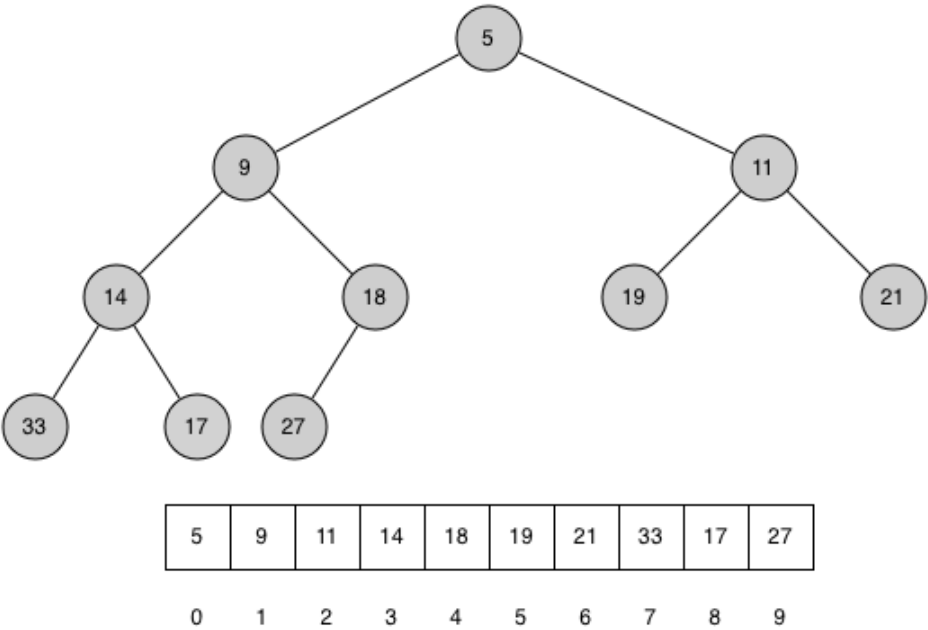


图 2 一棵完全二叉树及其列表表示

2. 堆的有序性

我们用来存储堆元素的方法依赖于堆的有序性。**堆的有序性是指：对于堆中任意元素 x 及其父元素 p ， p 都不大于 x 。**图 2 也展示出完全二叉树具备堆的有序性。

3. 堆操作

首先实现二叉堆的构造方法。既然用一个列表就可以表示整个二叉堆，那么构造方法要做的就是初始化这个列表。

```
1 class BinaryHeap:
2     def __init__(self):
3         self._heap = []
```

接下来实现 insert 方法。将元素加入列表的最简单、最高效的方法就是将元素追加到列表的末尾。追加操作的优点在于，它能保证完全树的性质，但缺点是很可能会破坏堆的结构性质。不过可以**写一个方法，通过比较新元素与其父元素来重新获得堆的有序性。**如果新元素小于其父元素，就将二者交换。图3展示了将新元素放到正确位置上所需的一系列交换操作。

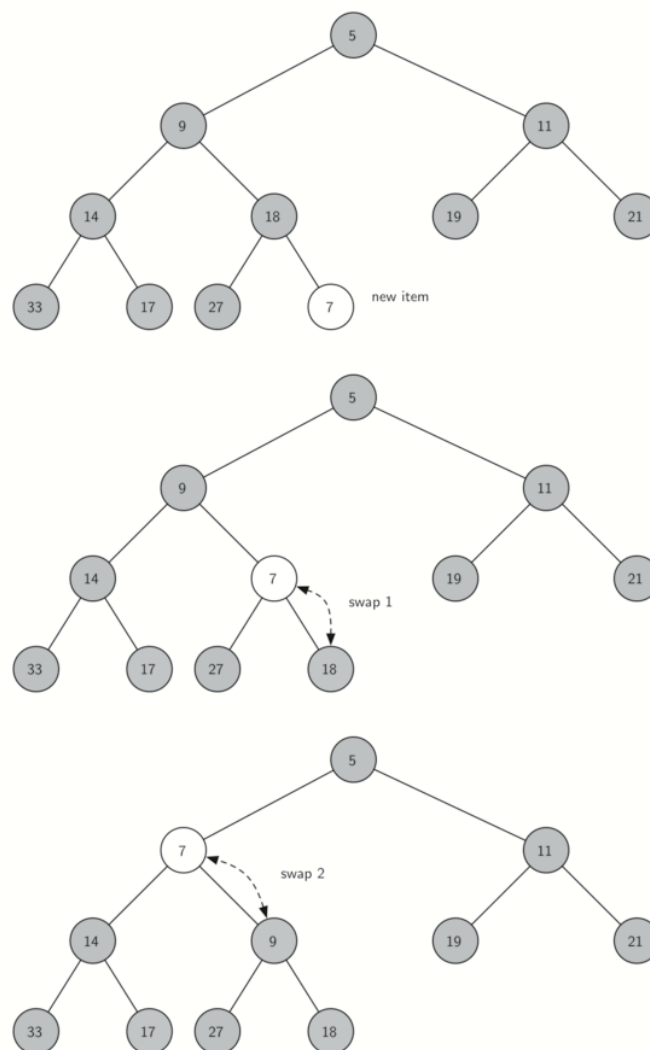


图3 将新元素往上移到正确位置 Percolate the New Node up to Its Proper Position

注意，**将元素往上移时，其实是在新元素及其父元素之间重建堆的性质。**此外，也保留了兄弟元素之间的堆性质。当然，如果新元素很小，需要继续往上一层交换。代码清单2给出了 `_perc_up` 方法的代码，该方法将元素一直沿着树向上移动，直到重获堆的性质。我们使用整数除法计算任意节点的父节点。就当前节点而言，父节点的下标就是(当前节点的下标-1)除以2。

代码清单2

```
1 def _perc_up(self, i):
2     while (i - 1) // 2 >= 0:
3         parent_idx = (i - 1) // 2
4         if self._heap[i] < self._heap[parent_idx]:
5             self._heap[i], self._heap[parent_idx] = (
6                 self._heap[parent_idx],
7                 self._heap[i],
8             )
9         i = parent_idx
```

现在准备好编写insert方法了。代码清单3给出了该方法的Python代码。其实，insert方法的大部分工作是由 `_perc_up` 方法完成的。当元素被追加到树中之后，`_perc_up` 方法将其移到正确的位置。

代码清单3

```
1 def insert(self, item):
2     self._heap.append(item)
3     self._perc_up(len(self._heap) - 1)
```

正确定义insert方法后，就可以编写delete方法。既然堆的有序性质要求根节点是树的最小元素，那么查找最小值就很简单。delete方法的难点在于，如何在移除根节点之后重获堆的结构性质和有序性。可以分两步重建堆。第一步，取出列表中的最后一个元素，将其移到根节点的位置。移动最后一个元素保证了堆的结构性质，但可能会破坏二叉堆的有序性。第二步，将新的根节点沿着树推到正确的位置，以重获堆的有序性。图4展示了将新的根节点移动到正确位置所需的一系列交换操作。

二叉堆BinHeap，明显可以排序Heapsort，树的特点，时间复杂度与归并排序Mergesort一致。取最小元素后，为了恢复有序性，堆顶与列表最后一个元素交换，这种远距离的交换，自然是不稳定No Stable 的排序。

Name	Best	Average	Worst	Memory	Stable	Method	Other notes
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Selection	
Merge sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	Merging	Highly parallelizable (up to $O(\log n)$ using the Three Hungarian's Algorithm)

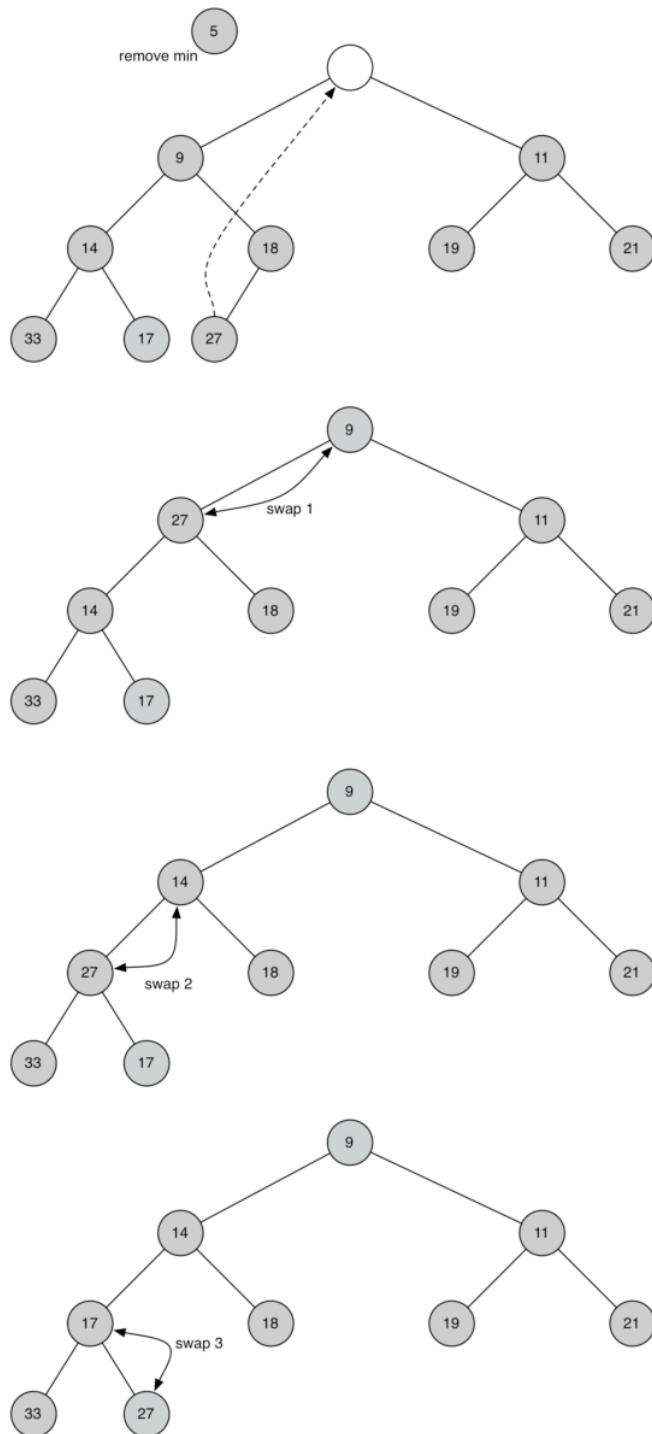


图4 Percolating the Root Node down the Tree

为了维护堆的顺序性质，需要做的就是将根节点与其小于根节点的较小子节点交换。在最初的交换之后，可能会重复这一交换过程，涉及某个节点及其子节点，直到该节点被交换到树中的某个位置，在这个位置上它已经小于其两个子节点。向下渗透节点的代码可以在清单4中的 `_perc_down` 和 `_get_min_child` 方法中找到。

代码清单4

```
1 def _perc_down(self, i):
2     while 2 * i + 1 < len(self._heap):
3         sm_child = self._get_min_child(i)
4         if self._heap[i] > self._heap[sm_child]:
```



```

5         self._heap[i], self._heap[sm_child] = (
6             self._heap[sm_child],
7             self._heap[i],
8         )
9     else:
10        break
11    i = sm_child
12
13    def _get_min_child(self, i):
14        if 2 * i + 2 > len(self._heap) - 1:
15            return 2 * i + 1
16        if self._heap[2 * i + 1] < self._heap[2 * i + 2]:
17            return 2 * i + 1
18        return 2 * i + 2

```

delete方法如代码清单5所示。同样，主要工作也由辅助函数完成。本例中的辅助函数是 `_perc_down`。

代码清单5

```

1    def delete(self):
2        self._heap[0], self._heap[-1] = self._heap[-1], self._heap[0]
3        result = self._heap.pop()
4        self._perc_down(0)
5        return result

```

为了结束我们对二叉堆的讨论，我们将探讨一种从键值列表构建整个堆的方法。你可能首先想到的方法如下：给定一个键值列表，可以通过逐个插入每个键来轻松构建一个堆。由于从一个空列表开始，它是有序的，因此可以使用二分查找以大约 $O(\log n)$ 的操作代价找到插入下一个键的正确位置。然而，要记住，在列表中间插入一个元素可能需要 $O(n)$ 操作来移动列表中的其余元素，以为新键腾出空间。因此，将所有键插入堆中总共需要 $O(n \log n)$ 操作。但是，**如果从一个完整的列表开始，那么我们可以在 $O(n)$ 操作内构建整个堆。**清单6展示了构建整个堆的代码。

如果从完整的列表开始，构建整个堆只需 $O(n)$ ，<https://oi-wiki.org/ds/binary-heap/> 有证明。

注意到叶节点无需调整，因此可从序列约 $n/2$ 的位置开始调整，可减少部分常数但不影响复杂度。之所以能 $O(n)$ 建堆，是因为堆性质很弱，二叉堆并不是唯一的。要是像排序那样的强条件就难说了。

代码清单6

```

1    def heapify(self, not_a_heap):
2        self._heap = not_a_heap[:]
3        i = len(self._heap) // 2 - 1    # 超过中点的节点都是叶子节点
4        while i >= 0:
5            self._perc_down(i)
6            i = i - 1

```

图5展示了buildHeap方法进行的交换过程，它将各节点从最初状态移到各自的正确位置上。尽管从树的中间开始，向根的方向操作，但是 `_perc_down` 方法保证了最大的节点总是沿着树向下移动。在这棵完全二叉树中，超过中点的节点都是叶子节点，没有任何子节点。当 $i = 1$ 时，从树的根节点往下移，可能需要经过多次交换。如你所见，9先被移出根节点，然后 `_perc_down` 会沿着树检查子节点，以确保尽量将它往下移。9的第2次交换对象是3。这样一来，9就移到了树的底层，不需要再做交换了。比较一系列交换操作后的列表表示（打印出来 `print(f'i = {i}, {self._heap}')`）将有助于理解，如图5所示。

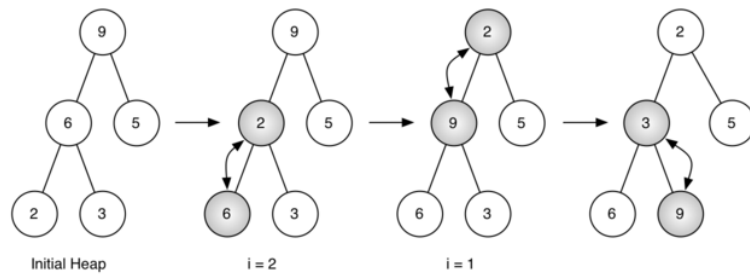


图5 根据列表[9, 6, 5, 2, 3]构建堆

```

1 class BinaryHeap:
2     def __init__(self):
3         self._heap = []
4
5     def _perc_up(self, i):
6         while (i - 1) // 2 >= 0:
7             parent_idx = (i - 1) // 2
8             if self._heap[i] < self._heap[parent_idx]:
9                 self._heap[i], self._heap[parent_idx] = (
10                     self._heap[parent_idx],
11                     self._heap[i],
12                 )
13             i = parent_idx
14
15     def insert(self, item):
16         self._heap.append(item)
17         self._perc_up(len(self._heap) - 1)
18
19     def _perc_down(self, i):
20         while 2 * i + 1 < len(self._heap):
21             sm_child = self._get_min_child(i)
22             if self._heap[i] > self._heap[sm_child]:
23                 self._heap[i], self._heap[sm_child] = (
24                     self._heap[sm_child],
25                     self._heap[i],
26                 )
27             else:
28                 break
29             i = sm_child
30
31     def _get_min_child(self, i):
32         if 2 * i + 2 > len(self._heap) - 1:
33             return 2 * i + 1
34         if self._heap[2 * i + 1] < self._heap[2 * i + 2]:
35             return 2 * i + 1
36         return 2 * i + 2
37
38     def delete(self):
39         self._heap[0], self._heap[-1] = self._heap[-1], self._heap[0]
40         result = self._heap.pop()
41         self._perc_down(0)
42         return result
43
44     def heapify(self, not_a_heap):
45         self._heap = not_a_heap[:]

```

```

46         i = len(self._heap) // 2 - 1    # 超过中点的节点都是叶子节点
47         while i >= 0:
48             print(f'i = {i}, {self._heap}')
49             self._perc_down(i)
50             i = i - 1
51
52
53     bh = BinaryHeap()
54     bh.heapify([9, 5, 6, 2, 3])
55     """
56     i = 1, [9, 5, 6, 2, 3]
57     i = 0, [9, 2, 6, 5, 3]
58     """
59
60     for _ in range(len(bh._heap)):
61         print(bh.delete())
62     """
63     2
64     3
65     5
66     6
67     9
68     """

```

构建堆的时间复杂度为 $O(n)$ ，这在直觉上可能并不显而易见。其核心逻辑在于： $\log n$ 这一项取决于树的高度，但在执行 `heapify`（自底向上建堆）时，绝大多数节点都分布在靠近叶子节点的位置，这些节点需要“下滤”的高度远小于 $\log n$ 。

利用建堆仅需 $O(n)$ 这一特性，我们可以设计出高效的堆排序算法，使其在最坏情况下的总时间复杂度仍能保持在 $O(n \log n)$ 。

5.2 编程题目

示例M04078: 实现堆结构

<http://cs101.openjudge.cn/practice/04078/>

定义一个数组，初始化为空。在数组上执行两种操作：

- 1、增添1个元素，把1个新的元素放入数组。
- 2、输出并删除数组中最小的数。

使用堆结构实现上述功能的高效算法。

输入

第一行输入一个整数 n ，代表操作的次数。

每次操作首先输入一个整数 $type$ 。

当 $type=1$ ，增添操作，接着输入一个整数 u ，代表要插入的元素。

当 $type=2$ ，输出删除操作，输出并删除数组中最小的元素。

$1 \leq n \leq 100000$ 。

输出

每次删除操作输出被删除的数字。

样例输入

```
1 | 4
2 | 1 5
3 | 1 1
4 | 1 7
5 | 2
```

样例输出

```
1 | 1
```

提示

每组测试数据的复杂度为 $O(n\log n)$ 的算法才能通过本次，否则会返回TLE(超时)
需要使用最小堆结构来实现本题的算法

这题目本意是练习自己写个BinaryHeap。当然机考时候，如果遇到这样题目，直接import heapq。

手搓栈、队列、堆、AVL等，考试前需要搓个遍。

```
1 class BinaryHeap:
2     def __init__(self):
3         self._heap = []
4
5     def _perc_up(self, i):
6         while (i - 1) // 2 >= 0:
7             parent_idx = (i - 1) // 2
8             if self._heap[i] < self._heap[parent_idx]:
9                 self._heap[i], self._heap[parent_idx] = (
10                     self._heap[parent_idx],
11                     self._heap[i],
12                 )
13             i = parent_idx
14
15     def insert(self, item):
16         self._heap.append(item)
17         self._perc_up(len(self._heap) - 1)
18
19     def _perc_down(self, i):
20         while 2 * i + 1 < len(self._heap):
21             sm_child = self._get_min_child(i)
22             if self._heap[i] > self._heap[sm_child]:
23                 self._heap[i], self._heap[sm_child] = (
24                     self._heap[sm_child],
25                     self._heap[i],
26                 )
27             else:
28                 break
29             i = sm_child
30
31     def _get_min_child(self, i):
32         if 2 * i + 2 > len(self._heap) - 1:
33             return 2 * i + 1
34         if self._heap[2 * i + 1] < self._heap[2 * i + 2]:
35             return 2 * i + 1
```

```

36         return 2 * i + 2
37
38     def delete(self):
39         self._heap[0], self._heap[-1] = self._heap[-1], self._heap[0]
40         result = self._heap.pop()
41         self._perc_down(0)
42         return result
43
44     def heapify(self, not_a_heap):
45         self._heap = not_a_heap[:]
46         i = len(self._heap) // 2 - 1    # 超过中点的节点都是叶子节点
47         while i >= 0:
48             #print(f'i = {i}, {self._heap}')
49             self._perc_down(i)
50             i = i - 1
51
52
53
54     n = int(input().strip())
55     bh = BinaryHeap()
56     for _ in range(n):
57         inp = input().strip()
58         if inp[0] == '1':
59             bh.insert(int(inp.split()[1]))
60         else:
61             print(bh.delete())

```

晴问9.7: 向下调整构建大顶堆

<https://sunnywhy.com/sfbj/9/7>

现有个不同的正整数，将它们按层序生成完全二叉树，然后使用**向下调整**的方式构建一个完整的大顶堆。最后按层序输出堆中的所有元素。

输入

第一行一个整数 $n(1 \leq n \leq 10^3)$ ，表示正整数的个数；

第二行 n 个整数 $a_i(1 \leq a_i \leq 10^4)$ ，表示正整数序列。

输出

输出 n 个整数，表示堆的层序序列，中间用空格隔开，行末不允许有多余的空格。

样例1

输入

```

1 | 6
2 | 3 2 6 5 8 7

```

输出

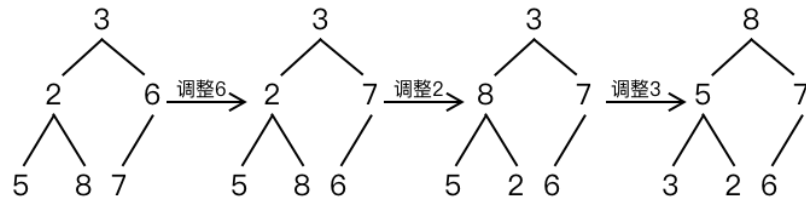
```

1 | 8 5 7 3 2 6

```

解释

调整前的完全二叉树和调整后的堆如下图所示。



```

1 class BinaryHeap:
2     def __init__(self):
3         self._heap = []
4
5     def _perc_up(self, i):
6         while (i - 1) // 2 >= 0:
7             parent_idx = (i - 1) // 2
8             if self._heap[i] < self._heap[parent_idx]:
9                 self._heap[i], self._heap[parent_idx] = (
10                     self._heap[parent_idx],
11                     self._heap[i],
12                 )
13             i = parent_idx
14
15     def insert(self, item):
16         self._heap.append(item)
17         self._perc_up(len(self._heap) - 1)
18
19     def _perc_down(self, i):
20         while 2 * i + 1 < len(self._heap):
21             sm_child = self._get_min_child(i)
22             if self._heap[i] > self._heap[sm_child]:
23                 self._heap[i], self._heap[sm_child] = (
24                     self._heap[sm_child],
25                     self._heap[i],
26                 )
27             else:
28                 break
29             i = sm_child
30
31     def _get_min_child(self, i):
32         if 2 * i + 2 > len(self._heap) - 1:
33             return 2 * i + 1
34         if self._heap[2 * i + 1] < self._heap[2 * i + 2]:
35             return 2 * i + 1
36         return 2 * i + 2
37
38     def delete(self):
39         self._heap[0], self._heap[-1] = self._heap[-1], self._heap[0]
40         result = self._heap.pop()
41         self._perc_down(0)
42         return result
43
44     def heapify(self, not_a_heap):
45         self._heap = not_a_heap[:]
46         i = len(self._heap) // 2 - 1 # 超过中点的节点都是叶子节点
47         while i >= 0:
48             #print(f'i = {i}, {self._heap}')
49             self._perc_down(i)
  
```

```

50         i = i - 1
51
52
53
54 n = int(input().strip())
55 heap = list(map(int, input().strip().split())) # [9, 5, 6, 2, 3]
56 heap = [-x for x in heap]
57
58 bh = BinaryHeap()
59 bh.heapify(heap)
60 ans = [-x for x in bh._heap]
61 print(*ans)

```

E18164: 剪绳子

greedy, huffman, <http://cs101.openjudge.cn/pctbook/M18164/>

小张要将一根长度为L的绳子剪成N段。准备剪的绳子的长度为L1,L2,L3...,LN，未剪的绳子长度恰好为剪后所有绳子长度的和。

每次剪断绳子时，需要的开销是此段绳子的长度。

比如，长度为10的绳子要剪成长度为2,3,5的三段绳子。长度为10的绳子切成5和5的两段绳子时，开销为10。再将5切成长度为2和3的绳子，开销为5。因此总开销为15。

请按照目标要求将绳子剪完最小的开销时多少。

已知， $1 \leq N \leq 20000$ ， $0 \leq L_i \leq 50000$

输入

第一行：N，将绳子剪成的段数。

第二行：准备剪成的各段绳子的长度。

输出

最小开销

样例输入

```

1 | 3
2 | 2 3 5

```

样例输出

```

1 | 15

```

来源：cs101-2017 期末机考备选

与 05333: Fence Repair 一样。 <http://cs101.openjudge.cn/practice/05333>

思路：剪绳子，实际上是 Huffman编码， <https://zhuanlan.zhihu.com/p/42238580>

```

1 # OJ18164
2 import sys

```

```

3  try: fin = open('test.in','r').readline
4  except: fin = sys.stdin.readline
5
6  n = int(fin())
7  import heapq
8  a = list(map(int, fin().split()))
9  heapq.heapify(a)
10 ans = 0
11 for i in range(n-1):
12     x = heapq.heappop(a)
13     y = heapq.heappop(a)
14     z = x + y
15     heapq.heappush(a, z)
16     ans += z
17 print(ans)

```

5.3 笔试简答@20210620

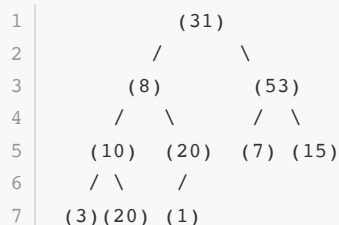
Q: 下图是一棵完全二叉树:

1)请根据初始建堆算法对该完全二叉树建堆, 请画出构建的小根堆(2分);

2)基于(1)中得到的堆, 删除其中的最小元素, 请用图给出堆的调整过程(2分);

3)基于(1)中得到的堆, 向其中插入元素2, 请给出堆的调整过程(2分)。

注:每移动一个元素视为一个执行步骤, 画出所有执行步骤。



说明: 根据二叉堆的“完全二叉树”结构性质建堆。从完全二叉树的最后一个非叶子节点开始, 向上遍历每个节点。对于每个节点, 进行下沉操作, 将节点与其子节点进行比较, 并交换位置直到满足小根堆的性质。重复步骤, 直到根节点。

1) 初始建堆算法的步骤如下:

- 从完全二叉树的最后一个非叶子节点开始, 向上遍历每个节点。
- 对于每个节点, 进行下沉操作, 将节点与其子节点进行比较, 并交换位置直到满足小根堆的性质。
- 重复上述步骤, 直到根节点。

最后一个非叶子节点是(20)。(请同学验证是否正确?)


```

1      (31)
2      /  \
3     (8)   (53)
4    /  \  /  \
5   (10) (1) (7) (15)
6   /  \  /
7  (3) (20) (20)

```

```

1      (31)
2      /  \
3     (8)   (53)
4    /  \  /  \
5   (3)   (1) (7) (15)
6   /  \  /
7  (10) (20) (20)

```

```

1      (31)
2      /  \
3     (8)   (7)
4    /  \  /  \
5   (3)   (1) (53) (15)
6   /  \  /
7  (10) (20) (20)

```

```

1      (31)
2      /  \
3     (1)   (7)
4    /  \  /  \
5   (3)   (8) (53) (15)
6   /  \  /
7  (10) (20) (20)

```

```

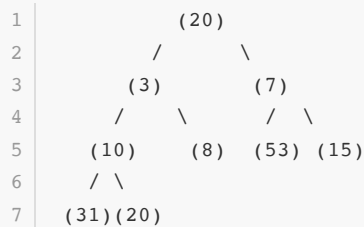
1      (1)
2      /  \
3     (3)   (7)
4    /  \  /  \
5   (10)  (8) (53) (15)
6   /  \  /
7  (31) (20) (20)

```

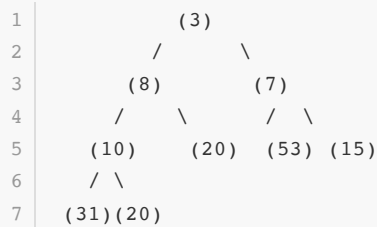
2) 删除最小元素的堆调整过程：

- 首先，将根节点与最后一个叶子节点交换位置。
- 然后，将根节点进行下沉操作，与其子节点进行比较并交换位置，直到满足小根堆的性质。

删除最小元素后，将最后一个元素20 移到根节点，然后进行堆的调整。



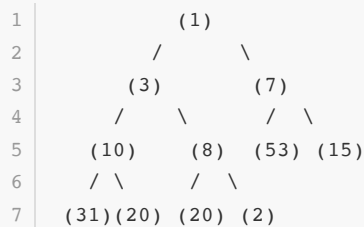
继续调整堆：



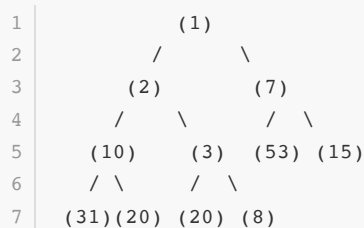
3) 插入元素的堆调整过程：

- 首先，将新元素插入到完全二叉树的最后一个位置。
- 然后，将新元素进行上浮操作，与其父节点进行比较并交换位置，直到满足小根堆的性质。

首先将元素2插入到堆的末尾，然后向上调整堆。



继续调整堆：



6 堆排序算法

堆排序：利用最大堆的性质，将列表从小到大排序。

```
1 class Node:
2     def __init__(self, key):
3         self.key = key
4
5 # 确保以索引 i 为根节点的子树满足最大堆的性质（父节点的值大于等于子节点的值）。
6 def adjust_heap(lst, i, size):
7     lchild = 2 * i + 1
8     rchild = 2 * i + 2
9     max_idx = i
10    if i < size // 2:
11        if lchild < size and lst[lchild].key > lst[max_idx].key:
12            max_idx = lchild
13        if rchild < size and lst[rchild].key > lst[max_idx].key:
14            max_idx = rchild
15        if max_idx != i:
16            lst[max_idx], lst[i] = lst[i], lst[max_idx]
17            adjust_heap(lst, max_idx, size)
18
19
20 def build_heap(lst):
21     size = len(lst)
22     for i in range(0, (size // 2))[:-1]: # 从最后一个非叶子节点开始，逐个向上调整，使整个列表成为一个最大
        堆。
23         adjust_heap(lst, i, size)
24
25
26 def heap_sort(lst):
27     size = len(lst)
28     build_heap(lst)
29     for i in range(0, size)[:-1]:
30         lst[0], lst[i] = lst[i], lst[0]
31         adjust_heap(lst, 0, i)
32
33
34 def print_list(lst):
35     return [node.key for node in lst]
36
37
38 if __name__ == "__main__":
39     lst = [Node(5), Node(9), Node(3), Node(8), Node(7)]
40     # 打印排序前的列表
41     print("Before sorting:", print_list(lst))
42
43     # 调用堆排序
44     heap_sort(lst)
45
46     # 打印排序后的列表
47     print("After sorting:", print_list(lst))
48
49 """
50 Before sorting: [5, 9, 3, 8, 7]
51 After sorting: [3, 5, 7, 8, 9]
```

总结

1. 时间复杂度：

- 构建堆的时间复杂度为 $O(n)$ 。
- 每次调整堆的时间复杂度为 $O(\log n)$ ，总共需要 n 次调整。
- 因此，堆排序的总体时间复杂度为 $O(n \log n)$ 。

2. 空间复杂度：

- 堆排序是原地排序算法，空间复杂度为 $O(1)$ 。

3. 稳定性：

- 堆排序是一种**不稳定排序**，因为在交换元素时可能会改变相同值元素的相对顺序。

4. 适用场景：

- 适用于需要高效排序且内存有限的场景，例如嵌入式系统或大规模数据排序。

7 二叉搜索树

二叉搜索树（Binary Search Tree, BST），它是映射的另一种实现。我们感兴趣的不是元素在树中的确切位置，而是**如何利用二叉树结构提供高效的搜索。**

二叉搜索树依赖于这样一个性质：**小于父节点的键都在左子树中，大于父节点的键则都在右子树中。**我们称这个性质为**二叉搜索性。**

7.1 编程题目

M01577: Falling Leaves (assign#7)

tree, <http://cs101.openjudge.cn/practice/01577/>

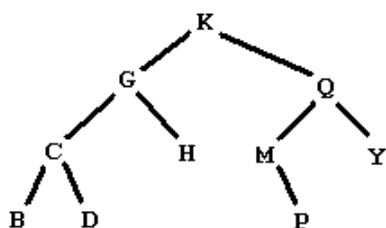


Figure 1

Figure 1 shows a graphical representation of a binary tree of letters. People familiar with binary trees can skip over the definitions of a binary tree of letters, leaves of a binary tree, and a binary search tree of letters, and go right to The problem.

A binary tree of letters may be one of two things:

1. It may be empty.
2. It may have a root node. A node has a letter as data and refers to a left and a right subtree. The left and right subtrees are also binary trees of letters.

In the graphical representation of a binary tree of letters:

1. Empty trees are omitted completely.
2. Each node is indicated by
 - Its letter data,
 - A line segment down to the left to the left subtree, if the left subtree is nonempty,
 - A line segment down to the right to the right subtree, if the right subtree is nonempty.

A leaf in a binary tree is a node whose subtrees are both empty. In the example in Figure 1, this would be the five nodes with data B, D, H, P, and Y.

The preorder traversal of a tree of letters satisfies the defining properties:

3. If the tree is empty, then the preorder traversal is empty.
4. If the tree is not empty, then the preorder traversal consists of the following, in order
 - The data from the root node,
 - The preorder traversal of the root's left subtree,
 - The preorder traversal of the root's right subtree.

The preorder traversal of the tree in Figure 1 is KGCBDHQMPY.

```
1  A tree like the one in Figure 1 is also a binary search tree of letters. A binary search
   tree of letters is a binary tree of letters in which each node satisfies:
2
3  The root's data comes later in the alphabet than all the data in the nodes in the left
   subtree.
4
5  The root's data comes earlier in the alphabet than all the data in the nodes in the right
   subtree.
6
7  The problem:
8
9  Consider the following sequence of operations on a binary search tree of letters
10
11 Remove the leaves and list the data removed
12 Repeat this procedure until the tree is empty
13 Starting from the tree below on the left, we produce the sequence of trees shown, and then
   the empty tree
14
15 ![img](https://raw.githubusercontent.com/GMyhf/img/main/img/1577_2.jpg)
16
17 by removing the leaves with data
18
19 BDHPY
20 CM
21 GQ
22 K
23
24 Your problem is to start with such a sequence of lines of leaves from a binary search tree
   of letters and output the preorder traversal of the tree.
25
26 **输入**
27
28 The input will contain one or more data sets. Each data set is a sequence of one or more
   lines of capital letters.
```

```

29
30 The lines contain the leaves removed from a binary search tree in the stages described
    above. The letters on a line will be listed in increasing alphabetical order. Data sets are
    separated by a line containing only an asterisk (*).
31
32 The last data set is followed by a line containing only a dollar sign ($). There are no
    blanks or empty lines in the input.
33
34 **输出**
35
36 For each input data set, there is a unique binary search tree that would produce the
    sequence of leaves. The output is a line containing only the preorder traversal of that tree,
    with no blanks.
37
38 样例输入

```

```

1 BDHPY
2 CM
3 GQ
4 K
5 *
6 AC
7 B
8 $

```

样例输出

```

1 KGCBDHQMPY
2 BAC

```

来源

Mid-Central USA 2000

```

1 class TreeNode:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8 def build_bst(leaves):
9     if not leaves:
10        return None
11
12    root = TreeNode(leaves[0])
13    for leaf in leaves[1:]:
14        insert_node(root, leaf)
15
16    return root
17
18
19 def insert_node(root, leaf):
20     if leaf < root.data:
21         if root.left is None:

```

```

22         root.left = TreeNode(leaf)
23     else:
24         insert_node(root.left, leaf)
25     else:
26         if root.right is None:
27             root.right = TreeNode(leaf)
28         else:
29             insert_node(root.right, leaf)
30
31
32 def preorder_traversal(root):
33     if root is None:
34         return []
35     traversal = [root.data]
36     traversal.extend(preorder_traversal(root.left))
37     traversal.extend(preorder_traversal(root.right))
38     return traversal
39
40
41 # 读取输入数据
42 flag = 0
43 while True:
44     leaves = []
45     while True:
46         line = input().strip()
47         if line == '*':
48             break
49         elif line == '$':
50             flag = 1
51             break
52         else:
53             leaves.extend(line)
54
55     # 构建二叉搜索树
56     root = build_bst(leaves[::-1])
57
58     # 输出前序遍历结果
59     traversal_result = preorder_traversal(root)
60     print(''.join(traversal_result))
61
62     if flag:
63         break

```

练习M22275: 二叉搜索树的遍历

<http://cs101.openjudge.cn/practice/22275/>

给出一棵二叉搜索树的前序遍历，求它的后序遍历

输入

第一行一个正整数 n ($n \leq 2000$) 表示这棵二叉搜索树的结点个数

第二行 n 个正整数，表示这棵二叉搜索树的前序遍历

保证第二行的 n 个正整数中， $1 \sim n$ 的每个值刚好出现一次

输出

一行n个正整数，表示这棵二叉搜索树的后序遍历

样例输入

```
1 | 5
2 | 4 2 1 3 5
```

样例输出

```
1 | 1 3 2 5 4
```

提示

树的形状为

```
4
/\
2 5
/\
1 3
```

```
1 # 管骏杰 生命科学院
2 # 中序遍历就是顺序排列，进而通过上次作业的思路根据前序中序推出后序
3 class Node:
4     def __init__(self, val):
5         self.val = val
6         self.left = None
7         self.right = None
8
9
10 def build(preorder, inorder):
11     if not preorder or not inorder:
12         return None
13     root_val = preorder[0]
14     root = Node(root_val)
15     root_index = inorder.index(root_val)
16     root.left = build(preorder[1:root_index + 1], inorder[:root_index])
17     root.right = build(preorder[root_index + 1:], inorder[root_index + 1:])
18     return root
19
20
21 def postorder(root):
22     if not root:
23         return []
24     if root.left is None and root.right is None:
25         return [root.val]
26     result = []
27     result += postorder(root.left)
28     result += postorder(root.right)
29     result += [root.val]
30     return result
31
32
33 input()
```



```

34 preorder = list(map(int, input().split()))
35 inorder = sorted(preorder)
36 root = build(preorder, inorder)
37 result = postorder(root)
38 print(' '.join(map(str, result)))

```

```

1 """
2 王昊 光华管理学院。思路：
3 建树思路：数组第一个元素是根节点，紧接着是小于根节点值的节点，在根节点左侧，直至遇到大于根节点值的节点，
4 后续节点都在根节点右侧，按照这个思路递归即可
5 """
6 class Node():
7     def __init__(self, val):
8         self.val = val
9         self.left = None
10        self.right = None
11
12
13 def buildTree(preorder):
14     if len(preorder) == 0:
15         return None
16
17     node = Node(preorder[0])
18
19     idx = len(preorder)
20     for i in range(1, len(preorder)):
21         if preorder[i] > preorder[0]:
22             idx = i
23             break
24
25     node.left = buildTree(preorder[1:idx])
26     node.right = buildTree(preorder[idx:])
27
28     return node
29
30 def postorder(node):
31     if node is None:
32         return []
33
34     output = []
35     output.extend(postorder(node.left))
36     output.extend(postorder(node.right))
37     output.append(str(node.val))
38
39     return output
40
41 n = int(input())
42 preorder = list(map(int, input().split()))
43 print(' '.join(postorder(buildTree(preorder))))

```

```

1 def post_order(pre_order):
2     if not pre_order:
3         return []
4     root = pre_order[0]
5     left_subtree = [x for x in pre_order if x < root]
6     right_subtree = [x for x in pre_order if x > root]
7     return post_order(left_subtree) + post_order(right_subtree) + [root]
8
9 n = int(input())
10 pre_order = list(map(int, input().split()))
11 print(' '.join(map(str, post_order(pre_order))))

```

练习M05455: 二叉搜索树的层次遍历

<http://cs101.openjudge.cn/practice/05455/>

二叉搜索树在动态查表中有特别的用处，一个无序序列可以通过构造一棵二叉搜索树变成一个有序序列，构造树的过程即为对无序序列进行排序的过程。每次插入的新的结点都是二叉搜索树上新的叶子结点，在进行插入操作时，不必移动其它结点，只需改动某个结点的指针，由空变为非空即可。

这里，我们想探究二叉树的建立和层次输出。

输入

只有一行，包含若干个数字，中间用空格隔开。（数字可能会有重复，对于重复的数字，只计入一个）

输出

输出一行，对输入数字建立二叉搜索树后进行按层次周游的结果。

样例输入

```

1 51 45 59 86 45 4 15 76 60 20 61 77 62 30 2 37 13 82 19 74 2 79 79 97 33 90 11 7 29 14 50 1 96
   59 91 39 34 6 72 7

```

样例输出

```

1 51 45 59 4 50 86 2 15 76 97 1 13 20 60 77 90 11 14 19 30 61 82 96 7 29 37 62 79 91 6 33 39 74
   34 72

```

提示

输入输出的最后都不带空格和回车换行

按输入顺序遍历数字，用一个 `seen` 集合来跳过重复值，对于每个新值，插入 BST。

```

1 from collections import deque
2 import sys
3
4 # 定义二叉树结点
5 class TreeNode:
6     def __init__(self, val):
7         self.val = val
8         self.left = None

```

```

9         self.right = None
10
11     # 插入结点到 BST 中
12     def insert(root, val):
13         if not root:
14             return TreeNode(val)
15         if val < root.val:
16             root.left = insert(root.left, val)
17         elif val > root.val:
18             root.right = insert(root.right, val)
19         return root
20
21     # 层次遍历
22     def level_order(root):
23         if not root:
24             return []
25         result = []
26         queue = deque([root])
27         while queue:
28             node = queue.popleft()
29             result.append(str(node.val))
30             if node.left:
31                 queue.append(node.left)
32             if node.right:
33                 queue.append(node.right)
34         return result
35
36     def main():
37         data = sys.stdin.read().strip().split()
38         seen = set()
39         root = None
40         for tok in data:
41             num = int(tok)
42             if num in seen:
43                 continue
44             seen.add(num)
45             root = insert(root, num)
46         # 输出层次遍历，最后不带多余空格或换行
47         out = ' '.join(level_order(root))
48         sys.stdout.write(out)
49
50     if __name__ == "__main__":
51         main()
52

```

问题是要求根据一串数字构建一棵二叉搜索树（BST），然后对这棵二叉搜索树进行层序遍历（也称为广度优先搜索）。

以下是分步计划：

1. 创建一个 `TreeNode` 类来表示树中的每个节点。
2. 创建一个 `insert` 函数，该函数接收一个节点和一个值作为输入，并将该值插入到以该节点为根的二叉搜索树中。
3. 创建一个 `level_order_traversal` 函数，该函数接收树的根节点作为输入，并返回树的层序遍历结果。
 - o 使用一个队列来存储待访问的节点。
 - o 当队列不为空时，从队列中取出一个节点，访问它，并将其子节点加入队列。
4. 读取输入的数字序列，构建二叉搜索树，执行层序遍历，并输出结果。

以下是实现此计划的 Python 代码：

```
1 class TreeNode:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6
7 def insert(node, value):
8     if node is None:
9         return TreeNode(value)
10    if value < node.value:
11        node.left = insert(node.left, value)
12    elif value > node.value:
13        node.right = insert(node.right, value)
14    return node
15
16 def level_order_traversal(root):
17     queue = [root]
18     traversal = []
19     while queue:
20         node = queue.pop(0)
21         traversal.append(node.value)
22         if node.left:
23             queue.append(node.left)
24         if node.right:
25             queue.append(node.right)
26     return traversal
27
28 numbers = list(map(int, input().strip().split()))
29 numbers = list(dict.fromkeys(numbers)) # remove duplicates
30 root = None
31 for number in numbers:
32     root = insert(root, number)
33 traversal = level_order_traversal(root)
34 print(' '.join(map(str, traversal)))
```

This code reads the sequence of numbers from the input, removes duplicates, constructs the BST, performs the level order traversal, and prints the result.

7.2 二叉搜索树实现快排

快速排序是一种基于分治法的排序算法，它通过选择一个元素作为基准（pivot），将数组分割为两个子数组，其中一个子数组的元素都小于基准，另一个子数组的元素都大于基准。然后，对两个子数组递归地应用相同的排序过程，直到排序完成。

二叉搜索树是一种有序的二叉树，它满足以下性质：

- 左子树中的所有节点的值都小于根节点的值。
- 右子树中的所有节点的值都大于根节点的值。
- 左子树和右子树也分别是二叉搜索树。

可以使用二叉搜索树来实现快速排序的过程。具体步骤如下：

1. 选择数组中的一个元素作为基准。
2. 创建一个空的二叉搜索树。

3. 将数组中的其他元素逐个插入二叉搜索树中。
4. 按照二叉搜索树的中序遍历（左子树、根节点、右子树）得到排序后的结果。

这种方法的时间复杂度为 $O(n \log n)$ ，其中 n 是数组的长度。每次插入操作都需要 $O(\log n)$ 的时间复杂度，总共进行 $n-1$ 次插入操作。

需要注意的是，**二叉搜索树的性能取决于树的平衡性**。如果二叉搜索树变得不平衡，性能可能会下降到 $O(n^2)$ 的时间复杂度。因此，在实际应用中，为了确保性能，通常会使用平衡二叉搜索树（如红黑树、AVL树）来实现快速排序。

```
1 class TreeNode:
2     def __init__(self, val):
3         self.val = val
4         self.left = None
5         self.right = None
6
7 def insert(root, val):
8     if root is None:
9         return TreeNode(val)
10    if val < root.val:
11        root.left = insert(root.left, val)
12    else:
13        root.right = insert(root.right, val)
14    return root
15
16 def inorder_traversal(root, result):
17     if root:
18         inorder_traversal(root.left, result)
19         result.append(root.val)
20         inorder_traversal(root.right, result)
21
22 def quicksort(nums):
23     if not nums:
24         return []
25     root = TreeNode(nums[0])
26     for num in nums[1:]:
27         insert(root, num)
28     result = []
29     inorder_traversal(root, result)
30     return result
31
32 # 示例输入
33 nums = [5, 2, 7, 1, 8, 3]
34
35 # 使用二叉搜索树实现快速排序
36 sorted_nums = quicksort(nums)
37
38 # 打印排序结果
39 print(sorted_nums)
```

8 平衡二叉搜索树

当二叉搜索树不平衡时，get和put等操作的性能可能降到 $O(n)$ 。本节将介绍一种特殊的二叉搜索树，它能自动维持平衡。这种树叫作AVL树，以其发明者G. M. Adelson-Velskii和E. M. Landis的姓氏命名。

AVL 平衡树的全称是 Adelson-Velsky and Landis 平衡树。它是由两位前苏联的计算机科学家，即Георгий Максимович Адельсон-Вельский (Georgy Maximovich Adelson-Velsky) 和Евгений Михайлович Ландис (Evgenii Mikhailovich Landis) 于1962年提出的一种自平衡二叉搜索树。

这种树的名称取自这两位科学家的姓氏的首字母缩写。AVL 平衡树通过在每个节点上维护一个平衡因子 (balance factor) 来实现平衡。**平衡因子是指节点的左子树高度与右子树高度之差的绝对值。**通过不断调整树的结构，AVL 树能够保持树的平衡，使得在最坏情况下的查找、插入和删除操作的时间复杂度保持在 $O(\log n)$ 。

AVL 平衡树的特点是在每次插入或删除节点时，会通过旋转操作来调整树的结构，使得平衡因子在特定的范围内，通常是 -1、0、1。这样的平衡状态能够保证树的高度始终保持在较小的范围内，提供了较快的查找和更新操作。

总结起来，AVL 平衡树是一种自平衡二叉搜索树，通过调整树的结构来保持树的平衡性，以提供高效的查找和更新操作。

AVL树实现映射抽象数据类型的方式与普通的二叉搜索树一样，唯一的差别就是性能。实现AVL树时，要记录每个节点的平衡因子。我们通过查看每个节点左右子树的高度来实现这一点。更正式地说，将平衡因子定义为左右子树的高度之差。

$$balanceFactor = height(leftSubTree) - height(rightSubTree)$$

根据上述定义，如果平衡因子大于零，称之为左倾；如果平衡因子小于零，就是右倾；如果平衡因子等于零，那么树就是完全平衡的。为了实现AVL树并利用平衡树的优势，将平衡因子为-1、0和1的树都定义为**平衡树**。一旦某个节点的平衡因子超出这个范围，就需要通过一个过程让树恢复平衡。图1展示了一棵右倾树及其中每个节点的平衡因子。

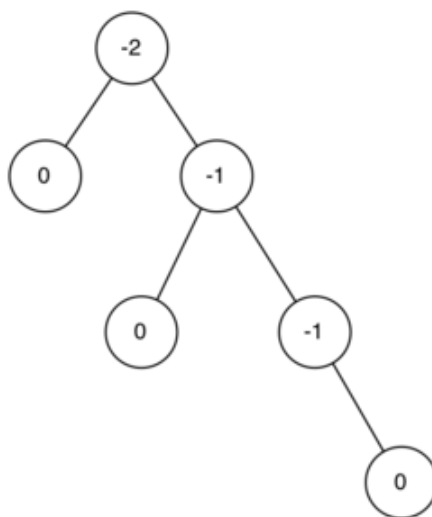


图1 带平衡因子的右倾树

④ Note

108.将有序数组转换为二叉搜索树 (assign#6)

<https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/>

给你一个整数数组 `nums`，其中元素已经按 **升序** 排列，请你将其转换为一棵 平衡二叉搜索树。

平衡二叉树是指该树所有节点的左右子树的高度相差不超过1。

8.1 AVL树的性能

先看看限定平衡因子带来的结果。保证树的平衡因子为-1、0或1，可以使关键操作获得更好的大 O 性能。首先考虑平衡因子如何改善最坏情况。有左倾与右倾这两种可能性。如果考虑高度为0、1、2和3的树，图2展示了应用新规则后最不平衡的左倾树。

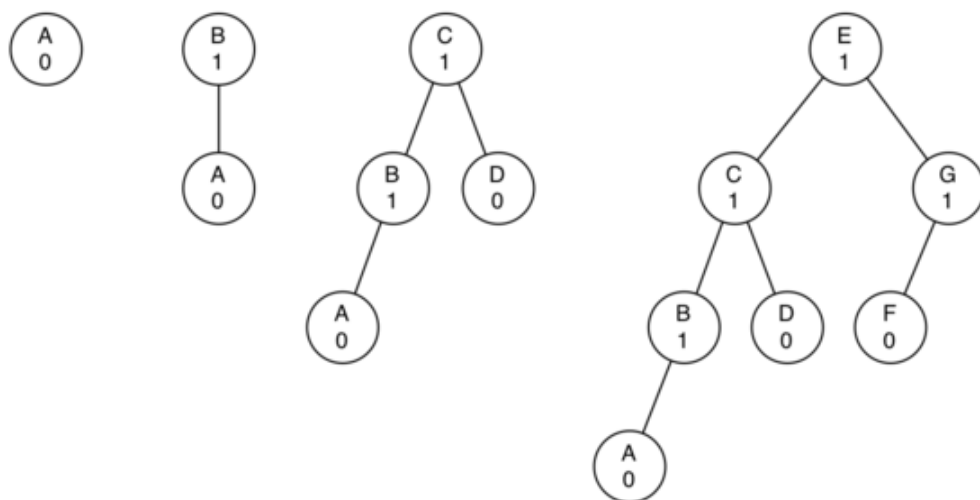


图2 左倾AVL树的最坏情况

查看树中的节点数之后可知，高度为0时有1个节点，高度为1时有2个节点（ $1 + 1 = 2$ ），高度为2时有4个节点（ $1 + 1 + 2 = 4$ ），高度为3时有7个节点（ $1 + 2 + 4 = 7$ ）。也就是说，当高度为 h 时，节点数 N_h 是：

$$N_h = 1 + N_{h-1} + N_{h-2}$$

你或许觉得这个公式很眼熟，因为它与斐波那契数列很相似。

8.1.1 编程题目

M27625: AVL树至少有几个结点

<http://cs101.openjudge.cn/practice/27625/>

输入 n ($0 < n < 50$), 输出一个 n 层的AVL树至少有多少个结点。

输入

n

输出

答案

样例输入

1 | 4

样例输出

1 | 7

来源：Guo Wei

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)
```

```

4 def avl_min_nodes(n):
5     if n == 0:
6         return 0
7     elif n == 1:
8         return 1
9     else:
10        return avl_min_nodes(n-1) + avl_min_nodes(n-2) + 1
11
12 n = int(input())
13 min_nodes = avl_min_nodes(n)
14 print(min_nodes)

```

```

1 def avl_min_nodes(n, memo):
2     if n == 0:
3         return 0
4     elif n == 1:
5         return 1
6     elif memo[n] != 0: # 如果已经计算过, 直接返回保存的结果
7         return memo[n]
8     else:
9         memo[n] = avl_min_nodes(n-1, memo) + avl_min_nodes(n-2, memo) + 1
10        return memo[n]
11
12 n = int(input())
13 memo = [0] * (n+1) # 创建一个数组来保存已计算的结果
14 min_nodes = avl_min_nodes(n, memo)
15 print(min_nodes)

```

M27626: AVL树最多有几层

<http://cs101.openjudge.cn/practice/27626/>

n个结点的AVL树最多有多少层?

输入

整数n。 $0 < n < 50,000,000$

输出

AVL树最多有多少层

样例输入

```
1 | 20
```

样例输出

```
1 | 6
```

来源: Guo Wei

AVL树是一种自平衡的二叉搜索树，其中每个节点的左右子树的高度最多相差1。为了确定具有 n 个节点的AVL树的最大高度，我们可以使用一个递归关系，该关系描述了给定高度的AVL树所能包含的最少节点数。

设 $N(h)$ 表示高度为 h 的AVL树的最少节点数，那么有如下递归关系：

$$1 \quad N(h) = N(h-1) + N(h-2) + 1$$

这里， $N(h-1)$ 是较高子树的最少节点数， $N(h-2)$ 是较矮子树的最少节点数， $+1$ 是根节点自身。

基本情况是：

$$\begin{array}{ll} 1 & N(1) = 1 \quad (\text{单个节点的树}) \\ 2 & N(0) = 0 \quad (\text{空树}) \end{array}$$

可以使用这个递归关系来计算任何高度的AVL树的最少节点数。然后，我们可以通过递增高度，直到计算出的节点数超过输入的 n ，来找出具有 n 个节点的AVL树的最大高度。

用于计算具有 n 个节点的AVL树的最大高度：

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)
4 def min_nodes(h):
5     if h == 0: return 0
6     if h == 1: return 1
7     return min_nodes(h-1) + min_nodes(h-2) + 1
8
9 def max_height(n):
10     h = 0
11     while min_nodes(h) <= n:
12         h += 1
13     return h - 1
14
15 n = int(input())
16 print(max_height(n))
```

因为与斐波那契数列很相似，可以根据它推导出由 AVL 树的节点数计算高度的公式。在斐波那契数列中，第 i 个数是：

$$F_0 = 0$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \text{ for all } i \geq 2$$

一个重要的事实是，随着斐波那契数列的增长， F_i/F_{i-1} 逐渐逼近黄金分割比例 Φ ， $\Phi = \frac{1+\sqrt{5}}{2}$ 。如果你好奇这个等式的推导过程，可以找一本数学书看看。我们在此直接使用这个等式，将 F_i 近似为 $F_i = \Phi^i / \sqrt{5}$ 。

```
1 def fibonacci_recursive(n):
2     if n <= 1:
3         return n
4     else:
5         return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)
6
7
8 def fibonacci_iterative(n):
9     if n <= 1:
```

```

10     return n
11 else:
12     a, b = 0, 1
13     for _ in range(2, n+1):
14         a, b = b, a + b
15     return b
16
17
18 phi = (1+5**0.5)/2
19
20 dp = [0]
21 print("The ith Fibonacci number, \t With golden ratio approximation")
22 for i in range(10):
23     result_recursive = fibonacci_recursive(i)
24     print(f"F{i}: {result_recursive}, ", end='')
25     print(f'{phi**i/(5**0.5)}')
26
27 """
28 The ith Fibonacci number,      With golden ratio approximation
29 F0: 0, 0.4472135954999579
30 F1: 1, 0.7236067977499789
31 F2: 1, 1.1708203932499368
32 F3: 2, 1.8944271909999157
33 F4: 3, 3.065247584249853
34 F5: 5, 4.959674775249769
35 F6: 8, 8.024922359499623
36 F7: 13, 12.984597134749393
37 F8: 21, 21.009519494249016
38 F9: 34, 33.99411662899841
39 """

```

$$\begin{aligned}
 N_0 &= 1 \\
 N_1 &= 2 \quad F_3 = 3 \\
 N_2 &= 4 \quad F_4 = 5 \\
 N_3 &= 7 \quad F_5 = 8
 \end{aligned}$$

由此，可以将 N_h 的等式重写为：

$$N_h = F_{h+2} - 1, h \geq 1$$

用黄金分割近似替换，得到：

$$N_h = \frac{\Phi^{h+2}}{\sqrt{5}} - 1$$

移项，两边以2为底取对数，求h，得到：

$$\begin{aligned}
 \log(N_h + 1) &= (h + 2) \log \Phi - \frac{1}{2} \log 5 \\
 h &= \frac{\log(N_h + 1) - 2 \log \Phi + \frac{1}{2} \log 5}{\log \Phi} \\
 h &\approx 1.44 \log N_h
 \end{aligned}$$

在任何时间，AVL树的高度都等于节点数取对数再乘以一个常数（1.44）。对于搜索AVL树来说，这是一件好事，因为时间复杂度被限制为 $O(\log N)$ 。

8.2 AVL树的实现

已经证明，保持 AVL 树的平衡会带来很大的性能优势，现在看看如何往树中插入一个键。所有新键都是以叶子节点插入的，因为新叶子节点的平衡因子是零，所以新插节点没有什么限制条件。但插入新节点后，必须更新父节点的平衡因子。新的叶子节点对其父节点平衡因子的影响取决于它是左子节点还是右子节点。如果是右子节点，父节点的平衡因子减一。如果是左子节点，则父节点的平衡因子加一。

假设现在已有一棵平衡二叉树，那么可以预见到，在往其中插入一个结点时，一定会有结点的平衡因子发生变化，此时可能会有结点的平衡因子的绝对值大于 1（这些平衡因子只可能是 2 或者 -2），这样以该结点为根结点的子树就是失衡的，需要进行调整。显然，只有在从根结点到该插入结点的路径上的结点才可能发生平衡因子变化，因此只需对这条路径上失衡的结点进行调整。可以证明，只要把最靠近插入结点的失衡结点调整到正常，路径上的所有结点就都会平衡。

当平衡的二叉排序树因插入结点而失去平衡时，仅需对最小不平衡子树进行平衡旋转处理即可。因为经过旋转处理之后的子树深度和插入之前相同，因而不影响插入路径上所有祖先结点的平衡度。

如果需要进行再平衡，该怎么做呢？高效的再平衡是让 AVL 树发挥作用同时不损性能的关键。为了让 AVL 树恢复平衡，需要在树上进行一次或多次旋转。

要理解什么是旋转，来看一个简单的例子。考虑图3中左边的树。这棵树失衡了，平衡因子是-2。要让它恢复平衡，我们围绕以节点A为根结点的子树做一次左旋。

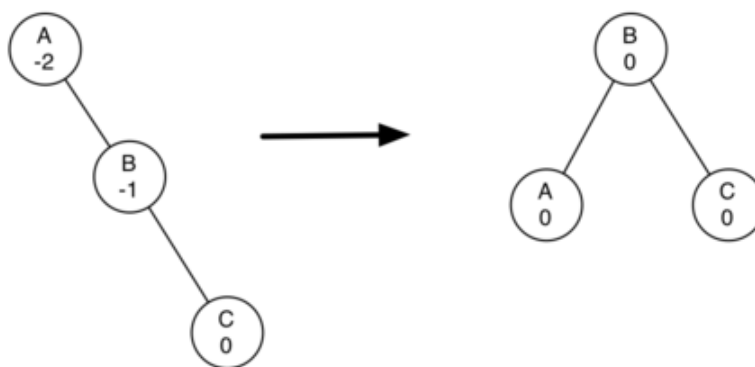


图3 通过左旋让失衡的树恢复平衡

本质上，左旋包括以下步骤。

- 将右子节点（节点B）提升为子树的根节点。
- 将旧根节点（节点A）作为新根节点的左子节点。
- 如果新根节点（节点B）已经有一个左子节点，将其作为新左子节点（节点A）的右子节点。注意，因为节点B之前是节点A的右子节点，所以此时节点A必然没有右子节点。因此，可以为它添加新的右子节点，而无须过多考虑。

来看一棵稍微复杂一点的树，并理解右旋过程。图4左边的是一棵左倾的树，根节点的平衡因子是2。右旋步骤如下。

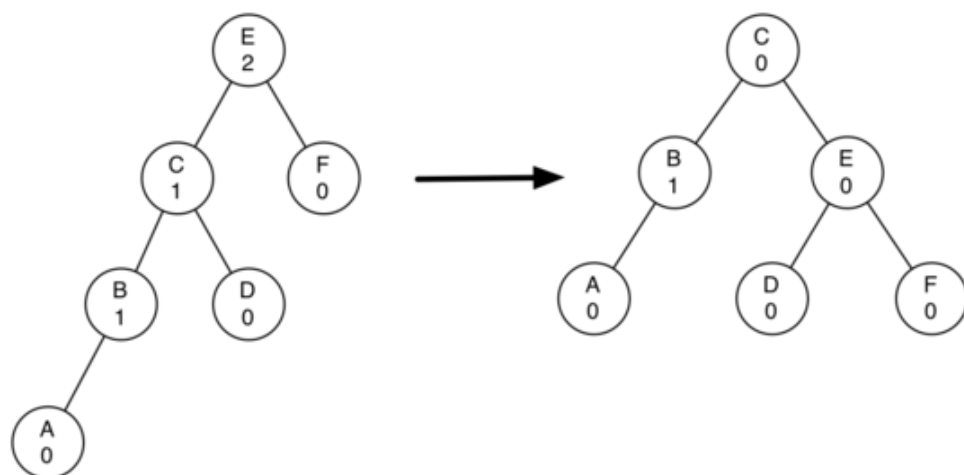


图4 通过右旋让失衡的树恢复平衡

- 将左子节点（节点C）提升为子树的根节点。
- 将旧根节点（节点E）作为新根节点的右子节点。
- 如果新根节点（节点C）已经有一个右子节点（节点D），将其作为新右子节点（节点E）的左子节点。注意，**因为节点C之前是节点E的左子节点，所以此时节点E必然没有左子节点。**因此，可以为它添加新的左子节点，而无须过多考虑。

假设最靠近插入结点的失衡结点是 A，显然它的平衡因子只可能是 2 或者 -2。很容易发现这两种情况完全对称，因此主要讨论结点 A 的平衡因子是 2 的情形。

由于结点 A 的平衡因子是 2，因此左子树的高度比右子树大 2，于是以结点 A 为根结点的子树一定是图4的两种形态 LL 型与 LR 型之一（注意：LL 和 LR 只表示树型，不是左右旋的意思），其中☆、★、◇、◆是图中相应结点的 AVL 子树，结点 A、B、C 的权值满足 $A > B > C$ 。可以发现，当结点 A 的左孩子的平衡因子是 1 时为 LL 型，是 -1 时为 LR 型。那么，为什么结点 A 的左孩子的平衡因子只可能是 1 或者 -1，而不可能是 0 呢？这是因为这种情况无法由平衡二叉树插入一个结点得到。（不信举个反例？）

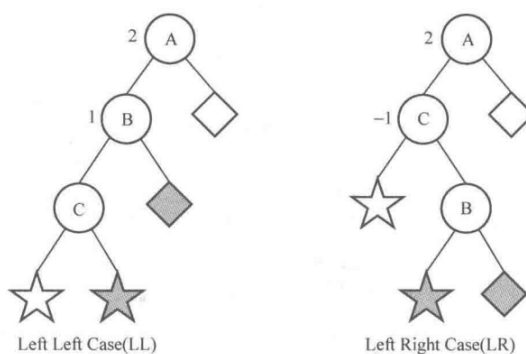


图4 树型之 LL 型与 LR 型（数字代表平衡因子）

补充说明，除了☆、★、◇、◆均为空树的情况以外，其他任何情况均满足在插入前底层两棵子树的高度比另外两棵子树的高度小 1，且插入操作一定发生在底层两棵子树上。例如对 LL 型来说，插入前子树的高度满足 $\star = \star = \diamond - 1 = \diamond - 1$ ，而在☆或★中插入一个结点后导致☆或★的高度加 1，使得结点 A 不平衡。（辅助理解，不需要记住）现在考虑怎样调整这两种树型，才能使树平衡。

先考虑 LL 型，可以把以 C 为根结点的子树看作一个整体，然后以结点 A 作为 root 进行右旋，便可以达到平衡，如图5 所示。

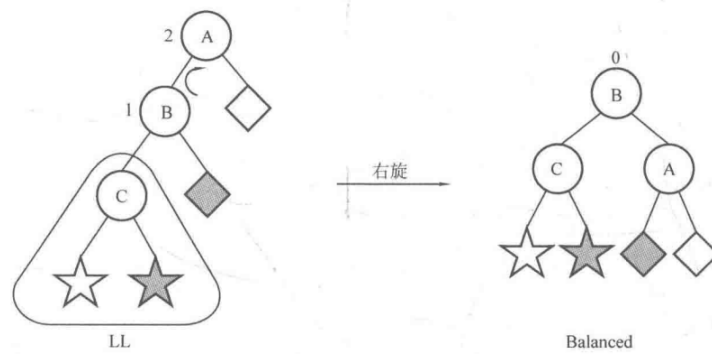


图5 LL 型调整示意图（数字代表平衡因子）

然后考虑 LR 型，可以先忽略结点 A，以结点 C 为 root 进行左旋，就可以把情况转化为 LL 型，然后按上面 LL 型的做法进行一次右旋即可，如图6 所示。

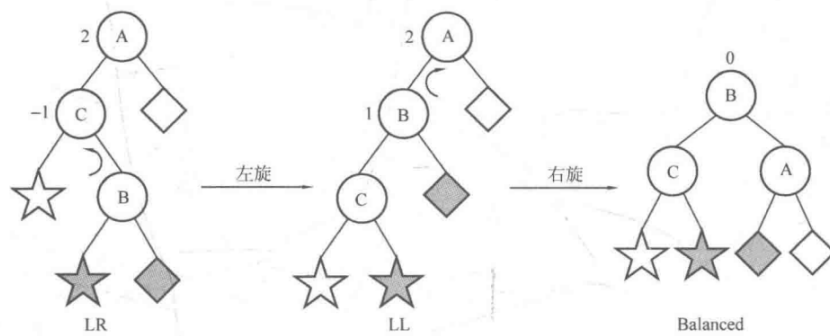


图6 LR型调整示意图（数字代表平衡因子）

至此,结点 A 的平衡因子是 2 的情况已经讨论清楚,下面简要说明平衡因子是 -2 的情况，显然两种情况是完全对称的。

由于结点 A 的平衡因子为 -2，因此右子树的高度比左子树大 2，于是以结点A为根结点的子树一定是图7 的两种形态 RR 型与 RL 型之一。注意，由于和上面讨论的 LL 型和 LR 型对称，此处结点 A、B、C 的权值满足 $A < B < C$ 。可以发现，当结点 A 的右孩子的平衡因子是 -1 时为 RR 型，是1时为 RL 型。

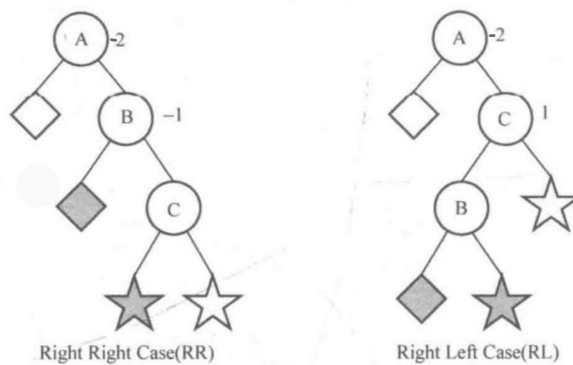


图7 树型之 RR型与RL型（数字代表平衡因子）

对 RR 型来说，可以把以 C 为根结点的子树看作一个整体，然后以结点 A 作为 root 进行左旋，便可以达到平衡，如图8 所示。

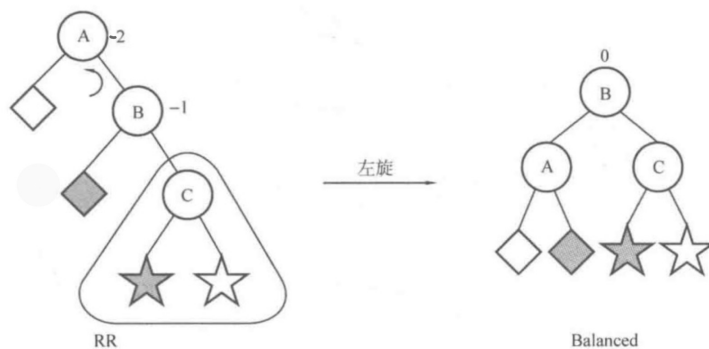


图8 RR 型调整示意图（数字代表平衡因子）

对 RL 型来说，可以先忽略结点 A，以结点 C 为 root 进行右旋，就可以把情况转化为 RR 然后按上面 RR 型的做法进行一次左旋即可，如图9 所示。

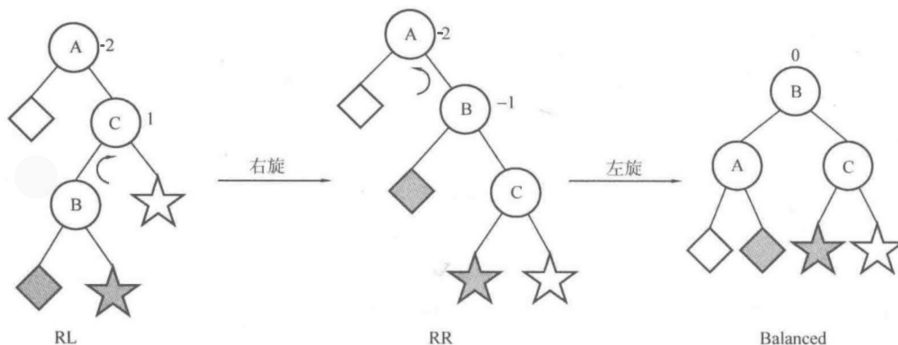


图9 RL型调整示意图（数字代表平衡因子）

至此，对LL 型、LR 型、RR 型、RL型的调整方法都已经讨论清楚。

通过维持树的平衡，可以保证get方法的时间复杂度为 $O(\log_2(n))$ 。但会给put操作的性能带来多大影响呢？我们来看看put操作。因为新节点作为叶子节点插入，所以更新所有父节点的平衡因子最多需要 $\log_2(n)$ 次操作——每一层一次。如果树失衡了，恢复平衡最多需要旋转两次。每次旋转的时间复杂度是 $O(1)$ ，所以put操作的时间复杂度仍然是 $O(\log_2(n))$ 。

至此，我们已经实现了一棵可用的AVL树。了解旋转的基本原理之后，来看编程题目。

8.2.1 编程题目

M晴问9.5: 平衡二叉树的建立

<https://sunnywhy.com/sfbj/9/5/359>

将 n 个互不相同的正整数先后插入到一棵空的AVL树中，求最后生成的AVL树的先序序列。

输入

第一行一个整数 $n(1 \leq n \leq 50)$ ，表示AVL树的结点个数；

第二行 n 个整数 $a_i (1 \leq a_i \leq 100)$ ，表示插入序列。

输出

输出 n 个整数，表示先序遍历序列，中间用空格隔开，行末不允许有多余的空格。

样例1

输入

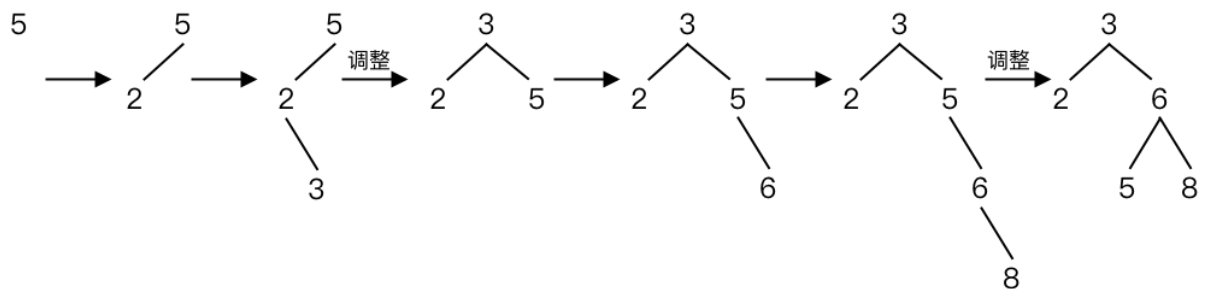
```
1 | 5
2 | 5 2 3 6 8
```

输出

```
1 | 3 2 6 5 8
```

解释

插入的过程如下图所示。



To solve this problem, you can follow these steps:

1. Read the input sequence.
2. Insert the values into an AVL tree. An AVL tree is a self-balancing binary search tree, and the heights of the two child subtrees of any node differ by at most one.
3. Perform a preorder traversal of the AVL tree and print the result.

Here is the Python code that implements this plan:

```
1 class Node:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6         self.height = 1
7
8 class AVL:
9     def __init__(self):
10        self.root = None
11
12    def insert(self, value):
13        if not self.root:
14            self.root = Node(value)
15        else:
16            self.root = self._insert(value, self.root)
17
```

```

18 def _insert(self, value, node):
19     if not node:
20         return Node(value)
21     elif value < node.value:
22         node.left = self._insert(value, node.left)
23     else:
24         node.right = self._insert(value, node.right)
25
26     node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))
27
28     balance = self._get_balance(node)
29
30     if balance > 1:
31         if value < node.left.value: # 树形是 LL
32             return self._rotate_right(node)
33         else: # 树形是 LR
34             node.left = self._rotate_left(node.left)
35             return self._rotate_right(node)
36
37     if balance < -1:
38         if value > node.right.value: # 树形是 RR
39             return self._rotate_left(node)
40         else: # 树形是 RL
41             node.right = self._rotate_right(node.right)
42             return self._rotate_left(node)
43
44     return node
45
46 def _get_height(self, node):
47     if not node:
48         return 0
49     return node.height
50
51 def _get_balance(self, node):
52     if not node:
53         return 0
54     return self._get_height(node.left) - self._get_height(node.right)
55
56 def _rotate_left(self, z):
57     y = z.right
58     T2 = y.left
59     y.left = z
60     z.right = T2
61     z.height = 1 + max(self._get_height(z.left), self._get_height(z.right))
62     y.height = 1 + max(self._get_height(y.left), self._get_height(y.right))
63     return y
64
65 def _rotate_right(self, y):
66     x = y.left
67     T2 = x.right
68     x.right = y
69     y.left = T2
70     y.height = 1 + max(self._get_height(y.left), self._get_height(y.right))
71     x.height = 1 + max(self._get_height(x.left), self._get_height(x.right))
72     return x
73
74 def preorder(self):
75     return self._preorder(self.root)

```



```

76
77     def _preorder(self, node):
78         if not node:
79             return []
80         return [node.value] + self._preorder(node.left) + self._preorder(node.right)
81
82 n = int(input().strip())
83 sequence = list(map(int, input().strip().split()))
84
85 avl = AVL()
86 for value in sequence:
87     avl.insert(value)
88
89 print(' '.join(map(str, avl.preorder())))

```

This code reads the sequence from the input, inserts its values into an AVL tree, performs a preorder traversal of the AVL tree, and then prints the result.

8.2.2 AVL树中删除节点

要实现从AVL树中删除节点，需要添加一个删除方法，并确保在删除节点后重新平衡树。

下面是AVL的删除方法 `_delete`：

```

1  class AVL:
2      # Existing code...
3
4      def delete(self, value):
5          self.root = self._delete(value, self.root)
6
7      def _delete(self, value, node):
8          if not node:
9              return node
10
11         if value < node.value:
12             node.left = self._delete(value, node.left)
13         elif value > node.value:
14             node.right = self._delete(value, node.right)
15         else:
16             if not node.left:
17                 temp = node.right
18                 node = None
19                 return temp
20             elif not node.right:
21                 temp = node.left
22                 node = None
23                 return temp
24
25             temp = self._min_value_node(node.right)
26             node.value = temp.value
27             node.right = self._delete(temp.value, node.right)
28
29             # if not node:
30             #     return node
31
32         node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))

```

```

33
34     balance = self._get_balance(node)
35
36     # Rebalance the tree
37     if balance > 1:
38         if self._get_balance(node.left) >= 0:
39             return self._rotate_right(node)
40         else:
41             node.left = self._rotate_left(node.left)
42             return self._rotate_right(node)
43
44     if balance < -1:
45         if self._get_balance(node.right) <= 0:
46             return self._rotate_left(node)
47         else:
48             node.right = self._rotate_right(node.right)
49             return self._rotate_left(node)
50
51     return node
52
53     def _min_value_node(self, node):
54         current = node
55         while current.left:
56             current = current.left
57         return current
58
59     # Existing code...

```

这段代码中的 `_delete` 方法用于删除节点。它首先检查树中是否存在要删除的节点，然后根据节点的左右子树情况执行相应的操作，以保持AVL树的平衡。

在 AVL 树中，删除节点时，当被删除的节点有两个子节点时，需要一些额外的步骤来保持树的平衡性。让我们详细讲解 `else` 分支中的情况：

```

1     else:
2         if not node.left:
3             temp = node.right
4             node = None
5             return temp
6         elif not node.right:
7             temp = node.left
8             node = None
9             return temp
10
11         temp = self._min_value_node(node.right)
12         node.value = temp.value
13         node.right = self._delete(temp.value, node.right)

```

1. 如果要删除的节点 `node` 没有左子节点，那么我们只需返回其右子节点。这是因为右子节点（如果存在）将占据 `node` 的位置，而不会影响树的平衡性。所以我们将 `node` 设置为 `None`，然后返回其右子节点即可。
2. 如果要删除的节点 `node` 没有右子节点，那么我们只需返回其左子节点。这与上述情况类似。
3. 如果要删除的节点 `node` 既有左子节点又有右子节点，那么我们需要找到 `node` 的右子树中的最小值节点，并将其值替换到 `node` 中，然后在右子树中删除这个最小值节点。这是因为右子树中的最小值节点是大于左子树中所有节点值且小于右子树中所有节点值的节点，它在替代被删除节点后能够保持树的平衡性。

函数 `_min_value_node` 用于找到树中的最小值节点，其实现如下：

```
1 def _min_value_node(self, node):
2     current = node
3     while current.left:
4         current = current.left
5     return current
```

这样，当我们删除带有两个子节点的节点时，我们选择将右子树中的最小值节点的值替换到要删除的节点中，然后递归地在右子树中删除这个最小值节点。

8.3 映射实现总结

用来实现映射这一抽象数据类型的多种数据结构，包括有序列表、散列表、二叉搜索树以及 AVL 树。表6-1总结了每个数据结构的性能。

表6-1 映射的不同实现间的性能对比

operation	Sorted List	Hash Table	Binary Search Tree	AVL Tree
put	$O(n)$	$O(1)$	$O(n)$	$O(\log_2 n)$
get	$O(\log_2 n)$	$O(1)$	$O(n)$	$O(\log_2 n)$
in	$O(\log_2 n)$	$O(1)$	$O(n)$	$O(\log_2 n)$
del	$O(n)$	$O(1)$	$O(n)$	$O(\log_2 n)$

9 并查集（Disjoint Set Union）

9.1 核心概念：什么是并查集？

9.1.1 直观类比

想象一个江湖，有很多小弟和帮派：

- **查询 (Find)**：问一个小弟：“你的帮主是谁？”小弟会沿着层级往上问，直到找到那个不认别人做大哥的“大Boss”（代表元素）。
- **合并 (Union)**：两个帮派要合并。帮派 A 的 Boss 认 帮派 B 的 Boss 做大哥。从此两个帮派合为一家。

9.1.2 数据结构本质

并查集是一种维护不相交集的数据结构，主要解决动态连通性问题。

- **存储方式**：通常使用父节点数组 `parent[]`。`parent[i]` 存储节点 `i` 的父节点。
- **根节点**：若 `parent[i] == i`，则 `i` 是该集合的代表（根节点）。

与前缀树 (Trie) 的对比：

- **并查集**：每个节点指向父节点（自下而上），关注的是“最终归属于哪个集合”。
- **前缀树**：每个节点指向子节点（自上而下），关注的是“路径构成的字符串”。

9.2 基本操作与实现

9.2.1 初始化

起初，每个人都是自成一派，自己的 Boss 就是自己。

```
1 parent = [i for i in range(n + 1)]
```

9.2.2 查找 (Find) - 递归实现

通过递归遍历父节点数组，直到找到根节点。

```
1 # 查找元素 i 所在集合的代表元素
2 def find(parent, i):
3     if parent[i] == i:
4         return i
5     else:
6         return find(parent, parent[i])
```

注：最坏情况下，树可能退化成链，时间复杂度为 $O(n)$ 。

9.2.3 合并 (Union)

找到两个元素各自的根节点，将其中一棵树连接到另一棵树的根下。

```
1 def union(parent, i, j):
2     root_i = find(parent, i)
3     root_j = find(parent, j)
4     if root_i != root_j:
5         parent[root_i] = root_j # 将 i 的根指向 j 的根
```

注：简单的合并可能导致树过高，影响后续查找效率。

9.3 算法优化：路径压缩与按秩合并

9.3.1 路径压缩 (Path Compression)

优化思路：在执行 `find` 的过程中，直接将被访问节点及其祖先指向根节点，从而极大地压扁树的高度。

```
1 def find(parent, i):
2     if parent[i] == i:
3         return i
4     else:
5         # 路径压缩：将父节点直接更新为根节点
6         parent[i] = find(parent, parent[i])
7     return parent[i]
```

9.3.2 按秩/大小合并 (Union by Rank/Size)

优化思路：始终将“矮树”合并到“高树”下（按秩），或将“小树”合并到“大树”下（按大小），防止树的高度增长过快。

按秩合并 (Union by Rank) 实现

```
1 class DisjSet:
2     def __init__(self, n):
```

```

3     self.parent = [i for i in range(n)]
4     self.rank = [0] * n # rank 表示树的高度
5
6     def find(self, x):
7         if self.parent[x] != x:
8             self.parent[x] = self.find(self.parent[x]) # 路径压缩
9         return self.parent[x]
10
11    def union(self, x, y):
12        root_x = self.find(x)
13        root_y = self.find(y)
14
15        if root_x != root_y:
16            # 始终将秩小的树接到秩大的树下
17            if self.rank[root_x] < self.rank[root_y]:
18                self.parent[root_x] = root_y
19            elif self.rank[root_x] > self.rank[root_y]:
20                self.parent[root_y] = root_x
21            else:
22                self.parent[root_y] = root_x
23                self.rank[root_x] += 1

```

按大小合并 (Union by Size) 实现

```

1 class UnionFind:
2     def __init__(self, n):
3         self.parent = list(range(n))
4         self.size = [1] * n # size 表示集合中的元素个数
5
6     def find(self, i):
7         if self.parent[i] != i:
8             self.parent[i] = self.find(self.parent[i]) # 路径压缩
9         return self.parent[i]
10
11    def union(self, i, j):
12        root_i = self.find(i)
13        root_j = self.find(j)
14        if root_i != root_j:
15            # 小树合并到大树
16            if self.size[root_i] < self.size[root_j]:
17                self.parent[root_i] = root_j
18                self.size[root_j] += self.size[root_i]
19            else:
20                self.parent[root_j] = root_i
21                self.size[root_i] += self.size[root_j]

```

9.4.3 复杂度分析

- **时间复杂度**：结合路径压缩与按秩合并后，均摊复杂度接近常数，即 $O(\alpha(n))$ ，其中 α 是反阿克曼函数（对于地球上观测到的所有 n ，该值通常不大于 4）。在实际应用中，可以视其为 **常数级** $O(1)$ 。
- **空间复杂度**： $O(n)$ ，用于存储父节点及秩信息。

9.4 编程题目

示例：M晴问9.6.1 学校的班级个数（统计连通分量）

<https://sunnywhy.com/sfbj/9/6/360>

现有一个学校，学校中有若干个班级，每个班级中有若干个学生，每个学生只会存在于一个班级中。如果学生A和学生B处于一个班级，学生B和学生C处于一个班级，那么我们称学生A和学生C也处于一个班级。

现已知学校中共 n 个学生（编号为从1到 n ），并给出 m 组学生关系（指定两个学生处于一个班级），问总共有多少个班级。

输入

第一行两个整数 m 、 n ($1 \leq n \leq 100, 1 \leq m \leq 100$)，分别表示学生个数、学生关系个数；

接下来 m 行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n, a \neq b$)，表示编号为 a 的学生和编号为 b 的学生处于一个班级。

输出

输出一个整数，表示班级个数。

样例1

输入

```
1 5 3
2 4 2
3 1 3
4 2 5
```

输出

```
1 2
```

解释

编号2、4、5的学生在同一个班级，编号1、3的学生在同一个班级，因此共有两个班级。

集合的大小，即统计最后有多少个根。

```
1 def find(x):
2     if parent[x] != x: # 如果不是根结点，继续循环
3         parent[x] = find(parent[x])
4     return parent[x]
5
6 def union(x, y):
7     parent[find(x)] = find(y)
8
9 n, m = map(int, input().split())
10 parent = list(range(n + 1)) # parent[i] == i, 则说明元素i是该集合的根结点
11
12 for _ in range(m):
13     a, b = map(int, input().split())
14     union(a, b)
15
16 classes = set(find(x) for x in range(1, n + 1))
17 print(len(classes))
```

示例：M晴问9.6.2 学校的班级人数（维护集合大小）

<https://sunnywhy.com/sfbj/9/6/361>

现有一个学校，学校中有若干个班级，每个班级中有若干个学生，每个学生只会存在于一个班级中。如果学生 A 和学生 B 处于一个班级，学生 B 和学生 C 处于一个班级，那么我们称学生 A 和学生 C 也处于一个班级。

现已知学校中共 n 个学生（编号为从 1 到 n ），并给出 m 组学生关系（指定两个学生处于一个班级），问总共有多少个班级，并按降序给出每个班级的人数。

输入

第一行两个整数 m 、 n ($1 \leq n \leq 100, 1 \leq m \leq 100$)，分别表示学生个数、学生关系个数；

接下来 m 行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n, a \neq b$)，表示编号为 a 的学生和编号为 b 的学生处于一个班级。

输出

第一行输出一个整数，表示班级个数；

第二行若干个整数，按降序给出每个班级的人数。整数之间用空格隔开，行末不允许有多余的空格。

样例1

输入

```
1 | 5 3
2 | 4 2
3 | 1 3
4 | 2 5
```

输出

```
1 | 2
2 | 3 2
```

解释

编号 2、4、5 的学生在同一个班级，编号 1、3 的学生在同一个班级，因此共有两个班级，人数分别是 3 和 2。

核心： 合并时更新 `size` 数组。

```
1 def find(x):
2     if parent[x] != x:
3         parent[x] = find(parent[x])
4     return parent[x]
5
6 def union(x, y):
7     root_x = find(x)
8     root_y = find(y)
9     if root_x != root_y:
10        parent[root_x] = root_y
11        size[root_y] += size[root_x]
12
```

```

13 n, m = map(int, input().split())
14 parent = list(range(n + 1))
15 size = [1] * (n + 1)
16
17 for _ in range(m):
18     a, b = map(int, input().split())
19     union(a, b)
20
21 # 关键：只取根节点的 size 值
22 classes = [size[x] for x in range(1, n + 1) if x == parent[x]]
23 print(len(classes))
24 print(' '.join(map(str, sorted(classes, reverse=True))))

```

练习：M晴天9.6.3 是否相同班级（连通）

<https://sunnywhy.com/sfbj/9/6/362>

现有一个学校，学校中有若干个班级，每个班级中有若干学生，每个学生只会存在于一个班级中。如果学生 A 和学生 B 处于一个班级，学生 B 和学生 C 处于一个班级，那么我们称学生 A 和学生 C 也处于一个班级。

现已知学校中共 n 个学生（编号为从 1 到 n ），并给出 m 组学生关系（指定两个学生处于一个班级）。然后给出 k 个查询，每个查询询问两个学生是否在同一个班级。

输入

第一行两个整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 10^5$)，分别表示学生个数、学生关系个数；

接下来 m 行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n, a \neq b$)，表示编号为 a 的学生和编号为 b 的学生处于一个班级。

然后一个整数 k ($1 \leq k \leq 10^3$)，表示查询个数；

接下来 k 行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n$)，表示询问编号为 a 的学生和编号为 b 的学生是否在同一个班级。

输出

每个查询输出一行，如果在同一个班级，那么输出 **Yes**，否则输出 **No**。

样例1

输入

```

1 5 3
2 4 2
3 1 3
4 2 5
5 2
6 4 5
7 1 2

```

输出

```

1 Yes
2 No

```


解释

编号 2、4、5 的学生在同一个班级，编号 1、3 的学生在同一个班级，因此编号 4 和 5 的学生在同一个班级，编号 1 和 2 的学生不在同一个班级。

相同班级：直接判断 `find(a) == find(b)`。

```
1 def find(x):
2     if parent[x] != x:
3         parent[x] = find(parent[x])
4     return parent[x]
5
6 def union(x, y):
7     parent[find(x)] = find(y)
8
9 n, m = map(int, input().split())
10 parent = list(range(n + 1))
11
12 for _ in range(m):
13     a, b = map(int, input().split())
14     union(a, b)
15
16 k = int(input())
17 for _ in range(k):
18     a, b = map(int, input().split())
19     if find(a) == find(b):
20         print('Yes')
21     else:
22         print('No')
```

练习：M晴问9.6.4 迷宫连通性（全连通）

<https://sunnywhy.com/sfbj/9/6/363>

现有一个迷宫，迷宫中有 n 个房间（编号为从 1 到 n ），房间与房间之间可能连通。如果房间 A 和房间 B 连通，房间 B 和房间 C 连通，那么我们称房间 A 和房间 C 也连通。给定 m 组连通关系（指定两个房间连通），问迷宫中的所有房间是否连通。

输入

第一行两个整数 n, m ($1 \leq n \leq 100, 1 \leq m \leq 100$)，分别表示房间个数、连通关系个数；

接下来行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n$)，表示编号为 a 的房间和编号为 b 的房间是连通的。

输出

如果所有房间连通，那么输出 Yes，否则输出 No。

样例1

输入

```
1 | 5 4
2 | 4 2
3 | 1 3
4 | 2 5
5 | 1 5
```

输出

```
1 | Yes
```

解释

所有房间都连通，因此输出 `Yes`。

样例2

输入

```
1 | 5 3
2 | 4 2
3 | 1 3
4 | 2 5
```

输出

```
1 | No
```

解释

编号 `2`、`4`、`5` 的房间互相连通，编号 `1`、`3` 的房间互相连通，因此没有全部互相连通，输出 `No`。

连通性：若最终集合个数为 1，则全连通。

```
1 def find(x):
2     if parent[x] != x:
3         parent[x] = find(parent[x])
4     return parent[x]
5
6 def union(x, y):
7     parent[find(x)] = find(y)
8
9 n, m = map(int, input().split())
10 parent = list(range(n + 1))
11
12 for _ in range(m):
13     a, b = map(int, input().split())
14     union(a, b)
15
16 sets = set(find(x) for x in range(1, n + 1))
17 if len(sets) == 1:
18     print('Yes')
19 else:
20     print('No')
```

练习：M晴天9.6.5 班级最高分

<https://sunnywhy.com/sfbj/9/6/364>

现有一个学校，学校中有若干个班级，每个班级中有若干个学生，每个学生只会存在于一个班级中。如果学生A和学生B处于一个班级，学生B和学生C处于一个班级，那么我们称学生A和学生C也处于一个班级。

现已知学校中共 n 个学生（编号为从1到 n ），每个学生有一个考试分数，再给出 m 组学生关系（指定两个学生处于一个班级），问总共有多少个班级，并按降序给出每个班级的最高考试分数。

输入

第一行两个整数 n 、 m ($1 \leq n \leq 100, 1 \leq m \leq 100$)，分别表示学生个数、学生关系个数；

第二行为用空格隔开的 n 个整数 ($0 \leq \text{每个整数} \leq 100$)，表示个学生的考试分数；

接下来 m 行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n$)，表示编号为 a 的学生和编号为 b 的学生处于一个班级。

输出

第一行输出一个整数，表示班级个数；

第二行若干个整数，按降序给出每个班级的最高考试分数。整数之间用空格隔开，行末不允许有多余的空格。

样例1

输入

```
1 5 3
2 88 90 86 92 95
3 4 2
4 1 3
5 2 5
```

输出

```
1 2
2 95 88
```

解释

编号2、4、5的学生在同一个班级，编号1、3的学生在同一个班级，因此共有两个班级，最高分数分别是编号1的学生的88分、编号5的学生的95分。

这个问题是一个典型的连通性问题。我们需要把具有关系的学生归到同一个集合（班级）中。并查集是处理这类问题最高效的数据结构。

```
1 def find(x):
2     if parent[x] != x:
3         parent[x] = find(parent[x])
4     return parent[x]
5
6 def union(x, y):
7     root_x = find(x)
8     root_y = find(y)
9     if root_x != root_y:
10        parent[root_x] = root_y
```

```

11     scores[root_y] = max(scores[root_y], scores[root_x])
12
13 n, m = map(int, input().split())
14 parent = list(range(n + 1))
15 scores = list(map(int, input().split()))
16 scores.insert(0, 0) # to make the scores 1-indexed
17
18 for _ in range(m):
19     a, b = map(int, input().split())
20     union(a, b)
21
22 classes_scores = [scores[find(x)] for x in range(1, n + 1) if x == parent[x]]
23 print(len(classes_scores))
24 print(' '.join(map(str, sorted(classes_scores, reverse=True))))

```

复杂度分析

- **时间复杂度：** $O(m \cdot \alpha(n))$ ，其中 α 是反阿克曼函数（几乎可以看作常数）。对于 $n=100, m=100$ 的规模，性能绰绰有余
- **空间复杂度：** $O(n)$ ，用于存储父节点数组和分数。

T01182: 食物链

dsu, <http://cs101.openjudge.cn/practice/01182>

动物王国中有三类动物A,B,C，这三类动物的食物链构成了有趣的环形。A吃B，B吃C，C吃A。

现有N个动物，以1－N编号。每个动物都是A,B,C中的一种，但是我们并不知道它到底是哪一种。

有人用两种说法对这N个动物所构成的食物链关系进行描述：

第一种说法是"1 X Y"，表示X和Y是同类。

第二种说法是"2 X Y"，表示X吃Y。

此人对N个动物，用上述两种说法，一句接一句地说出K句话，这K句话有的是真的，有的是假的。当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

- 1) 当前的话与前面的某些真的话冲突，就是假话；
- 2) 当前的话中X或Y比N大，就是假话；
- 3) 当前的话表示X吃X，就是假话。

你的任务是根据给定的N（ $1 \leq N \leq 50,000$ ）和K句话（ $0 \leq K \leq 100,000$ ），输出假话的总数。

输入

第一行是两个整数N和K，以一个空格分隔。

以下K行每行是三个正整数 D, X, Y，两数之间用一个空格隔开，其中D表示说法的种类。

若D=1，则表示X和Y是同类。

若D=2，则表示X吃Y。

输出

只有一个整数，表示假话的数目。

样例输入

```

1 100 7
2 1 101 1
3 2 1 2
4 2 2 3
5 2 3 3
6 1 1 3
7 2 3 1
8 1 5 5

```

样例输出

```

1 3

```

来源: Noi 01

逻辑点:

1. 建立 $3N$ 大小的并查集, 划分三个区间: $[1, N]$ 为同类, $[N+1, 2N]$ 为食物, $[2N+1, 3N]$ 为天敌。
2. 逻辑合并:
 - 如果 X 与 Y 是同类: 合并 $(X, Y), (X + N, Y + N), (X + 2N, Y + 2N)$ 。
 - 如果 X 吃 Y : 合并 $(X + N, Y), (X, Y + 2N), (X + 2N, Y + N)$ 。

```

1 class DisjointSet:
2     def __init__(self, n):
3         # 设 [1, n] 区间表示同类, [n+1, 2*n] 表示 x 吃的动物, [2*n+1, 3*n] 表示吃 x 的动物。
4         self.parent = [i for i in range(3 * n + 1)] # 每个动物有三种可能的类型, 用 3 * n 来表示每种类型的
并查集
5         self.rank = [0] * (3 * n + 1)
6
7     def find(self, u):
8         if self.parent[u] != u:
9             self.parent[u] = self.find(self.parent[u])
10        return self.parent[u]
11
12    def union(self, u, v):
13        pu, pv = self.find(u), self.find(v)
14        if pu == pv:
15            return False
16        if self.rank[pu] > self.rank[pv]:
17            self.parent[pv] = pu
18        elif self.rank[pu] < self.rank[pv]:
19            self.parent[pu] = pv
20        else:
21            self.parent[pv] = pu
22            self.rank[pu] += 1
23        return True
24
25
26 def is_valid(n, k, statements):
27     dsu = DisjointSet(n)
28
29     def find_disjoint_set(x):
30         if x > n:
31             return False

```

```

32         return True
33
34     false_count = 0
35     for d, x, y in statements:
36         if not find_disjoint_set(x) or not find_disjoint_set(y):
37             false_count += 1
38             continue
39         if d == 1: # X and Y are of the same type
40             if dsu.find(x) == dsu.find(y + n) or dsu.find(x) == dsu.find(y + 2 * n):
41                 false_count += 1
42             else:
43                 dsu.union(x, y)
44                 dsu.union(x + n, y + n)
45                 dsu.union(x + 2 * n, y + 2 * n)
46         else: # X eats Y
47             if dsu.find(x) == dsu.find(y) or dsu.find(x + 2*n) == dsu.find(y):
48                 false_count += 1
49             else: # [1,n] 区间表示同类, [n+1,2*n]表示x吃的动物, [2*n+1,3*n]表示吃x的动物
50                 dsu.union(x + n, y)
51                 dsu.union(x, y + 2 * n)
52                 dsu.union(x + 2 * n, y + n)
53
54     return false_count
55
56
57 if __name__ == "__main__":
58     N, K = map(int, input().split())
59     statements = []
60     for _ in range(K):
61         D, X, Y = map(int, input().split())
62         statements.append((D, X, Y))
63     result = is_valid(N, K, statements)
64     print(result)
65

```

《挑战程序设计竞赛（第2版）》的2.4.4并查集，也有讲到。

```

1  # 并查集, https://zhuanlan.zhihu.com/p/93647900/
2  '''
3  我们设[0,n]区间表示同类, [n,2*n]区间表示x吃的动物, [2*n,3*n]表示吃x的动物。
4
5  如果是关系1:
6      将y和x合并。将y吃的与x吃的合并。将吃y的和吃x的合并。
7  如果是关系2:
8      将y和x吃的合并。将吃y的与x合并。将y吃的与吃x的合并。
9  原文链接: https://blog.csdn.net/qq\_34594236/article/details/72587829
10 '''
11 # p = [0]*150001
12
13 def find(x): # 并查集查询
14     if p[x] == x:
15         return x
16     else:
17         p[x] = find(p[x]) # 父节点设为根节点。目的是路径压缩。
18     return p[x]
19

```

```

20 n,k = map(int, input().split())
21
22 p = [0]*(3*n + 1)
23 for i in range(3*n+1): #并查集初始化
24     p[i] = i
25
26 ans = 0
27 for _ in range(k):
28     a,x,y = map(int, input().split())
29     if x>n or y>n:
30         ans += 1; continue
31
32     if a==1:
33         if find(x+n)==find(y) or find(y+n)==find(x):
34             ans += 1; continue
35
36     # 合并
37     p[find(x)] = find(y)
38     p[find(x+n)] = find(y+n)
39     p[find(x+2*n)] = find(y+2*n)
40 else:
41     if find(x)==find(y) or find(y+n)==find(x):
42         ans += 1; continue
43     p[find(x+n)] = find(y)
44     p[find(y+2*n)] = find(x)
45     p[find(x+2*n)] = find(y+n)
46
47 print(ans)

```

9.5 总结：并查集的三个灵魂问题

1. 什么时候用？ 动态维护等价关系、判断连通性、最小生成树 (Kruskal)。
2. 怎么变快？ 路径压缩 + 按秩合并。
3. 怎么统计？ 在根节点记录 size（大小）或 max/min（极值）。

A 非递归线段树（Iterative Segment Tree）

Segment tree | Efficient implementation

<https://www.geeksforgeeks.org/segment-tree-efficient-implementation/>

Last Updated : 20 Jan, 2025

线段树是一种极其高效的数据结构，用于解决区间查询和单点更新问题。本文重点介绍一种基于数组索引、非递归的实现方式，其代码比传统的递归版更简洁且执行效率更高。

1. 问题描述

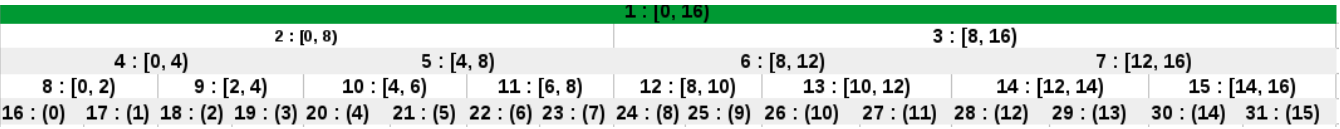
我们需要维护一个长度为 n 的数组 $arr[0 \dots n - 1]$ ，并支持以下两种操作：

1. 区间查询：计算索引 l 到 r 之间的元素之和 ($0 \leq l \leq r \leq n - 1$)。
2. 单点更新：将数组指定索引 i 的值修改为 x ($arr[i] = x$)。

方法	区间查询	单点更新
普通数组循环	$O(n)$	$O(1)$
前缀和数组	$O(1)$	$O(n)$
线段树	$O(\log n)$	$O(\log n)$

2. 结构原理

考虑一个包含 16 个元素的数组，其线段树结构如下图所示：



- **存储结构**：我们使用一个大小为 $2N$ 的数组 `tree` 来存储树。
- **叶子节点**：原数组的元素存储在 `tree` 的索引 $[N, 2N - 1]$ 中。因此，原数组索引为 i 的元素对应树中的 `tree[i + n]`。
- **内部节点**：索引为 i 的节点，其左孩子为 $2i$ ，右孩子为 $2i + 1$ 。
- **父节点**：索引为 i 的节点，其父节点为 $i // 2$ （位运算 `i >> 1`）。

3. Python 高效实现

在实现中，我们利用位运算（如 `<< 1` 代替 `*2`）来进一步提升性能。

```
1  # 线段树非递归实现（以区间求和为例）
2
3  def build(arr, n):
4      """构建线段树"""
5      # 将叶子节点插入树的后半部分
6      for i in range(n):
7          tree[n + i] = arr[i]
8
9      # 自底向上计算父节点的值
10     for i in range(n - 1, 0, -1):
11         # tree[i] = 左孩子 + 右孩子
12         tree[i] = tree[i << 1] + tree[i << 1 | 1]
13
14 def update(p, value, n):
15     """单点更新：将索引 p 处的值改为 value"""
16     p += n
17     tree[p] = value
18
19     # 向上回溯更新所有祖先节点
20     i = p
21     while i > 1:
22         # tree[i>>1] 是父节点，tree[i] 和 tree[i^1] 是左右两个子节点
23         tree[i >> 1] = tree[i] + tree[i ^ 1]
24         i >>= 1
25
26 def query(l, r, n):
27     """区间查询：求 [l, r) 的和（左闭右开）"""
```



```

28     res = 0
29     l += n
30     r += n
31
32     while l < r:
33         # 如果 l 是右孩子 (奇数), 则它不在父节点的完整覆盖范围内
34         if l & 1:
35             res += tree[l]
36             l += 1
37         # 如果 r 是右孩子 (奇数), 则 r-1 是左孩子, 将其计入结果
38         if r & 1:
39             r -= 1
40             res += tree[r]
41
42         # 移向上一层
43         l >>= 1
44         r >>= 1
45     return res
46
47 # 示例
48 if __name__ == "__main__":
49     a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
50     n = len(a)
51     tree = [0] * (2 * n)
52
53     build(a, n)
54     print(f"区间 [1, 3) 的和: {query(1, 3, n)}") # 输出 5 (2+3)
55
56     update(2, 1, n) # 将索引 2 的值从 3 改为 1
57     print(f"更新后区间 [1, 3) 的和: {query(1, 3, n)}") # 输出 3 (2+1)

```

类封装

```

1 class SegmentTree:
2     def __init__(self, data, func=lambda x, y: x + y, default=0):
3         """
4         func: 结合律函数 (sum, max, min, gcd等)
5         default: 恒等元 (sum为0, max为-inf, min为inf)
6         """
7         self.n = len(data)
8         self.func = func
9         self.default = default
10        self.tree = [default] * (2 * self.n)
11
12        # 1. 填入叶子节点
13        for i in range(self.n):
14            self.tree[i + self.n] = data[i]
15        # 2. 自底向上构建
16        for i in range(self.n - 1, 0, -1):
17            self.tree[i] = self.func(self.tree[i << 1], self.tree[i << 1 | 1])
18
19    def update(self, p, value):
20        """将索引 p 处的元素修改为 value, O(log n)"""
21        p += self.n
22        self.tree[p] = value
23        while p > 1:

```

```

24         # 向上更新,  $p \gg 1$  是定位左右兄弟的技巧
25         self.tree[p >> 1] = self.func(self.tree[p], self.tree[p >> 1])
26         p >>= 1
27
28     def query(self, l, r):
29         """查询区间  $[l, r)$  的值,  $O(\log n)$ """
30         res = self.default
31         l += self.n
32         r += self.n
33         while l < r:
34             if l & 1: # 如果 l 是右孩子, 单独计入并右移
35                 res = self.func(res, self.tree[l])
36                 l += 1
37             if r & 1: # 如果 r 是右孩子, 说明 r-1 是左孩子, 计入 r-1
38                 r -= 1
39                 res = self.func(res, self.tree[r])
40             l >>= 1
41             r >>= 1
42         return res
43
44 if __name__ == "__main__":
45     a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
46     sol = SegmentTree(a, lambda x, y: x + y)
47
48     print(f"区间  $[1, 3)$  的和: {sol.query(1, 3)}") # 输出 5 (2+3)
49
50     sol.update(2, 1) # 将索引 2 的值从 3 改为 1
51     print(f"更新后区间  $[1, 3)$  的和: {sol.query(1, 3)}") # 输出 3 (2+1)

```

4. 查询逻辑深度解析

为什么只在索引为奇数时累加？

在查询区间 $[l, r)$ 时：

1. **左端点 l** ：如果 l 是偶数，它是父节点的左孩子，父节点可能完全被包含在区间内，所以我们直接上移到父节点。如果 l 是奇数，它是父节点的右孩子，这意味着父节点包含了区间外的值，所以我们只累加当前 `tree[l]`，然后 $l + 1$ 变成偶数，再进入父节点。
2. **右端点 r** ：同理，如果 r 是奇数，说明 $r-1$ （真正的区间右界）是一个左孩子，我们必须单独处理它。

这种方法避免了复杂的边界判断，且通过 $l \gg= 1$ 不断减半，确保了 $O(\log n)$ 的时间复杂度。

5. 综合示例：

线段树题目。

- 什么时候用？：当你看到题目涉及到“区间更新”、“区间查询”，并且普通的“前缀和”或“滑动窗口”无法处理动态变化的数据时，线段树就是首选方案。
- 与其他算法对比：在力扣上，很多线段树能做的题，**树状数组（Binary Indexed Tree / Fenwick Tree）**也能做。树状数组代码更短、常数更小，但线段树的优势在于它的**通用性**（不仅能做求和，还能做区间最大公约数、区间最值、甚至是复杂的动态规划优化）。

A. 基础版：单点更新 + 区间查询

示例M307.区域和检索 - 数组可修改

binary indexed tree, segment tree <https://leetcode.cn/problems/range-sum-query-mutable/>

给你一个数组 `nums`，请你完成两类查询。

1. 其中一类查询要求 **更新** 数组 `nums` 下标对应的值
2. 另一类查询要求返回数组 `nums` 中索引 `left` 和索引 `right` 之间（包含）的 `nums` 元素的 **和**，其中 `left <= right`

实现 `NumArray` 类：

- `NumArray(int[] nums)` 用整数数组 `nums` 初始化对象
- `void update(int index, int val)` 将 `nums[index]` 的值 **更新** 为 `val`
- `int sumRange(int left, int right)` 返回数组 `nums` 中索引 `left` 和索引 `right` 之间（包含）的 `nums` 元素的 **和**（即，`nums[left] + nums[left + 1], ..., nums[right]`）

示例 1：

```
1  输入：
2  ["NumArray", "sumRange", "update", "sumRange"]
3  [[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
4  输出：
5  [null, 9, null, 8]
6
7  解释：
8  NumArray numArray = new NumArray([1, 3, 5]);
9  numArray.sumRange(0, 2); // 返回 1 + 3 + 5 = 9
10 numArray.update(1, 2);   // nums = [1,2,5]
11 numArray.sumRange(0, 2); // 返回 1 + 2 + 5 = 8
```

提示：

- `1 <= nums.length <= 3 * 104`
- `-100 <= nums[i] <= 100`
- `0 <= index < nums.length`
- `-100 <= val <= 100`
- `0 <= left <= right < nums.length`
- 调用 `update` 和 `sumRange` 方法次数不大于 `3 * 104`

```
1  class NumArray:
2
3      def __init__(self, nums: List[int]):
4          """
5          func: 结合律函数 (sum, max, min, gcd等)
6          default: 恒等元 (sum为0, max为-inf, min为inf)
7          """
8          self.n = len(nums)
9          self.func = lambda x, y: x + y
10         self.default = 0
```

```

11     self.tree = [self.default] * (2 * self.n)
12
13     # 1. 填入叶子节点
14     for i in range(self.n):
15         self.tree[i + self.n] = nums[i]
16     # 2. 自底向上构建
17     for i in range(self.n - 1, 0, -1):
18         self.tree[i] = self.func(self.tree[i << 1], self.tree[i << 1 | 1])
19
20     def update(self, index: int, val: int) -> None:
21         p = index
22         """将索引 p 处的元素修改为 value, O(log n)"""
23         p += self.n
24         self.tree[p] = val
25         while p > 1:
26             # 向上更新, p ^ 1 是定位左右兄弟的技巧
27             self.tree[p >> 1] = self.func(self.tree[p], self.tree[p ^ 1])
28             p >>= 1
29
30
31     def sumRange(self, left: int, right: int) -> int:
32         l, r = left, right + 1
33         """查询区间 [l, r) 的值, O(log n)"""
34         res = self.default
35         l += self.n
36         r += self.n
37         while l < r:
38             if l & 1: # 如果 l 是右孩子, 单独计入并右移
39                 res = self.func(res, self.tree[l])
40                 l += 1
41             if r & 1: # 如果 r 是右孩子, 说明 r-1 是左孩子, 计入 r-1
42                 r -= 1
43                 res = self.func(res, self.tree[r])
44             l >>= 1
45             r >>= 1
46         return res
47
48
49     # Your NumArray object will be instantiated and called as such:
50     # obj = NumArray(nums)
51     # obj.update(index,val)
52     # param_2 = obj.sumRange(left,right)

```

示例Codeforces 1364A - XXXXX

brute force, data structures, number theory, two pointers, 1200,

<https://codeforces.com/problemset/problem/1364/A>

Ehab loves number theory, but for some reason he hates the number x . Given an array a , find the length of its longest subarray such that the sum of its elements **isn't** divisible by x , or determine that such subarray doesn't exist.

An array a is a subarray of an array b if a can be obtained from b by deletion of several (possibly, zero or all) elements from the beginning and several (possibly, zero or all) elements from the end.

Input

The first line contains an integer t ($1 \leq t \leq 5$) — the number of test cases you need to solve. The description of the test cases follows.

The first line of each test case contains 2 integers n and x ($1 \leq n \leq 10^5$, $1 \leq x \leq 10^4$) — the number of elements in the array a and the number that Ehab hates.

The second line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^4$) — the elements of the array a .

Output

For each testcase, print the length of the longest subarray whose sum isn't divisible by x . If there's no such subarray, print -1 .

Example

input

```
1 3
2 3 3
3 1 2 3
4 3 4
5 1 2 3
6 2 2
7 0 6
```

output

```
1 2
2 3
3 -1
```

Note

In the first test case, the subarray $2, 3$ has sum of elements 5 , which isn't divisible by 3 .

In the second test case, the sum of elements of the whole array is 6 , which isn't divisible by 4 .

In the third test case, all subarrays have an even sum, so the answer is -1 .

题目大意：寻找最长的一段子数组，使其元素之和不能被 x 整除。

针对这个题目虽然线段树不是最优解，但是利用上述线段树模板，Pypy3 提交可以AC。时间复杂度是 $O(n \log n)$ 。如果用sum求和， $O(n^2)$ ，Pypy3也会在test3 超时。

```
1 import sys
2
3 # 设置递归深度和快速输入
4 input = sys.stdin.read
5
6 def solve():
7     raw_data = input().split()
8     if not raw_data:
9         return
10
11     ptr = 0
12     t = int(raw_data[ptr])
```

```

13     ptr += 1
14     output = []
15
16     for _ in range(t):
17         n = int(raw_data[ptr])
18         x = int(raw_data[ptr + 1])
19         ptr += 2
20         a = [int(val) % x for val in raw_data[ptr : ptr + n]]
21         ptr += n
22
23         # 初始化线段树
24         tree = [0] * (2 * n)
25
26         # 构造函数
27         for i in range(n):
28             tree[n + i] = a[i]
29         for i in range(n - 1, 0, -1):
30             tree[i] = tree[i << 1] + tree[i << 1 | 1]
31
32         def get_query(l, r):
33             res = 0
34             l += n
35             r += n
36             while l < r:
37                 if l & 1:
38                     res += tree[l]
39                     l += 1
40                 if r & 1:
41                     r -= 1
42                     res += tree[r]
43                 l >>= 1
44                 r >>= 1
45             return res
46
47         # 逻辑处理
48         ans = -1
49         # 从左向右缩小, 找到第一个不被 x 整除的前缀
50         l, r = 0, n - 1
51         while l <= r:
52             if get_query(l, r + 1) % x != 0:
53                 ans = max(ans, r - l + 1)
54                 break
55             l += 1
56
57         # 从右向左缩小, 找到第一个不被 x 整除的后缀
58         l, r = 0, n - 1
59         while l <= r:
60             if get_query(l, r + 1) % x != 0:
61                 ans = max(ans, r - l + 1)
62                 break
63             r -= 1
64
65         output.append(str(ans))
66
67     sys.stdout.write("\n".join(output) + "\n")
68
69 if __name__ == "__main__":
70     solve()

```

B. 离散化 + 线段树

力扣上很多题目的坐标范围非常大（比如 10^9 ），但实际出现的点很少（比如 10^5 ）。这时需要先将坐标离散化，然后再建树。

T327. 区间和的个数

binary indexed tree, segment tree, binary search, <https://leetcode.cn/problems/count-of-range-sum/>

给你一个整数数组 `nums` 以及两个整数 `lower` 和 `upper`。求数组中，值位于范围 `[lower, upper]`（包含 `lower` 和 `upper`）之内的 **区间和的个数**。

区间和 `S(i, j)` 表示在 `nums` 中，位置从 `i` 到 `j` 的元素之和，包含 `i` 和 `j` ($i \leq j$)。

示例 1:

```
1 输入: nums = [-2,5,-1], lower = -2, upper = 2
2 输出: 3
3 解释: 存在三个区间: [0,0]、[2,2] 和 [0,2]，对应的区间和分别是: -2、-1、2。
```

示例 2:

```
1 输入: nums = [0], lower = 0, upper = 0
2 输出: 1
```

提示:

- `1 <= nums.length <= 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`
- `-10^5 <= lower <= upper <= 10^5`
- 题目数据保证答案是一个 **32 位** 的整数

这是一道非常经典的问题。直接使用普通的线段树会遇到两个挑战:

- 值域过大**: 区间和的范围可能非常大 (-2^{31} 到 $2^{31} - 1$)，不能直接以数值作为索引。
- 动态性质**: 我们需要统计的是“之前出现的、落在某个范围内的前缀和个数”。

为了解决这两个问题，我们需要结合 **前缀和** 和 **坐标离散化**，再套用 `SegmentTree` 类模版。

解题思路

1. 前缀和转换:

令 $P[i]$ 为前 i 个元素的和。区间和 $S(i, j) = P[j + 1] - P[i]$ 。

题目要求: $lower \leq P[j + 1] - P[i] \leq upper$ 。

等价于: $P[j + 1] - upper \leq P[i] \leq P[j + 1] - lower$ 。

当我们遍历到 $P[j + 1]$ 时，我们需要统计之前有多少个 $P[i]$ 落在 $[P[j + 1] - upper, P[j + 1] - lower]$ 范围内。

2. 坐标离散化:

由于 $P[i]$ 的值可能非常大，但我们关心的不同值最多只有 $N + 1$ 个（即所有的前缀和）。我们将所有前缀和排序并去重，用它们的**排名（索引）**来构建线段树。

3. 线段树用途:

这里的线段树不是用来存 `nums` 的值，而是用来存**频率**。`tree[idx]` 表示“前缀和排名为 `idx` 的数出现了多少次”。

Python 实现代码

```
1 import bisect
2
3 class SegmentTree:
4     def __init__(self, n):
5         self.n = n
6         self.tree = [0] * (2 * n)
7
8     def update(self, p, value):
9         """单点更新: 将索引 p 处的值设为 value (在这里是更新频率)"""
10        p += self.n
11        self.tree[p] = value
12        i = p
13        while i > 1:
14            # 父节点等于两个子节点之和
15            self.tree[i >> 1] = self.tree[i] + self.tree[i ^ 1]
16            i >>= 1
17
18    def query(self, l, r):
19        """区间查询: 查询排名在 [l, r) 范围内的前缀和个数"""
20        res = 0
21        l += self.n
22        r += self.n
23        while l < r:
24            if l & 1:
25                res += self.tree[l]
26                l += 1
27            if r & 1:
28                r -= 1
29                res += self.tree[r]
30            l >>= 1
31            r >>= 1
32        return res
33
34 class Solution:
35     def countRangeSum(self, nums: list[int], lower: int, upper: int) -> int:
36         # 1. 计算前缀和 P
37         # P[0] = 0, P[1] = nums[0], ...
38         P = [0]
39         curr = 0
40         for x in nums:
41             curr += x
42             P.append(curr)
43
44         # 2. 离散化所有可能出现的前缀和数值
45         # 排序并去重, 建立数值到索引的映射
46         all_values = sorted(list(set(P)))
47         val_map = {val: i for i, val in enumerate(all_values)}
48         n_unique = len(all_values)
49
50         # 3. 初始化线段树 (记录每个前缀和出现的次数)
51         st = SegmentTree(n_unique)
52         count_freq = [0] * n_unique # 记录每个索引当前出现的频次
53
54         ans = 0
```



```

55     # 4. 遍历前缀和
56     for val in P:
57         # 我们需要寻找之前的 P[i] 满足:
58         # val - upper <= P[i] <= val - lower
59
60         # 使用 bisect 找到对应的排名范围
61         # left_idx: 第一个 >= (val - upper) 的排名
62         # right_idx: 第一个 > (val - lower) 的排名
63         left_idx = bisect.bisect_left(all_values, val - upper)
64         right_idx = bisect.bisect_right(all_values, val - lower)
65
66         # 在线段树中查询这个排名区间内的个数
67         if left_idx < right_idx:
68             ans += st.query(left_idx, right_idx)
69
70         # 更新当前前缀和的频率
71         idx = val_map[val]
72         count_freq[idx] += 1
73         st.update(idx, count_freq[idx])
74
75     return ans
76

```

关键点解释:

1. 为什么 `ans += query` 在 `update` 之前?

- 因为区间和 $S(i, j)$ 要求 $i \leq j$ 。在计算 $P[j + 1]$ 时, 我们只能统计它之前出现的 $P[i]$ 。先查询再插入, 保证了查询到的都是索引小于当前位置的前缀和。

2. `bisect_left` 和 `bisect_right` 的妙用:

- `all_values` 存储了所有可能的前缀和。
- 我们要找范围 $[L, R]$ 内的数。
- `bisect_left(all_values, L)` 返回第一个大于等于 L 的索引。
- `bisect_right(all_values, R)` 返回第一个大于 R 的索引。
- 在线段树中查询 `[left_idx, right_idx)` 正好覆盖了所有数值在 $[L, R]$ 之间的前缀和的频次。

3. 复杂度分析:

- 时间复杂度:** 计算前缀和 $O(N)$, 排序去重 $O(N \log N)$, 遍历并操作线段树 $O(N \log N)$ 。总复杂度 $O(N \log N)$, 完全可以通过 10^5 的数据量。
- 空间复杂度:** 线段树和映射表需要 $O(N)$ 空间。

B 前缀树 (Trie / 字典树) 深度解析

参考源: GeeksforGeeks - Trie Data Structure

<https://www.geeksforgeeks.org/introduction-to-trie-data-structure-and-algorithm-tutorials/>

Last Updated : 1 Sep, 2025

Trie (发音同 "try") 又称**前缀树**或**字典树**。它是一种树形数据结构, 专门用于高效地存储和检索字符串数据集中的键。其名称源自单词 "retrieval" (检索), 核心思想是利用字符串的公共前缀来减少查询时间、节省空间。

核心特性

- **公共前缀共享**：如果两个字符串有共同的前缀，它们在 Trie 中会共享同一段祖先路径。
- **有序性**：Trie 可以方便地按字母顺序对字符串进行排序。
- **前缀匹配**：它是处理“以某前缀开头”的搜索任务（如自动补全）的最佳选择。

1. 为什么选择 Trie 而不是哈希表？

虽然哈希表也能在 $O(1)$ 的平均时间内完成搜索，但 Trie 在以下场景具有显著优势：

1. **前缀搜索**：Trie 可以在 $O(L)$ 时间内找到具有相同前缀的所有单词，而哈希表不支持前缀匹配。
2. **字典序排序**：Trie 结构天然有序，遍历即可获得字典序排列，而哈希表是无序的。
3. **无哈希冲突**：Trie 不存在哈希碰撞问题。
4. **最坏情况保障**：在最坏情况下，Trie 的搜索效率仍为 $O(L)$ （ L 为字符串长度），而哈希表在发生大量碰撞时性能会退化。
5. **节省空间**：当存在大量公共前缀时，Trie 存储字符串的实际空间比哈希表更少。

2. Trie 的结构属性

- **根节点**：每个 Trie 都有一个不包含字符的根节点。
- **边代表字符**：从一个节点到其子节点的边代表一个特定的字符（通常通过指针数组或哈希映射实现）。
- **标志位 (isEndOfWord)**：每个节点包含一个布尔标志，用于指示从根到该节点的路径是否构成一个完整的单词。

在我的 Python 代码实现中：

```
1 class TrieNode:
2     def __init__(self):
3         self.children = [None] * 26
4         # self.isEndOfWord = False # 传统做法
5         self.word_count = 0        # 在这个实现中，用 word_count 代替 isEndOfWord
```

选择使用 `word_count` 属性来替代传统的 `isEndOfWord` 布尔标志。

原因和好处：

1. **支持重复单词的插入**：如果允许插入重复的单词，`isEndOfWord` 只能标记“是”或“否”，无法区分插入了多少个。而 `word_count` 可以轻松记录。例如，如果你插入两次 "apple"，`word_count` 会变成 2。
2. **简化删除逻辑**：
 - 在删除一个单词时，我们只需要将 `word_count` 减一。
 - 判断一个单词是否存在，只需检查 `word_count > 0`。
 - 判断一个节点是否是单词的结尾，只需检查 `word_count > 0`。
 - 这些比单独的布尔标志和额外的计数要方便得多。

所以，总结来说：

- **`isEndOfWord` (传统方式)**：表示“从根节点到此节点代表的路径，是否至少构成一个完整的单词”。
- **`self.word_count` (我的实现方式)**：表示“从根节点到此节点代表的路径，总共代表多少个完整的单词”。

当 `self.word_count` 大于 0 时，就意味着这个节点是至少一个单词的结尾。

因此，在 `insert` 方法中：

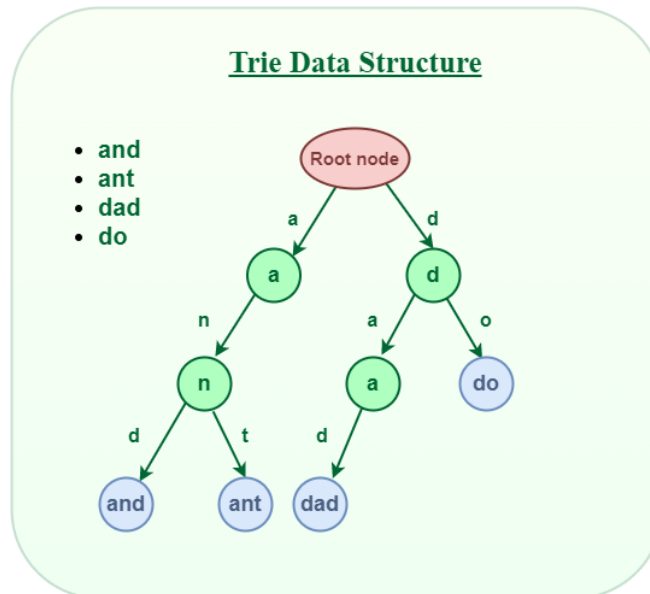
```

1 # ... 循环处理完所有字符 ...
2 # 单词处理完毕后，将当前节点的 word_count 增加1
3 current.word_count += 1

```

这里 `current.word_count += 1` 就等同于传统 `isEndOfWord = True` 的功能，并且还能记录重复插入的情况。

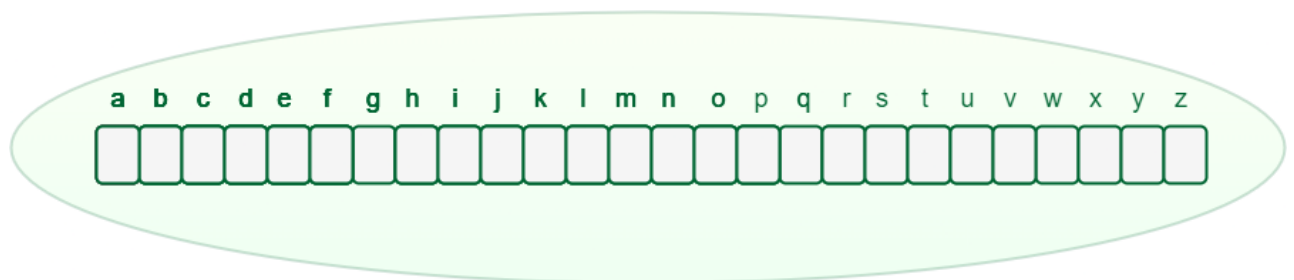
- **节点容量**：如果仅处理小写英文字母（a-z），则每个节点包含一个大小为 26 的指针数组。



Trie 数据结构是如何工作的？

我们已经知道，Trie 数据结构可以包含任意数量的字符，包括**字母**、**数字**和**特殊字符**。但在本文中，我们将讨论仅包含字符 **a-z** 的字符串。因此，每个节点只需要 26 个指针，其中第 **0** 索引表示字符 '**a**'，第 **25** 索引表示字符 '**z**'。

任何小写英文字母单词都可以以 **a-z** 开头，接着单词的下一个字母可以是 **a-z**，第三个字母也可以是 **a-z**，依此类推。因此，为了存储一个单词，我们需要一个大小为 **26** 的数组（容器），初始时所有字符都为空，因为还没有插入任何单词，它看起来如下所示：



An array of pointers inside every Trie node

让我们看看单词 **and** 和 **ant** 是如何存储在 Trie 数据结构中的：

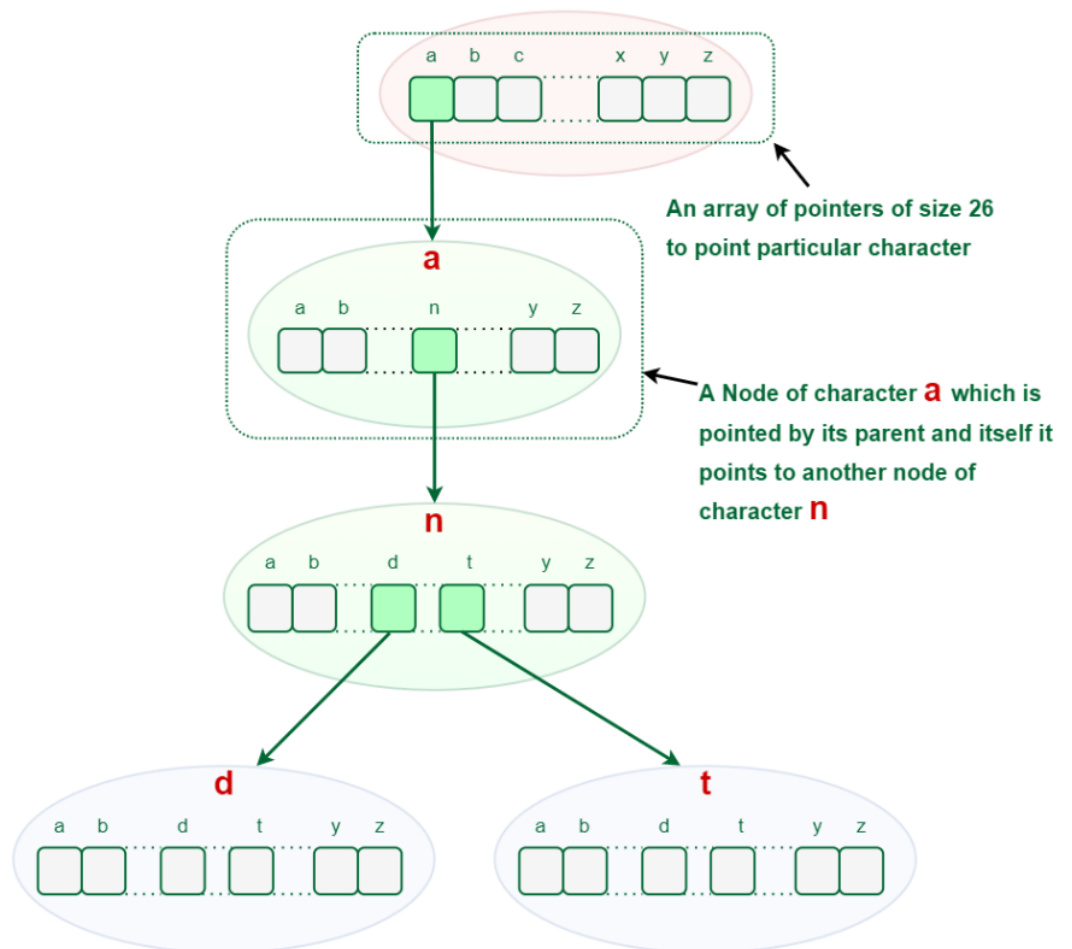
1. 在 Trie 数据结构中存储 **and**：
 - 单词 **and** 以字母 **a** 开头，因此我们在 Trie 节点中标记位置 **a** 为已占用，这表示使用了字符 **a**。
 - 放置第一个字符后，对于第二个字符仍然有 **26 种可能性**，因此从 **a** 出发，再次有一个大小为 **26** 的数组，用于存储第二个字符。
 - 第二个字符是 **n**，因此从 **a** 出发，我们移动到 **n** 并在第二个数组中标记 **n** 为已使用。

- 在 **n** 之后，第三个字符是 **d**，因此在相应的数组中标记位置 **d** 为已使用。

2. 在 Trie 数据结构中存储 **ant**：

- 单词 **ant** 以 **a** 开头，而根节点中 **a** 的位置已经被填充。因此，无需再次填充它，只需移动到 Trie 中的节点 **a**。
- 对于第二个字符 **n**，我们可以观察到在 **a** 节点中 **n** 的位置已经被填充。因此，无需再次填充，只需移动到 Trie 中的节点 **n**。
- 对于单词的最后一个字符 **t**，在 **n** 节点中 **t** 的位置尚未被填充。因此，填充 **n** 节点中 **t** 的位置，并移动到 **t** 节点。

在存储单词 **and** 和 **ant** 后，Trie 将如下所示：

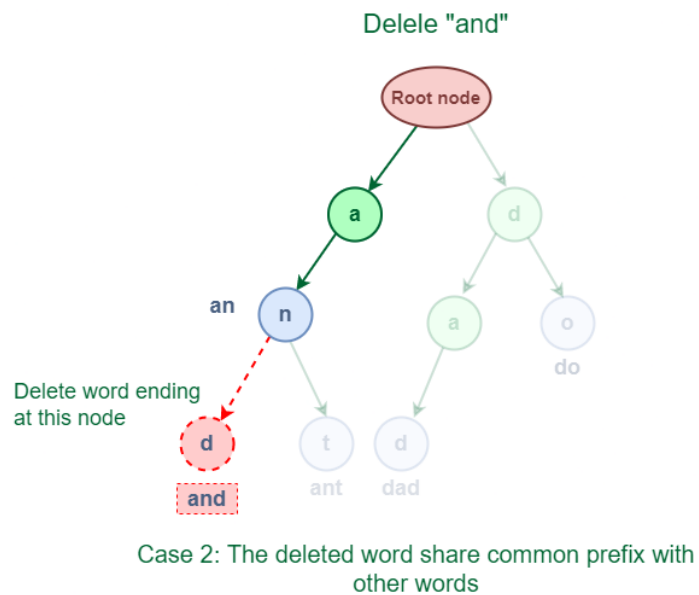


3. Trie 的基本操作

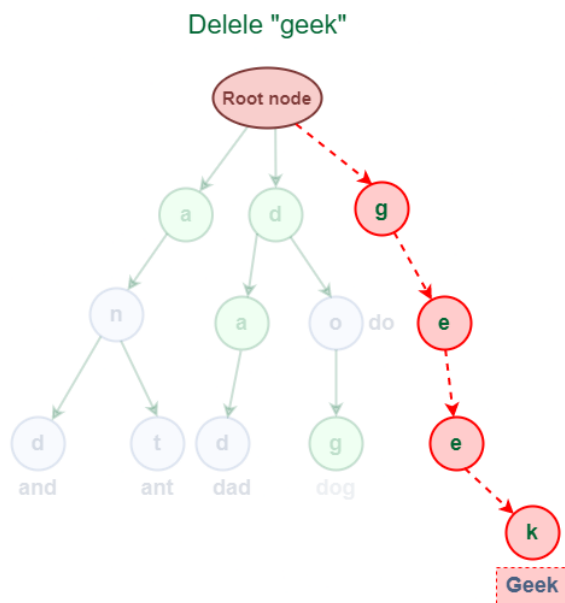
3.1 插入操作 (Insertion)

每插入一个单词，我们从根节点开始：

- 检查当前字符对应的子节点是否存在。
- 若不存在，创建一个新节点。
- 移动到对应的子节点，继续处理下一个字符。



The deleted word does not share any common prefix with other words in Trie.



Trie 的删除操作是整个数据结构中最难理解的部分，因为它涉及递归和回溯（Backtracking）。

通俗地说，删除一个单词不仅要把结尾的标志位去掉，还要从下往上检查：如果某些节点变得“没用了”（既不是其他单词的结尾，也没有子节点了），就应该把它们从内存中物理删除。

我们可以把这个递归函数拆解为三个阶段：向下寻找、逻辑删除、回溯清理。

1. 递归逻辑拆解

第一阶段：向下寻找 (Going Down)

```
1 char_idx = self._char_to_index(word[index])
2 node = current.children[char_idx]
3 if not node:
4     return False # 没找到这个字符，说明单词不存在，直接返回
```

程序会一直递归，直到找到单词的最后一个字符所在的节点。

第二阶段：到达终点与逻辑删除 (Base Case)

当我们到达单词的最后一个字符时：

```
1 if index == len(word):
2     if current.word_count == 0:
3         return False # 说明单词原本就不存在
4     current.word_count -= 1 # 逻辑删除：计数减一，现在这个路径不再代表这个单词了
5
6 # 判断是否要物理删除：
7 # 如果该节点没有子节点(all(child is None))，说明它是孤立的，返回 True 告诉父节点可以删我
8 return all(child is None for child in current.children)
```

第三阶段：向上回溯清理 (Coming Up / Backtracking)

这是最精妙的地方。当递归函数“返回”时，代码会从单词的最后一个字符往根节点方向逐层检查：

```
1 # 接收子节点的反馈：子节点是否说“我没用了，请删掉我”？
2 should_delete_child = self._delete_recursive(node, word, index + 1)
3
4 if should_delete_child:
5     current.children[char_idx] = None # 物理删除：父节点断开与子节点的连接
6
7 # 删完子节点后，父节点也要问自己：
8 # 我现在是不是也没用了？（不是其他单词结尾，且也没有其他孩子了）
9 return current.word_count == 0 and all(child is None for child in current.children)
```

2. 举个例子

假设 Trie 中存了两个词：“tea” 和 “ted”。

情况 A：删除 “ted”

1. 向下寻找：顺着 t -> e -> d 找到 d。
2. 终点：将 d 节点的 word_count 减为 0。
3. 判断：d 节点没有子节点。于是返回 True 给 e。
4. 回溯到 e：收到 True，于是执行 e.children['d'] = None。
5. e 问自己：我还是单词结尾吗？不是。我还有其他孩子吗？有（还有个 a）。
6. 结果：e 返回 False 给 t。清理停止。
7. 最终状态：删除了 d 节点，保留了 t -> e -> a。

情况 B：删除 “tea”（此时只剩 “tea”）

1. 向下寻找：到 a。

2. 终点: `a` 减为 0。`a` 没有孩子, 返回 `True` 给 `e`。
3. 回溯到 `e`: 删掉孩子 `a`。`e` 现在既不是结尾也没有孩子了, 返回 `True` 给 `t`。
4. 回溯到 `t`: 删掉孩子 `e`。`t` 现在既不是结尾也没有孩子了, 返回 `True` 给 `root`。
5. 结果: 整条路径 `t-e-a` 被物理清理干净。

3. 核心总结: 什么时候可以物理删除一个节点?

一个节点可以被物理删除 (`None`), 必须同时满足以下 2 个条件:

1. 没有子节点: 没有其他单词经过它。
2. 不是单词结尾: 它本身不代表一个完整的单词。

代码中的体现:

```
current.word_count == 0 and all(child is None for child in current.children)
```

为什么要用递归?

因为删除是从字符串末尾开始判断的。我们需要先“潜入”到最深处, 然后在“浮上来”的过程中顺手把没用的节点拆掉。如果不使用递归, 你就得手动记录路径上的所有节点, 处理起来会非常麻烦。

4. 代码实现 (Python)

以下是 Trie 的完整实现, 包括插入、搜索和前缀匹配。

```
1 class TrieNode:
2     def __init__(self):
3         # 26个小写字母的指针数组
4         self.children = [None] * 26
5         # 记录以此节点结尾的单词数量
6         self.word_count = 0
7
8 class Trie:
9     def __init__(self):
10        self.root = TrieNode()
11
12    def _char_to_index(self, ch):
13        """将字符映射到 0-25 的索引"""
14        return ord(ch) - ord('a')
15
16    def insert(self, word):
17        """插入单词"""
18        current = self.root
19        for char in word:
20            index = self._char_to_index(char)
21            if not current.children[index]:
22                current.children[index] = TrieNode()
23            current = current.children[index]
24        # 标记单词结束
25        current.word_count += 1
26
27    def search(self, word):
28        """搜索完整单词是否存在"""
29        current = self.root
```



```

30         for char in word:
31             index = self._char_to_index(char)
32             if not current.children[index]:
33                 return False
34             current = current.children[index]
35         return current.word_count > 0
36
37     def starts_with(self, prefix):
38         """检查是否存在以给定前缀开头的单词"""
39         current = self.root
40         for char in prefix:
41             index = self._char_to_index(char)
42             if not current.children[index]:
43                 return False
44             current = current.children[index]
45         return True
46
47     def delete(self, word):
48         """删除单词（简化逻辑版）"""
49         return self._delete_recursive(self.root, word, 0)
50
51     def _delete_recursive(self, current, word, index):
52         if index == len(word):
53             if current.word_count == 0:
54                 return False # 单词不存在
55             current.word_count -= 1
56             # 如果没有子节点，说明可以被父节点物理删除
57             return all(child is None for child in current.children)
58
59         char_idx = self._char_to_index(word[index])
60         node = current.children[char_idx]
61         if not node:
62             return False
63
64         should_delete_child = self._delete_recursive(node, word, index + 1)
65
66         if should_delete_child:
67             current.children[char_idx] = None
68             # 如果当前节点不再是单词结尾，且没有其他子节点，则继续向上删除
69             return current.word_count == 0 and all(child is None for child in current.children)
70
71         return False
72
73     # 测试代码
74     if __name__ == '__main__':
75         trie = Trie()
76         words = ["and", "ant", "do", "geek", "dad", "ball"]
77         for w in words:
78             trie.insert(w)
79
80         print(f"搜索 'do': {trie.search('do')}") # True
81         print(f"搜索 'bat': {trie.search('bat')}") # False
82         print(f"前缀 'an': {trie.starts_with('an')}") # True
83
84         trie.delete("geek")
85         print(f"删除 'geek' 后搜索: {trie.search('geek')}") # False

```

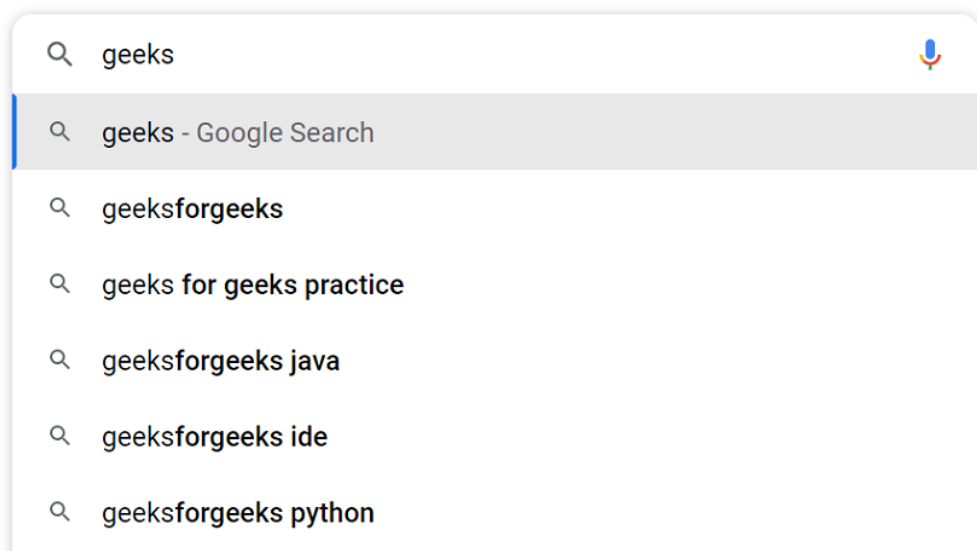
5. 复杂度分析

操作	时间复杂度	空间复杂度
插入	$O(L)$	$O(L \times N)$
搜索	$O(L)$	$O(1)$
删除	$O(L)$	$O(1)$

- L 是单词的长度。
- N 是插入单词的总数。
- **空间分析**：Trie 的主要缺点是内存消耗大。对于每个节点，无论是否用到，都需要分配一个固定大小的指针数组。在稀疏数据中，可以使用 `Hash Map` 代替数组来优化空间。

6. 应用场景

1. **自动补全 (Autocomplete)**：搜索引擎根据用户输入的前缀实时返回候选词。



2. **拼写检查器 (Spell Checkers)**：在字典中快速查找单词是否存在，并寻找相似单词。
3. **IP 路由匹配 (Longest Prefix Matching)**：在网络传输中，路由器根据 IP 地址的最长前缀来选择转发路径。
4. **敏感词过滤**：构建敏感词库，在文本流中高效扫描匹配。

7. 优缺点总结

- **优点**:
 - 查询速度极快且稳定，与数据规模无关，仅与单词长度有关。
 - 支持高效的前缀匹配和排序。
- **缺点**:
 - **内存消耗大**：每个节点都存储大量指针，可能导致很多内存碎片。
 - 若单词库非常稀疏且无公共前缀，哈希表的内存效率通常更高。

C kd 树 (K-Dimensional Tree)

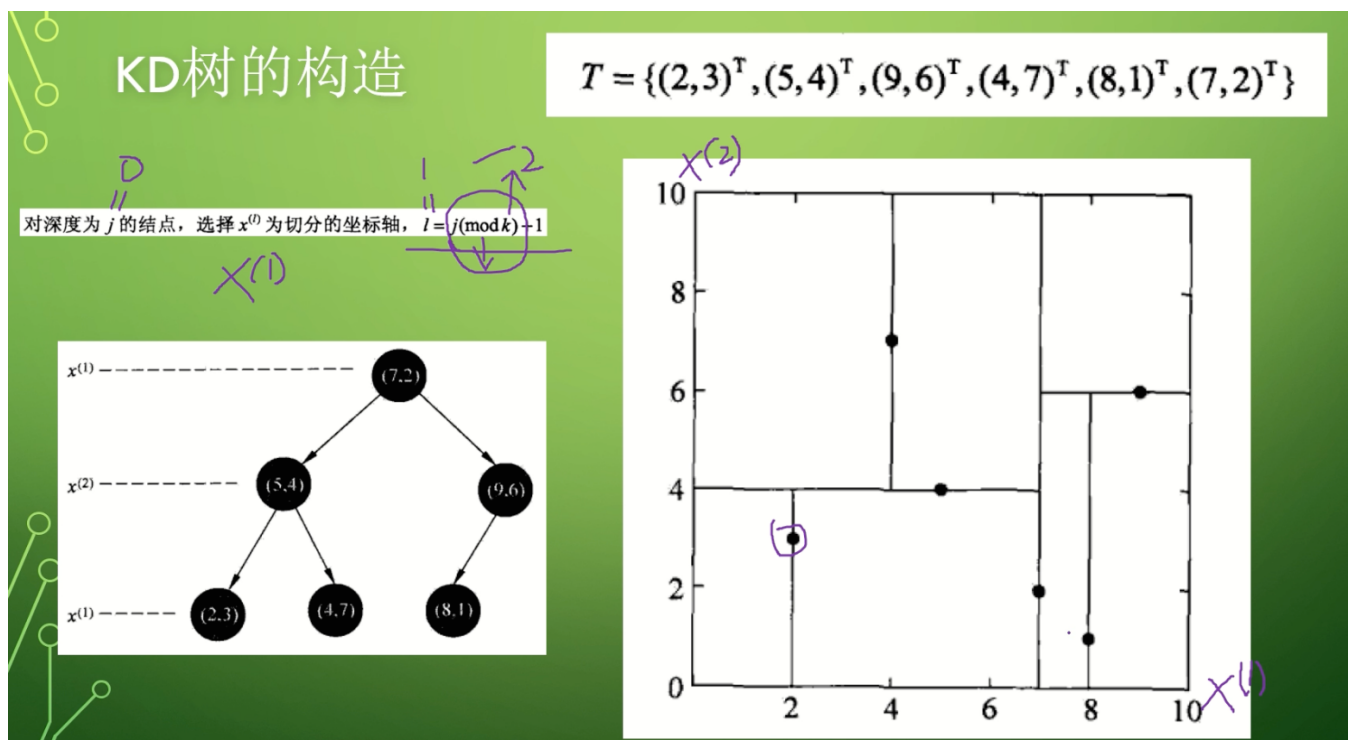
1. 什么是 kd 树?

kd 树是一种对 k 维空间中的实例点进行存储以便进行快速检索的二叉树数据结构。

- **核心思想**: 通过不断地用垂直于坐标轴的超平面将 k 维空间切分, 构成一系列的 k 维超矩形区域。
- **应用场景**: 主要用于多维空间的搜索, 如最近邻搜索 (Nearest Neighbor Search) 和范围搜索 (Range Search)。在 k 近邻算法 (k -NN) 中, 使用 kd 树可以显著提高检索效率。

1.1 构造算法要点

1. **选择切分维度**: 通常随深度循环选择坐标轴。例如: 深度为 j 的节点选择第 $l = (j \bmod k) + 1$ 个维度。
2. **选择切分点**: 选择该维度上的中位数作为切分点。这样得到的 kd 树是平衡的, 搜索效率接近 $O(\log N)$ 。
3. **递归构造**: 根据切分点将数据集划分为左右子集, 递归建立子树。



1.2 搜索算法要点 (最近邻)

1. **二分查找到叶子**: 从根开始, 根据目标点在切分维度的取值, 快速找到包含目标点的叶节点。
2. **回溯更新**: 从叶节点向上回溯, 计算当前节点与目标点的距离, 更新“当前最近点”。
3. **剪枝优化 (核心)**: 检查“以目标点为球心、当前最近距离为半径的超球体”是否与切分超平面相交。如果**不相交**, 则无需进入另一侧子树, 从而节省大量计算。

2. 代码实现 (Python)

2.1 构造与搜索实现

```

1 import numpy as np
2 from math import sqrt
3 from collections import namedtuple
4
5 # 定义结果元组
6 Result = namedtuple("Result", "nearest_point nearest_dist nodes_visited")
7
8 class KdNode:
9     def __init__(self, dom_elt, split, left, right):
10         self.dom_elt = dom_elt # 当前节点存储的点 (k维向量)
11         self.split = split # 切分维度的索引
12         self.left = left # 左子树
13         self.right = right # 右子树
14
15 class KdTree:
16     def __init__(self, data):
17         k = len(data[0]) if data else 0
18
19         def create_node(split, data_set):
20             if not data_set:
21                 return None
22
23             # 1. 按照指定维度排序并取中位数
24             data_set.sort(key=lambda x: x[split])
25             mid = len(data_set) // 2
26
27             split_next = (split + 1) % k
28             return KdNode(
29                 dom_elt=data_set[mid],
30                 split=split,
31                 left=create_node(split_next, data_set[:mid]),
32                 right=create_node(split_next, data_set[mid + 1:])
33             )
34
35         self.root = create_node(0, data)
36
37     def find_nearest(tree, target):
38         k = len(target)
39
40         def travel(node, target, best_res):
41             if node is None:
42                 return best_res
43
44             nodes_visited = 1
45             s = node.split
46             pivot = node.dom_elt
47
48             # 1. 递归向下查找“最近”的区域
49             if target[s] <= pivot[s]:
50                 nearer_node, further_node = node.left, node.right
51             else:
52                 nearer_node, further_node = node.right, node.left
53
54             # 先在更近的一侧搜索
55             best_res = travel(nearer_node, target, best_res)
56
57             current_nearest = best_res.nearest_point
58             current_dist = best_res.nearest_dist

```

```

59     nodes_visited += best_res.nodes_visited
60
61     # 2. 计算当前节点到目标点的欧氏距离
62     dist_to_pivot = sqrt(sum((x - y) ** 2 for x, y in zip(pivot, target)))
63     if dist_to_pivot < current_dist:
64         current_dist = dist_to_pivot
65         current_nearest = pivot
66
67     # 3. 剪枝: 检查超球体是否与另一侧区域相交
68     # 目标点在切分维度上到超平面的距离
69     dist_to_hyperplane = abs(pivot[s] - target[s])
70
71     if current_dist >= dist_to_hyperplane:
72         # 如果可能存在更近的点, 则进入另一侧子树
73         res_further = travel(further_node, target, Result(current_nearest, current_dist, 0))
74         nodes_visited += res_further.nodes_visited
75         if res_further.nearest_dist < current_dist:
76             current_nearest = res_further.nearest_point
77             current_dist = res_further.nearest_dist
78
79     return Result(current_nearest, current_dist, nodes_visited)
80
81     return travel(tree.root, target, Result(None, float('inf'), 0))
82
83 # 测试
84 data = [[2, 3], [5, 4], [9, 6], [4, 7], [8, 1], [7, 2]]
85 kd = KdTree(data)
86 res = find_nearest(kd, [3, 4.5])
87 print(f"最近点: {res.nearest_point}, 距离: {res.nearest_dist:.4f}")

```

3. 优化建议与注意事项

1. 切分维度的选择:

- 简单的做法是轮询: `split = depth % k`。
- 更高级的做法是选择方差最大的维度作为切分维度。方差越大, 说明点在该维度越分散, 切分后能更好地平衡空间。

2. 大数据量场景:

- 在构建 *kd* 树时, 频繁的排序 `data_set.sort()` 会导致复杂度变为 $O(k \cdot N \log^2 N)$ 。可以使用 **快速选择算法 (QuickSelect)** 来寻找中位数, 将构建复杂度优化到 $O(k \cdot N \log N)$ 。

3. 退化问题:

- 如果数据维度 *k* 非常大 (如 $k > 20$) , *kd* 树的效率会迅速下降, 甚至退化为全空间扫描 (即所谓的“维度之咒”)。在这种情况下, 通常使用 **LSH (局部敏感哈希)** 或 **球树 (Ball Tree)** 。

练习: M973.最接近原点的 K 个点

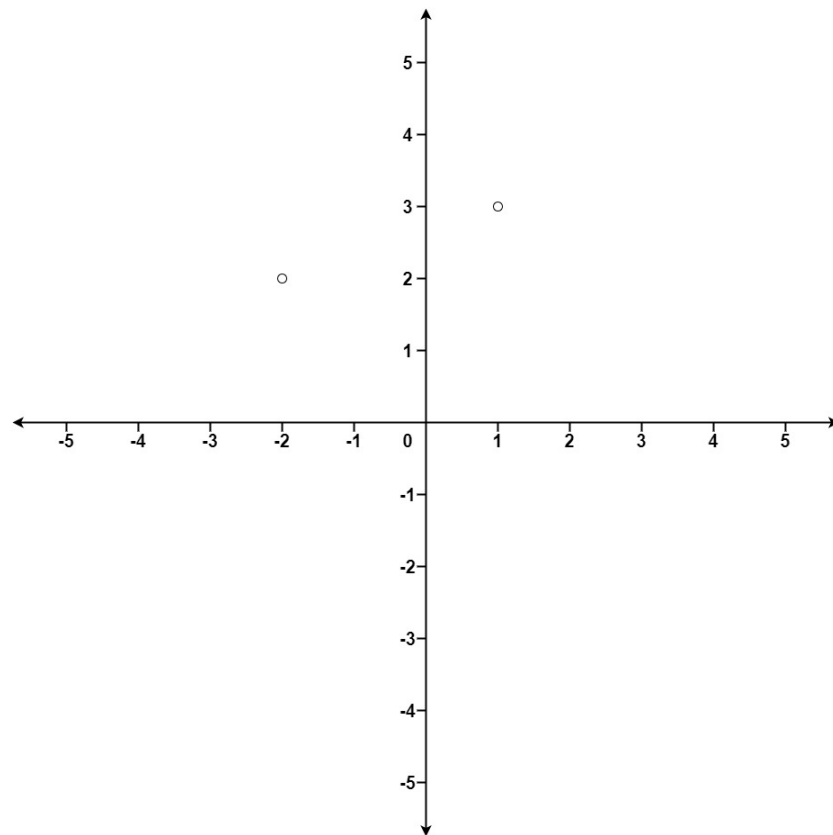
kd tree, heap, <https://leetcode.cn/problems/k-closest-points-to-origin/>

给定一个数组 `points` , 其中 `points[i] = [xi, yi]` 表示 X-Y 平面上的一个点, 并且是一个整数 `k` , 返回离原点 `(0,0)` 最近的 `k` 个点。

这里, 平面上两点之间的距离是 **欧几里德距离** ($\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$) 。

你可以按 **任何顺序** 返回答案。除了点坐标的顺序之外, 答案 **确保** 是 **唯一** 的。

示例 1:



```
1 输入: points = [[1,3],[-2,2]], k = 1
2 输出: [[-2,2]]
3 解释:
4 (1, 3) 和原点之间的距离为 sqrt(10),
5 (-2, 2) 和原点之间的距离为 sqrt(8),
6 由于 sqrt(8) < sqrt(10), (-2, 2) 离原点更近。
7 我们只需要距离原点最近的 k = 1 个点, 所以答案就是 [[-2,2]]。
```

示例 2:

```
1 输入: points = [[3,3],[5,-1],[-2,4]], k = 2
2 输出: [[3,3],[-2,4]]
3 (答案 [[-2,4],[3,3]] 也会被接受。)
```

提示:

- `1 <= k <= points.length <= 10^4`
- `-10^4 < xi, yi < 10^4`

- **题目大意:** 给定一组坐标, 求离原点最近的 K 个点。
- **KD树联系:** 这是标准的 k -NN 问题。虽然通常用大顶堆 (Heap) 解决, 但用 kd 树进行空间索引在数据量巨大且有多次查询时更有优势。

针对力扣 973 题，虽然最常用的解法是 **大顶堆 (Heap)** 或 **快速选择 (QuickSelect)**，但既然我们正在学习 *kd* 树，我将为你展示如何利用 *kd* 树来解决这个问题，并对比其他主流解法。

解法一：kd 树实现（空间索引法）

在 *kd* 树中找 K 个最近邻，与找 1 个最近邻的区别在于：我们需要维护一个大小为 K 的**大顶堆**，用来保存当前找到的最近的 K 个点。

```
1 import heapq
2
3 class KdNode:
4     def __init__(self, point, split, left=None, right=None):
5         self.point = point
6         self.split = split
7         self.left = left
8         self.right = right
9
10 class Solution:
11     def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
12         # 1. 构建 KD 树
13         def build_tree(data, depth):
14             if not data:
15                 return None
16             axis = depth % 2 # 2维空间，只有 0 和 1
17             data.sort(key=lambda x: x[axis])
18             mid = len(data) // 2
19             return KdNode(
20                 point=data[mid],
21                 split=axis,
22                 left=build_tree(data[:mid], depth + 1),
23                 right=build_tree(data[mid+1:], depth + 1)
24             )
25
26         root = build_tree(points, 0)
27
28         # 2. 搜索 K 个最近邻
29         # heap 中存储的是 (-距离, 点)，因为 Python 的 heapq 是小顶堆，
30         # 用负数可以模拟大顶堆，方便弹出距离最远的点
31         self.max_heap = []
32
33         def get_dist(p):
34             return p[0]**2 + p[1]**2 # 省略开方以优化性能
35
36         def search(node):
37             if not node:
38                 return
39
40             dist = get_dist(node.point)
41             axis = node.split
42
43             # 维护大小为 k 的大顶堆
44             if len(self.max_heap) < k:
45                 heapq.heappush(self.max_heap, (-dist, node.point))
46             elif dist < -self.max_heap[0][0]:
47                 heapq.heapreplace(self.max_heap, (-dist, node.point))
48
49             # 决定搜索方向
```

```

50         target_val = 0 # 因为是离原点(0,0)的距离, 目标点值永远是0
51         if target_val < node.point[axis]:
52             near, far = node.left, node.right
53         else:
54             near, far = node.right, node.left
55
56         search(near)
57
58         # 剪枝: 如果另一侧还可能存在更近的点, 则进入另一侧
59         # 这里的“墙”是 node.point[axis], 目标点在 0
60         # 距离墙的平方距离是 (node.point[axis] - 0)^2
61         if len(self.max_heap) < k or (node.point[axis]**2 < -self.max_heap[0][0]):
62             search(far)
63
64     search(root)
65     return [p for d, p in self.max_heap]

```

解法二：大顶堆（最稳健解法）

这是面试中最推荐的解法。遍历所有点，维护一个大小为 K 的大顶堆。时间复杂度 $O(N \log K)$ 。

```

1  import heapq
2
3  class Solution:
4      def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
5          heap = [] # 大顶堆
6          for x, y in points:
7              dist = -(x**2 + y**2) # 取负数变大顶堆
8              if len(heap) < k:
9                  heapq.heappush(heap, (dist, x, y))
10             elif dist > heap[0][0]:
11                 heapq.heapreplace(heap, (dist, x, y))
12
13         return [[x, y] for d, x, y in heap]

```

解法三：快速选择（QuickSelect, 理论最优）

类似快排的分治思想，不需要完全排序，只需把最小的 K 个数放到数组左边。时间复杂度平均 $O(N)$ 。

```

1  import random
2
3  class Solution:
4      def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
5          def get_dist(p):
6              return p[0]**2 + p[1]**2
7
8          def quick_select(left, right, k):
9              if left >= right: return
10
11             pivot_idx = random.randint(left, right)
12             points[left], points[pivot_idx] = points[pivot_idx], points[left]
13
14             pivot_dist = get_dist(points[left])
15             i = left
16             for j in range(left + 1, right + 1):

```



```

17         if get_dist(points[j]) < pivot_dist:
18             i += 1
19             points[i], points[j] = points[j], points[i]
20
21     points[left], points[i] = points[i], points[left]
22
23     # i 是 pivot 最终所在的位置
24     num_left = i - left + 1
25     if k < num_left:
26         quick_select(left, i - 1, k)
27     elif k > num_left:
28         quick_select(i + 1, right, k - num_left)
29
30     quick_select(0, len(points) - 1, k)
31     return points[:k]

```

总结与对比

方法	时间复杂度	优点	缺点
排序	$O(N \log N)$	代码最简单。	性能一般，处理了不必要的排序。
大顶堆	$O(N \log K)$	适合流式数据（海量数据无法一次存入内存）。	需要额外 $O(K)$ 空间。
快速选择	$O(N)$	理论最快。	稍微复杂，若基准点选不好可能退化。
<i>kd</i> 树	建树 $O(N \log N)$ 查询 $O(K \log N)$	多次查询时极快。适合空间检索。	仅单次查询时，建树开销太大。

针对力扣这道题：

- 如果你只需要查询一次，请使用快速选择或大顶堆。
- 如果你有一个包含 100 万个点的地图，用户会频繁点击不同位置查询最近的点，请务必使用 *kd* 树。

力扣（LeetCode）相关题目

在力扣中，很少有题目强制要求手写 *kd* 树，但很多几何空间检索、最近邻、二维区域查询类的题目，使用 *kd* 树的思想是最高效的解法。

1. [973. 最接近原点的 K 个点](#) (Medium)

- 题目大意：给定一组坐标，求离原点最近的 K 个点。
- **KD树联系**：这是标准的 k -NN 问题。虽然通常用大顶堆（Heap）解决，但用 *kd* 树进行空间索引在数据量巨大且有多次查询时更有优势。

2. [742. 二叉树最近的叶节点](#) (Medium)

- 题目大意：在树中找到离给定节点最近的叶子节点。
- **KD树联系**：此题虽然是普通二叉树，但搜索逻辑（向下递归+向上回溯）与 *kd* 树的最近邻搜索非常相似。

3. [1200. 最小绝对差](#) (Easy)

- 题目大意：找到数组中所有差值绝对值最小的元素对。

- **KD树联系**：这是一维空间的最近邻对问题。*kd* 树在一维情况下退化为二叉搜索树（BST）。
- 4. [LCP 21. 追逐游戏](#) (Hard)
- **题目大意**：涉及图上节点距离的计算。
- **KD树联系**：复杂的空间位置关系题目，理解 *kd* 树的剪枝思想对解决这类最短路径/距离剪枝问题有很大启发。

附录

D 树这章程序对应类图

D.1 生成类图

https://github.com/Yuqiu-Yang/problem_solving_with_algorithms_and_data_structures_using_python

下载后，到ch6目录，生成类图。

<https://stackoverflow.com/questions/260165/whats-the-best-way-to-generate-a-uml-diagram-from-python-source-code>

You may have heard of [Pylint](#) that helps statically checking Python code. Few people know that it comes with a tool named [Pyreverse](#) that draws UML diagrams from the Python code it reads. Pyreverse uses Graphviz as a backend.

It is used like this:

```
1 | pyreverse -o png -p yourpackage .
```

where the `.` can also be a single file.

Generating UML Diagrams

<https://www.bhavaniravi.com/python/generate-uml-diagrams-from-python-code>

brew install pylint

brew install Graphviz

在 ch6 目录下运行

```
% pyreverse -o png *.py
```

Format png is not supported natively. Pyreverse will try to generate it using Graphviz...

Analysed 12 modules with a total of 6 imports

产生俩文件

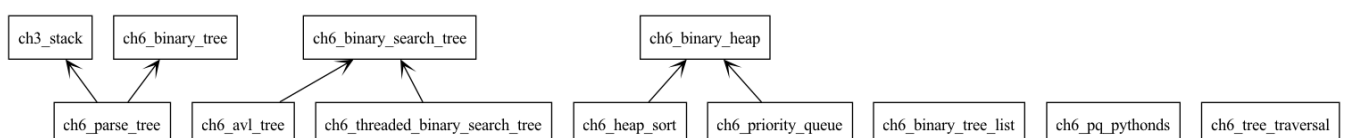


图 packages.png

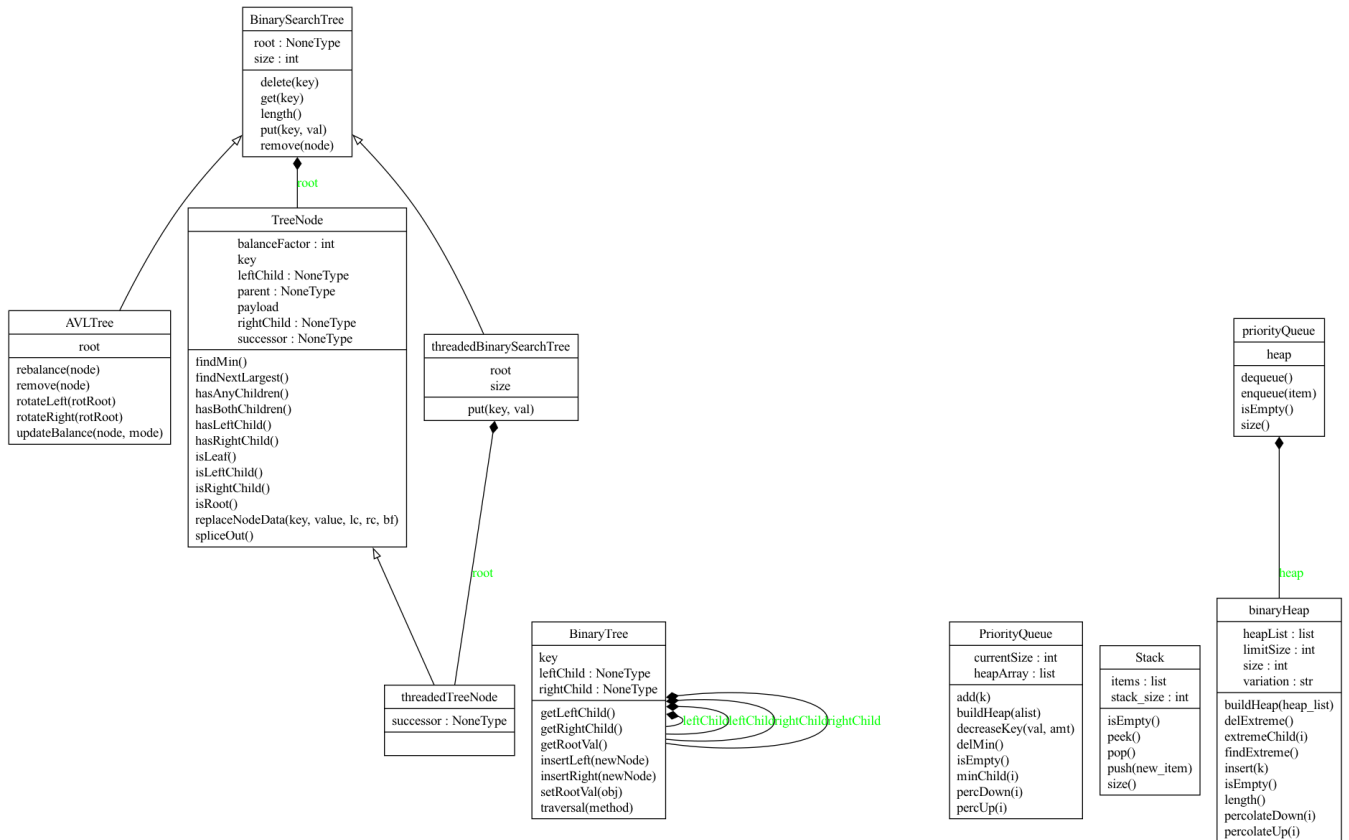
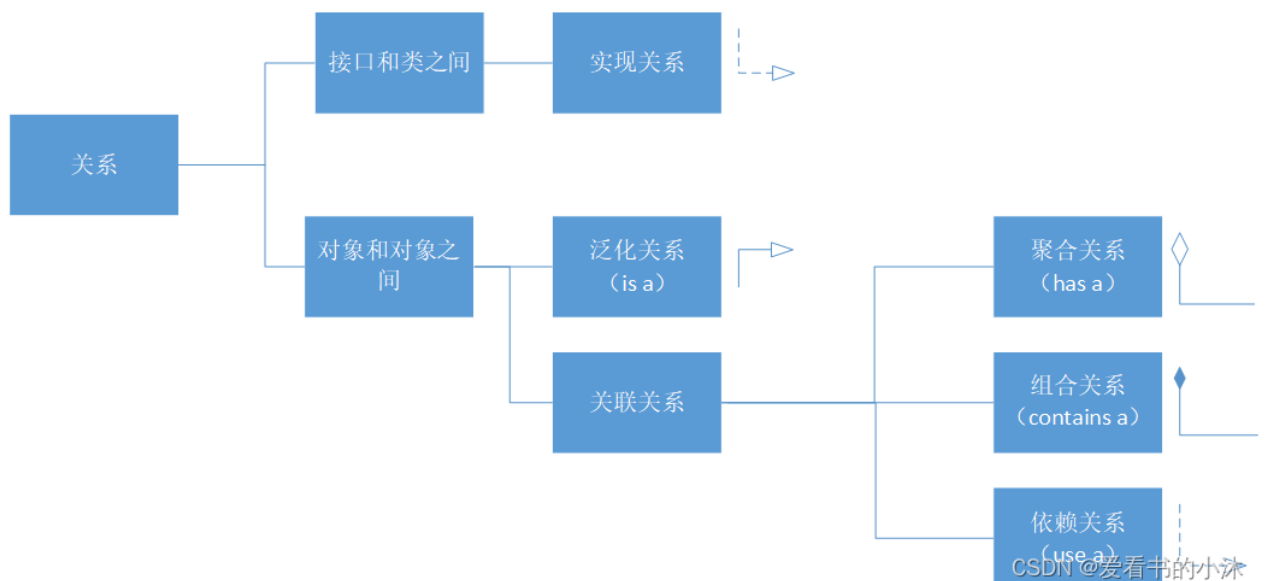


图 classes.png

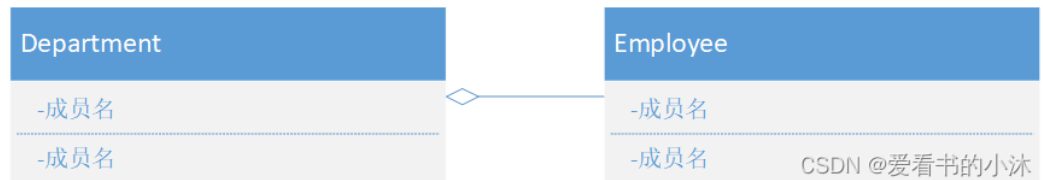
D.2 在UML类图中，常见的连线 and 符号



在UML（统一建模语言）类图中，常见的连线和符号包括以下这些：

1. 关联(Association)

- 普通关联：用一条直线表示两个类之间的关系，通常在关联的两端可以有箭头，箭头指向被关联的类，表示导航方向。
- 双向关联：一条简单的线，表示两个类相互知道对方。
- 聚合关系(Aggregation)：用一个空心的菱形加一条线来表示，空心菱形位于整体的一端。表示一个类是另一个类的部分，但它们之间没有强耦合关系，部分可以存在独立于整体的情况。菱形指向整体类。例如：

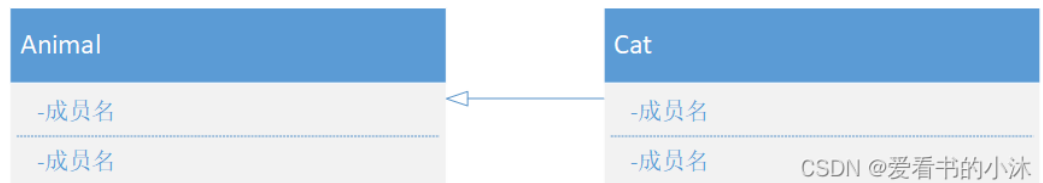


- 组合关系(Composition)：用一个实心的菱形加一条线来表示，实心菱形位于整体的一端。表示一个类是另一个类的整体部分，它们具有生命周期上的整体关系。菱形指向整体类。例如：



2. 泛化(Generalization)

- 用一条带有空心箭头的直线表示，箭头指向父类，表示子类继承自父类。表示继承关系，一个类是另一个类的子类，继承了父类的属性和方法。例如：



3. 实现(Implementation)

- 用一条带有空心箭头的虚线表示，箭头指向接口，表示类实现了接口。

4. 依赖(Dependency)

- 用一条带有箭头的虚线表示，箭头指向被依赖的类。

在类图中，类通常用带有三个部分的矩形来表示：

- 顶部部分：显示类名，如果是抽象类，则用斜体表示。
- 中间部分：显示类的属性或字段。
- 底部部分：显示类的方法或操作。

还有一些其他的符号和约定，比如表示多重性的数字（例如，1...* 表示一个到多个），以及用来表示接口、抽象类等的特殊图标。在类图中，你也可以使用注释框（用一条虚线连接的矩形框）来添加对关系或类的额外说明。

参考

<https://runestone.academy/ns/books/published/pythonds3/index.html>

https://github.com/Yuqiu-Yang/problem_solving_with_algorithms_and_data_structures_using_python

<https://github.com/wesleyjtann/Problem-Solving-with-Algorithms-and-Data-Structures-Using-Python>

【小沐学Python】UML类图的箭头连线关系总结 (python+graphviz)

<https://blog.csdn.net/hhy321/article/details/132651062>